BASES DE DONNES

- Généralités
- Les bases de données relationnelles
- Normalisation des relations
- Le Modèle entité association
- L'algèbre relationnel
- Le langage SQL
- Sécurité des bases de données

Par GBO Hervé ***Tél*** : 95564920 /97188083 ***e-mail*** : hervegbo@yahoo.fr

CHAPITRE I : GENERALITES

I - INTRODUCTION

Dans les formes traditionnelles de l'information, les données stockées sur des supports constituent des fichiers. Ces fichiers, construits pour répondre aux besoins des applications informatiques possèdent la plupart du temps des éléments communs, des relations qui ne sont pas exploitées du fait qu'ils sont utilisés isolement et indépendamment les uns des autres. Cette manière d'organiser les informations présente trois inconvénients :

- Une grande complexité à cause du grand nombre de fichiers et à leur création qui dépend des besoins particuliers de chaque application.
- La redondance des informations à cause de la duplication sur plusieurs fichiers de certaines données. Ce qui est d'une part coûteux en moyen de stockage et d'autre part pose le problème de cohérence car les mises à jour d'une même donnée localisée sur plusieurs fichiers utilisés par des applications différentes, ne peut pas toujours être simultanée.

Par ailleurs, la prise de décision est une part importante de la vie d'une société. Mais elle nécessite d'être bien informée sur la situation et donc d'avoir des informations à jour et disponibles immédiatement.

D'autre part, les utilisateurs, ne veulent plus de systèmes d'information constitués d'un ensemble de programmes inflexibles et de données inaccessibles à tout non spécialiste; ils souhaitent des systèmes d'informations globaux, cohérents, directement accessibles (sans qu'ils aient besoin soit d'écrire des programmes soit de demander à un programmeur de les écrire pour eux) et des réponses immédiates aux questions qu'ils posent. Il a donc fallu rechercher des solutions tenant compte à la fois des désirs des utilisateurs et des progrès techniques. Cette recherche a abouti au concept de base de données.

II - DEFINITIONS ET AVANTAGES D'UNE BASE DE DONNEE

Définition

Une base de données est un ensemble d'informations qui modélise les objets d'une partie du monde réel, qui peut servir de support à une application informatique et qui est exhaustif, non redondant, structuré et persistant.

- ***La partie du monde réel*** est le sujet étudié ou le domaine circonscrit par la base de données. exemple : Gestion de la scolarité des étudiants, Gestion des patients d'un hôpital, etc.
- On appelle ***objets d'une partie du monde réel***, toute entité, ayant un sens pour le sujet étudié, qui peut lui fournir des informations et qui possède au moins deux occurrences (ou exemplaires). On distingue des objets abstraits et des objets réels. Exemple, dans le cas de la gestion de la scolarité des étudiants on distingue les objets réels suivants : Etudiants, Profs, Salle de classe, etc. et les objets abstraits suivants : Matière, Filière, etc.
- ***Modéliser (dessiner, structurer, schématiser) les objets d'une partie du monde réel*** revient à organiser les données suivants les différentes entités et à définir les liens entre ces entités.

Avantages

Les avantages que l'on peut attendre d'une base de données sont nombreux :

- *Saisie unique* : Les données sont saisies une seule fois et stockées indépendamment des programmes qui doivent les traiter.
- *Mise à jour unique* : Comme une donnée n'est stockée qu'une seule fois, le problème souvent mal résolu de la mise à jour de cette donnée dans différents fichiers ne se pose plus. Les différents traitements qui utiliseront cette donnée seront toujours très cohérents.

- *Gain de place au stockage* : La suppression des redondances réduit le volume nécessaire au stockage.
- *Accès plus facile à l'information* : Chaque donnée est stockée indépendamment des programmes qui l'utilisent. Elle est disponible pour les utilisateurs potentiels qui interrogeront la base. L'ensemble des informations de l'Entreprise peut ainsi être partagé entre les services sans qu'aucun n'en conserve l'exclusivité.
- *Possibilité d'évolution* : Une base de donnée bien conçue est indépendante des traitements. Elle peut en principe permettre la prise en compte facile de nouvelles applications si les possibilités d'accès aux données ont été bien prévues lors de sa conception.

III - LES SGBD

Un système de gestion de base de données est un logiciel qui permet de décrire, modifier, interroger et administrer les données d'une base de données.

1. Objectifs des systèmes de gestion de bases de données

Les bases de données et les systèmes de gestion de bases de données ont été créés pour répondre à un certain nombre de besoins et pour résoudre un certain nombre de problèmes.
Les objectifs d'un SGBD sont les suivants :

- ***Indépendance physique***
  La façon dont les données sont définies doit être indépendante des structures de stockages utilisées.
- ***Indépendance logique***
  Un même ensemble de données peut être vu différemment par des utilisateurs différents. Toutes ces visions personnelles des données doivent être intégrées dans une vision globale.
- ***Facilité d'utilisation et Manipulations des données par des non informaticiens***
  Les SGBD offrent des interfaces conviviales et des outils de gestion pour faciliter la manipulation et l'administration des données.
  Il faut pouvoir accéder aux données sans savoir programmer ce qui signifie des langages "quasi naturels" comme SQL.
- ***Stockage efficace des données*** : Les SGBD sont conçus pour stocker de grandes quantités de données de manière efficace, en minimisant l'espace de stockage nécessaire et en optimisant les performances d'accès aux données.
- ***Efficacité des accès aux données***
  Les SGBD sont conçus pour permettre un accès rapide et efficace aux données, en utilisant des techniques d'indexation et d'optimisation des requêtes.
  Les langages de requête comme SQL doivent permettre d'obtenir des réponses aux interrogations en un temps "raisonnable". Ils doivent donc être optimisés et, entre autres, il faut un mécanisme permettant de minimiser le nombre d'accès disques. Tout ceci, bien sûr, de façon complètement transparente pour l'utilisateur.
- ***Administration centralisée des données***
  Des visions différentes des données (entre autres) se résolvent plus facilement si les données sont administrées de façon centralisée.
- ***Non redondance des données***
  Afin d'éviter les problèmes lors des mises à jour, chaque donnée ne doit être présente qu'une seule fois dans la base.
- ***Cohérence et intégrité des données***
  Les SGBD garantissent l'intégrité des données en appliquant des contraintes et des règles de validation pour s'assurer que les données stockées sont cohérentes et précises.

Les données sont soumises à un certain nombre de contraintes d'intégrité qui définissent un état cohérent de la base. Elles doivent pouvoir être exprimées simplement et vérifiées automatiquement à chaque insertion, modification ou suppression des données.

- ***Accès concurrent aux données***
  Il s'agit de permettre à plusieurs utilisateurs d'accéder aux mêmes données au même moment. Si ce problème est simple à résoudre quand il s'agit uniquement d'interrogations et quand on est dans un contexte mono utilisateur, cela n'est plus le cas quand il s'agit de modifications dans un contexte multi utilisateurs. Il s'agit alors de pouvoir :

  - permettre à deux (ou plus) utilisateurs de modifier la même donnée "en même temps" ;
  - assurer un résultat d'interrogation cohérent pour un utilisateur consultant une table pendant qu'un autre la modifie.

- ***Sécurité des données***
  Les données doivent pouvoir être protégées contre les accès non autorisés. Pour cela, il faut pouvoir associer à chaque utilisateur des droits d'accès aux données.
- ***Résistance aux pannes***
  Que se passe-t-il si une panne survient au milieu d'une modification, si certains fichiers contenant les données deviennent illisibles? Les pannes, bien qu'étant assez rares, se produisent quand même de temps en temps. Il faut pouvoir, lorsque l'une d'elles arrive, récupérer une base dans un état "sain". Ainsi, après une panne intervenant au milieu d'une modification deux solutions sont possibles : soit récupérer les données dans l'état dans lequel elles étaient avant la modification, soit terminer l'opération interrompue.

Il existe des SGBD de complexité variable qui possèdent tout ou partie des propriétés ci-dessus. Prenons en exemple deux produits assez caractéristiques : le SGBD relationnel Oracle et le SGBD relationnel Access. Le SGBD Oracle est un SGBD relationnel utilisé pour des applications critiques et qui offre un maximum des caractéristiques présentées ici. Le SGBD Access est un SGBD dans le monde de l'informatique individuelle qui présente l'avantage d'une grande facilité d'utilisation et qui peut convenir à des applications de taille réduite ou moyenne. L'aspect convivial de ce dernier étant évident. En revanche, les niveaux de performance et de sécurité ne sont pas comparables.

2. Architecture des SGBD

Les SGBD reposent sur trois niveaux d'abstraction qui assurent l'indépendance logique et physique des données, autorisent la manipulation de données, garantissent l'intégrité des données et optimisent l'accès aux données. L'architecture d'un SGBD a trois niveaux :

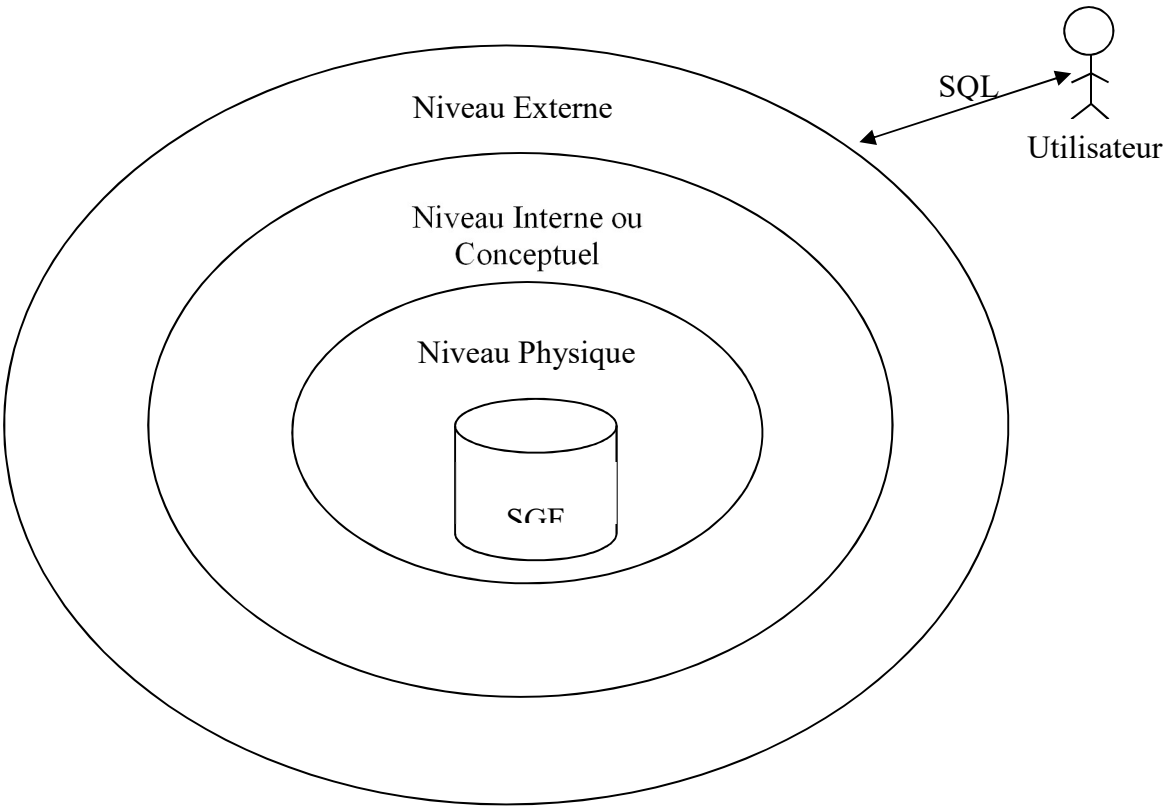
- ***Le niveau externe.***
  Il regroupe toutes les possibilités d'accès aux données par les différents usagers. Ces accès, éventuellement distants, peuvent se faire via différents types d'interfaces et langages plus ou moins élaborés. Ce niveau détermine le schéma externe qui contient les vues des utilisateurs sur la base de données c'est à dire le sous-ensemble de données accessibles ainsi que certains assemblages d'information et éventuellement des informations calculées. Il peut donc exister plusieurs schémas externes représentant différentes vues sur la base de données avec des possibilités de recouvrement.

- ***Le niveau conceptuel.***
  Il correspond à la vision des données générale indépendante des applications individuelles et de la façon dont les données sont stockées. Cette représentation est en adéquation avec le modèle de

données utilisé. Dans le cas des SGBD relationnels, il s'agit d'une vision tabulaire où la sémantique de l'information est exprimée en utilisant les concepts de relation, attributs et de contraintes d'intégrité. Le niveau conceptuel est défini au travers du schéma conceptuel.

- ***Le niveau physique.***

Il regroupe les services de gestion de la mémoire secondaire. Il s'appuie sur un système de gestion de fichiers pour définir la politique de stockage ainsi que le placement des données. Cette politique est définie en fonction des volumes de données traitées, des relations sémantiques entre les données ainsi qu'en fonction de l'environnement matériel disponible.

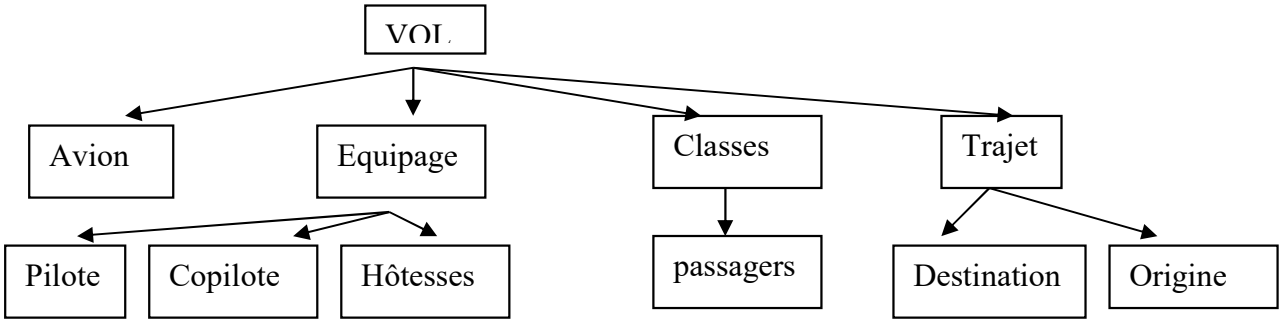


IV- LES DIFFERENTS MODELES D'UNE BASE DE DONNEES

Le principal objectif des bases de données est de rendre indépendant les données vis à vis des applications. L'accès à ces données par des applications pose des problèmes. Ainsi, pour simplifier ces problèmes d'accès, plusieurs modèles logiques de base de données et de systèmes de gestion de base de donnée ont vu le jour. On distingue par ordre chronologique :

1 - Le modèle hiérarchique

Historiquement le premier, il consiste à organiser les données de façon arborescente; ce qui constitue, une structure simple à gérer. Cette structure est une hiérarchie ou chaque élément n'a qu'un supérieur. Le nombre de connexion est limité: il n'y en a pas entre les branches de même niveau.



Ce modèle ne permet que des interrogations simples. Par exemple, quel est le trajet du vol 512 ou quel est le pilote du vol 304. Mais, il n'est pas aisé de savoir sur quel vol est inscrit le passager Mahigôh.

2 - Le Modèle Réseau

Le modèle réseau est une extension du modèle précédent: il permet d'établir des connexions entre les différents éléments. De cette manière, on dispose d'un plus grand nombre d'interrogations possibles mais, elles doivent être toujours prévues lors de la construction de la base de données.

3- Le Modèle Relationnel

Le modèle relationnel Permet de se libérer de la contrainte suivante : Connaître à l'avance les interrogations que l'on effectuera. Ainsi, les données sont stockées sous la forme de relation dans des tables. Ce type de structure permet d'établir des connexions au moment de l'exécution. On pourra donc effectuer toute sorte d'interrogations plus ou moins complexes.
L'accès aux bases de données relationnelles s'effectue en appliquant les trois opérations de base suivantes: la projection, la sélection et la jointure .
Avec le modèle relationnel, il est nécessaire de convertir les données d'application sous forme de tables. L'interface entre une application et une base de données est effectuée à l'aide de langages spécifiques, dont le plus connu est SQL

Chapitre II – Les bases de données relationnelles

Le modèle relationnel a été proposé par E.F. Codd en 1970. Il est souvent considéré comme le plus simple et le plus élégant des modèles. Sa simplicité est due à une vision tabulaire des données très intuitive. En effet, dans ce modèle, les données sont stockées dans des tables, sans préjuger de la façon dont les informations sont stockées dans la machine. Un ensemble de données sera donc modélisé par un ensemble de tables ou relations.
Le succès du modèle relationnel auprès des chercheurs, concepteurs et utilisateurs est dû à la puissance et à la simplicité de ses concepts. En outre, contrairement à certains autres modèles, il repose sur des bases théoriques solides, notamment la théorie des ensembles.

Les objectifs du modèle relationnel :

- proposer des schémas de données faciles à utiliser,
- améliorer l'indépendance logique et physique,
- mettre à la disposition des utilisateurs des langages de haut niveau pouvant éventuellement être utilisés par des non informaticiens (SQL),
- optimiser les accès à la base de données,
- améliorer l'intégrité et la confidentialité,
- fournir une approche méthodologique dans la construction des schémas,
- Permettre un haut degré d'indépendance entre les applications (programmes, interfaces) et la représentation interne des données (fichiers, chemins d'accès)
- Etablir une base solide pour traiter les problèmes de cohérence et de redondance des données

De façon informelle, on peut définir une base de données relationnelle comme étant une base dans laquelle

- Les données sont organisées sous forme de tableaux à deux dimensions, encore appelées relations et chaque ligne un uplet, une occurrence, un enregistrement ou un tuple,
- les données sont manipulées par des opérateurs de l'algèbre relationnelle,
- l'état cohérent de la base est défini par un ensemble de contraintes d'intégrité.

Le modèle relationnel présente également de nombreux avantages dus au fait qu'il soit basé sur la théorie des ensembles : Langage de manipulation des données ensemblistes grâce à l'algèbre relationnelle et grâce à des langages assertionnels qui permettent de spécifier ce que l'on souhaite obtenir sans dire comment l'obtenir. Le SGBD est responsable de la politique d'exécution des requêtes.

Au modèle relationnel est associée la théorie de la normalisation des relations qui permet de se débarrasser des incohérences au moment de la conception d'une base de données.

I. *Concepts de base*

Attribut **:** C'est la plus petite information (atomique) qui a un sens pour le domaine d'étude.

Relation **:** c'est un ensemble d'attributs qui caractérisent un individu (Etudiant), un objet (Filière) ou un évènement (Composer). Elle peut être considérée comme un tableau dans lequel chaque colonne correspond à un attribut et porte un nom et chaque ligne représente un enregistrement, tuple, uplet ou une occurrence.

Schéma de relation : Un schéma de relation R, est utilisé pour décrire une relation et est noté :
R(A1, A2, …, An) ou R est le nom de la relation et Ai les attributs qui décrivent la relation.

Illustration ou extension d'une relation : c'est la représentation d'une relation sous forme de tableau avec comme entête de colonne, les attributs et comme ligne, les occurrences de la relation. Exemple si on considère la relation Etudiant de schéma :

Etudiant(Matricule, nom, prénom, sexe, codfil)

Une extension de cette relation serait le tableau suivant :

Matricule	Nom	Prénom	Sexe	Codfil
1	Magengo	Guttembert	M	IG
2	Jolie	Joliette	F	MMI
3	Mahigôh	Bitôh	M	IIR
4	Gros	Séchard	M	IIR

Clé primaire d'une relation **: c'**est un ensemble minimum d'attributs dont la connaissance des valeurs permet d'identifier un enregistrement unique de la relation considérée. Une clé primaire est invariante dans le temps. Lors de la définition d'un schéma cette clé primaire est soulignée.

Clés secondaires, externes ou étrangères d'une relation **:** ce sont des attributs qui sont clé primaire de relation et qui se retrouvent dans d'autres relations permettant ainsi d'établir un lien entre les relations. Lors de la définition d'un schéma ces clés étrangères sont précédées du symbole #.

Base de données relationnelles **:** Base de données dont le schéma est un ensemble de schémas de relations normalisés (en troisième forme normale) et dont les occurrences sont les tuples de ces relations.

Système de gestion de bases de données relationnelles **:** C'est un logiciel supportant le modèle relationnel, et qui peut manipuler les données avec des opérateurs relationnels.

Les contraintes d'intégrité : Une contrainte d'intégrité est une propriété, invariante dans le temps qui permet de contrôler l'intégrité des données.
Il existe différents types de contraintes d'intégrité:

- les contraintes liées au modèle (pas de doublons dans une relation, etc.);
- les contraintes de domaine (nb_heure < 100; Note d'un étudiant comprise entre 0 et 20);
- les contraintes référentielles dites de clé étrangère qui imposent que la valeur d'attribut de la relation r_1 apparaissent comme valeur de clé dans une autre relation r_2.

II. - ***Normalisation des relations***

Pour éviter des redondances, pour faciliter les modifications, pour obtenir des structures de données plus claires, il est conseillé de normaliser les relations. La normalisation consistera à décomposer une relation sans perte d'informations, à partir de la notion de dépendance fonctionnelle. L'objectif de la normalisation est d'aboutir à un schéma conceptuel représentant les entités et les associations canoniques du monde réel.

1. Les dépendances fonctionnelles

a. Définition et notation

- On dit qu'un attribut A est en dépendance fonctionnelle avec un autre attribut B (Ou un attribut B dépend fonctionnellement d'un attribut A) et on note : **A→B** si pour toute valeur de A, il ne peut correspondre au plus une valeur de B
- On dit que deux attributs A et B sont en dépendance fonctionnelle avec un autre attribut X et on note : **A,B → X** si pour toute valeur de A, il peut correspondre plusieurs valeurs de B et réciproquement, pour toute valeur de B, il peut correspondre plusieurs valeurs de A et pour toute valeur de A et de B, il correspond au plus, une valeur de X

Les attributs se trouvant à gauche de la flèche sont appelés ***Source*** de la dépendance fonctionnelle et sont généralement des identifiants d'objets.
Les attributs se trouvant à droite de la flèche sont appelés ***Cible*** de la dépendance fonctionnelle et peuvent être un identifiant ou un attribut non identifiant.
Exemple : Numéro_matricule → Nom,
Référence_produit → libellé_produit
Référence_produit, numfact → Pu_produit

b. Propriété des dépendances fonctionnelles

Les propriétés les plus utilisées sont :

- ***La réflexivité*** : On parle de réflexivité lorsque tout attribut ou ensemble d'attribut, détermine lui-même ou une partie de lui-même. Exemple, A→A ou A,A→B ou A,B→B

- ***L'augmentation*** : On parle d'augmentation dans une dépendance fonctionnelle dont la source est composée d'au moins 2 attributs, si et seulement si, une partie de la source est en dépendance fonctionnelle avec un attribut en but c'est-à-dire : Si A,B→C on dit qu'il y a augmentation, si A→C ou B→C
  Remarque : Dans une dépendance fonctionnelle, l'attribut source peut être enrichi d'un ou de plusieurs autres attributs sans altérer la dépendance fonctionnelle. En somme, Si A→B alors, on peut écrire A,C→B. La dépendance restera toujours fonctionnelle mais, elle contiendra une augmentation.

- ***L'addition*** : Toutes les dépendances fonctionnelles de même source peuvent être additionnées pour donner une seule dépendance fonctionnelle c'est-à-dire, Si A→B et A→C alors, on peut écrire par addition que A→B,C

- ***Transitivité*** : On dit qu'une dépendance fonctionnelle est une transitivité si elle peut se déduire de deux autres dépendances fonctionnelles telles que : Si A→B et B→C alors on peut déduire par transitivité que A→C.
  En somme, dans une dépendance fonctionnelle, si un attribut en but dépend d'un autre attribut que la source (Ou encore, si deux attributs en but sont en dépendance fonctionnelle) On dit qu'il y a transitivité. Exemple : si A→B,C et (B→C ou C→B), on peut déduire que la dépendance fonctionnelle A→B,C contient une transitivité.

c. Les Dépendances fonctionnelles élémentaires

Une dépendance fonctionnelle est dite élémentaire si et seulement si elle ne contient pas d'augmentation c'est-à-dire si l'attribut en but dépend de toute la clé primaire et non d'une partie de la clé primaire. En somme la dépendance fonctionnelle A,C→B est élémentaire si et seulement si je ne peux avoir A→B ou C→B.
Exemple : Numéro_matricule, Code_matière →Nom n'est pas élémentaire car l'attribut Numéro_matricule permet à lui tout seul de déterminer le Nom. C'est-à-dire qu'on peut avoir Numéro_matricule→Nom

Remarque
Toute dépendance fonctionnelle dont la source n'est pas composée est automatiquement élémentaire.

d. Les Dépendances fonctionnelles élémentaires directes

Une dépendance fonctionnelle élémentaire est dite Directe si et seulement si elle ne contient pas de transitivité c'est-à-dire si l'attribut en but dépend uniquement de la clé primaire en source et non d'un autre attribut en but. En somme la dépendance fonctionnelle élémentaire A →B,C est directe si et seulement si je ne peux avoir C→B ou B→C.
Exemple : la dépendance fonctionnelle élémentaire Numéro_matricule→ code_filière, libellé_filière n'est pas directe car l'attribut code_filière permet de déterminer le libellé de la filière. En effet, pour un code filière, on ne peut faire correspondre qu'un et un seul libellé de filière.

Résumé

Dépendance fonctionnelle (DF)	Au plus une valeur
Dépendance fonctionnelle élémentaire (DFE)	Pas d'augmentation
Dépendance fonctionnelle élémentaire directe (DFED)	Pas de transitivité

2. Les formes normales

La normalisation est une opération qui consiste à décomposer une relation en plusieurs relations ne contenant ni augmentation ni transitivité sans perte d'information. Elle se fait en trois étapes qui sont : la Première forme normale (1FN), la deuxième forme normale (2FN), et la troisième forme normale (3FN)

a – *La 1ère forme normale*

Une relation est en 1ière forme normale si tous ses constituants (attributs) sont simples, si elle admet une clé primaire (qui est composée du plus petit ensemble d'attribut) en dépendance fonctionnelle avec les autres attributs de la relation
En somme, pour qu'une relation soit en première forme normale, il faut lui trouver une clé primaire. Cette clé primaire doit être l'ensemble minimum d'attribut qui soit en dépendance fonctionnelle avec tous les autres attributs de la relation.
Ex : Soit la relation commander(numéro_bon, référence_produit, date_commande, libellé_produit, pu, qté_commandée, Adresse_livraison, Numéro_client, Nom_client).
Mettez la en première forme normale sachant que l'adresse de livraison se compose de la ville, du quartier et de la rue.

Pour mettre une relation en première forme normal, il faut :

- ***Décomposer tous les attributs qui sont décomposables et dont les différentes parties ont un sens pour le domaine étudié***

Adresse de livraison doit donc disparaitre et les attributs ville, quartier et rue doivent être ajoutés

- ***Rechercher la clé primaire en faisant l'inventaire des attributs qui peuvent être clé primaire(les numéro, code et référence) et en éliminant ceux qui sont inutiles c'est-à-dire ceux qui sont but de dépendance fonctionnelle d'un autre attribut et qui n'interviendront pas comme source dans une dépendance fonctionnelle à source composée.***

numéro_bon, Numéro_client, référence_produit sont les attributs qui ressemblent à des clés primaires

numéro_bon→ Numéro_client car un bon de commande n'appartient qu'à un seul client. Tous les attributs ayant comme source de dépendance fonctionnelle numéro_client, seront donc par transitivité en dépendance fonctionnelle avec numéro_bon. On peut donc sortir numéro_client de la liste des attributs qui feront partie de la clé primaire et garder seulment numéro_bon, référence_produit

- ***La dernière étape consiste à vérifier que la clé primaire trouvée est effectivement en DF avec les autres attributs de la relation***

numéro_bon, référence_produit →Numéro_client, date_commande, libellé_produit, pu, qté_commandée, ville, quartier, rue, Numéro_client, Nom_client

On peut donc conclure que la relation commander(**numéro_bon**, **référence_produit**, date_commande, libellé_produit, pu, qté_commandée, ville, quartier, rue, Numéro_client, Nom_client) est en 1FN

b – *la 2ième forme normale*

Une relation est en 2ième forme normale si elle est en 1ière forme normale et si tous ses constituants sont en dépendance fonctionnelle élémentaire avec la clé primaire. C'est à dire que pour être en 2ième forme normale, une relation doit être en 1ière forme normale et tout attribut autre que la clé primaire ne doit pas dépendre d'une partie de la clé primaire. Plus simplement, on peut dire qu'une relation est en deuxième forme normale si elle est en première forme normale et si elle ne contient pas d'augmentation.
Ex : La relation commander est - elle en 2ième forme normale ? Si non, mettez la en 2ième forme normale.

numéro_bon, référence_produit →Numéro_client, date_commande, libellé_produit, pu, qté_commandée, ville, quartier, rue, Nom_client

Pour mettre une relation en deuxième forme normale, il faut l'éclater pour la débarrasser des augmentations dans la dépendance fonctionnelle entre la clé primaire et les autres attributs.
Pour ce faire, il faut prendre chaque attribut en but de la dépendance fonctionnelle et rechercher la partie de la source qui le détermine. Dans la DF numéro_bon, référence_produit →Numéro_client, il n'y a que l'attribut numéro_bon qui détermine le numéro du client. L'attribut référence_produit doit alors disparaitre de la source de la dépendance fonctionnelle. On fait la même chose pour tous les attributs en but et on obtient donc les dépendances fonctionnelles élémentaires suivantes :
numéro_bon →Numéro_client, date_commande, ville, quartier, rue, nom_client
référence_produit → libellé_produit, pu
numéro_bon, référence_produit → qté_commandée

Chaque DFE deviendra donc une relation dont la clé primaire est la source de la DFE et les attributs en but, les attributs de la relation.
R1 (**numéro_bon**, Numéro_client, date_commande, ville, quartier, rue, nom_client)
R2 (**référence_produit**, libellé_produit, pu)
R3 (**numéro_bon**, **référence_produit**, qté_commandée)

c - *La troisième forme normale*

Une relation est en 3ième forme normale si elle est en 2ième forme normale et si tous ses constituants sont en dépendance fonctionnelle élémentaire directe avec la clé primaire. C'est à dire, qu'aucune des dépendances

fonctionnelles élémentaires de cette relation ne peut se déduire par transitivité ou encore que chaque attribut dépend directement de la clé primaire et non d'un autre attribut en but que la clé primaire.
Ex : La relation commander est - elle en 3ième forme normale ? Si non, mettez la en troisième forme normale.

Dans la DFE numéro_bon →Numéro_client, date_commande, ville, quartier, rue, nom_client on constate que des attributs en but ne dépendent pas uniquement de la clé primaire numéro_bon. En effet, il existe la DF Numéro_client → nom_client
Il faut donc éclater cette DFE qui deviendra
numéro_bon →Numéro_client, date_commande, ville, quartier, rue
Numéro_client → nom_client
Les autres dépendances fonctionnelles élémentaires n'ayant pas de transitivité, elles sont directes et restent telles. Notre relation Commander deviendra donc :

Bon_commande (**numéro_bon**, #Numéro_client, date_commande, ville, quartier, rue)
Client (**Numéro_client**, nom_client)
Produit (**référence_produit**, libellé_produit, pu)
Détail_commande (#**numéro_bon**, #**référence_produit**, qté_commandée)

Résumé

1ère forme normale	Dépendance fonctionnelle (DF)	Au plus une valeur
2ième forme normale	Dépendance fonctionnelle élémentaire (DFE)	Pas d'augmentation
3ième forme normale	Dépendance fonctionnelle élémentaire directe (DFED)	Pas de transitivité

Chapitre 3 : Le Modèle conceptuel des données ou modèle Entité-Association

Le modèle conceptuel des données est une représentation statique du système d'information de l'entreprise qui met en évidence sa sémantique. Il a pour but de décrire de façon formelle les données qui seront utilisées par le système d'information. Il s'agit donc d'une représentation des données, facilement compréhensible. Cet aspect recouvre les mots qui décrivent le système ainsi que les liens existants entre ces mots. Le formalisme adopté par la méthode Merise pour réaliser cette description est basé sur le concept « entité- association ».

I. Les concepts de base

a. La propriété (ou attribut ou rubrique)

La propriété est une information élémentaire, c'est-à-dire non déductible d'autres informations, qui présente un intérêt pour le domaine étudié. Il correspond à l'attribut en base de données relationnelle. *Par exemple, si l'on considère le domaine de gestion des commandes d'une société de vente par correspondance, les données : « référence article », « désignation article », « prix unitaire HT », « taux de TVA » sont des propriétés pertinentes pour ce domaine. La donnée « prix unitaire TTC » n'est, d'après la définition, pas une propriété car ses valeurs peuvent être retrouvées à partir des propriétés «prix unitaire HT » et « taux de TVA ».*

Chaque valeur prise par une propriété est appelée occurrence. *Des occurrences de la rubrique « désignation article » sont par exemple : « râteau », « bêche », « scie », …*

Une propriété est dite simple ou encore atomique si chacune des valeurs qu'elle regroupe n'est pas décomposable. *La propriété « Adresse », n'est pas élémentaire car elle peut être décomposée en trois propriétés : la rue, le code postal et la ville.*

b. L'entité ou individu ou objet

Définition

Une entité est la représentation d'un élément matériel ou immatériel ayant un rôle dans le système que l'on désire décrire. Chaque entité est composée de propriétés, données élémentaires permettant de la décrire.

Exemple : Filière, Etudiant, Matière

Au premier abord, on peut définir l'entité comme étant un regroupement bien pensé, donc sensé, de plusieurs attributs. *Par exemple, on considère l'entité Etudiant qui regroupe les propriétés : Matricule, Nom, sexe et datedenaissance*. Le droit d'entrée d'une propriété dans une entité est soumis à d'autres facteurs que le bon sens, et ce sont ces facteurs que l'on va étudier.

L'entité est défini par :

- Une existence propre et une utilité pour le domaine d'étude
- Des occurrences multiples (au moins 2)
- Au moins une propriété

Pour représenter une entité, on utilise un rectangle de la forme :

<Nom de l'entité>
<u><Identifiant></u> <Propriété 2> . . . . <Propriété n>

Le nom de l'entité est un mot, généralement une abréviation pour une raison de simplification de l'écriture. Il s'agit par contre de vérifier qu'à chaque entité correspond un et un seul nom, et réciproquement.

c. Les identifiants

Un identifiant est un ensemble minimum d'attributs d'une entité dont la connaissance des valeurs permet d'identifier un enregistrement unique de l'entité considérée. Un identifiant est invariant dans le temps et est souligné lors de la représentation d'une entité.

Ainsi, pour une entité, l'attribut (ou les attributs) choisi comme identifiant ne peut avoir qu'une valeur unique. Deux occurrences de cette entité ne pourront jamais avoir la même valeur pour l'identifiant et donc, la connaissance d'une valeur de la rubrique identifiante détermine la connaissance des valeurs des autres rubriques de l'entité

Chaque entité doit posséder au moins un attribut identifiant, et l'ensemble de ses attributs identifiants doivent être renseignés à la création d'une occurrence de cette entité.

d. Occurrence d'entité ou d'individu

C'est un exemplaire de l'entité ou une ligne dans le tableau illustrant l'entité. Exemple, prenons l'entité Etudiant :

Représentation

Etudiant
<u>Matricule</u> Nom Prenom Sexe

Illustration

Matricule	Nom	Prénom	Sexe
1	Magengo	Guttembert	M
2	Djangoni	Mandélai	F
3	Jolie	Joliette	F
4	Agogo	Bidas	M

« Magengo » est une occurrence de la propriété Nom

La ligne

1	Magengo	Guttembert	M

Est une occurrence de l'entité Etudiant

e. L'association

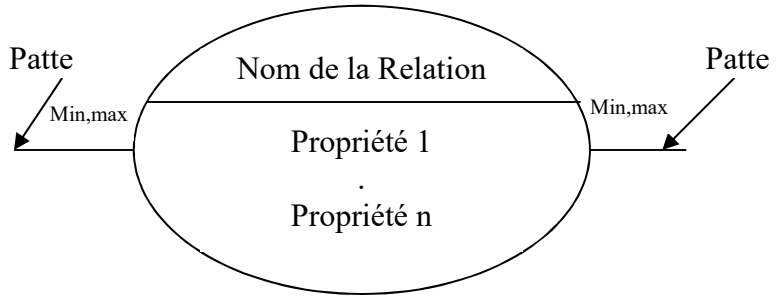
Définition

Une association est un lien sémantique entre plusieurs entités.

Voici les dénominations des différentes associations selon le nombre d'intervenants:

- Une association **récursive** (ou *réflexive*) relie la même entité
- Une association **binaire** relie deux entités
- Une association **ternaire** relie trois entités
- Une association **n-aire** relie n entités. Les associations sont représentées par des hexagones (parfois des ellipses) dont l'intitulé décrit le type de lien qui relie les entités (généralement un verbe).

Une association peut porter ou non des propriétés.



Elle est reliée au rectangle représentant les entités par des traits appelés pattes. Sur chacune des pattes de l'association, sont spécifiés les cardinalités minimum et maximum

f. Les cardinalités

Les cardinalités d'une entité dans une relation mesurent le nombre minimum et le nombre maximum de fois qu'une occurrence de cette entité participe à l'association.

Un couple de cardinalités placé entre une entité E et une association A représente le nombre minimal et maximal d'occurrences de l'association A qui peuvent être liées à une occurrence de l'entité E

On a généralement quatre types de cardinalités:

- 0,1 : (Au plus 1) l'occurrence de l'entité participe au plus une fois à l'association,
- 1,1 : (Un et un seul) l'occurrence de l'entité participe une et une seule fois à l'association.
- 1,N : (Au moins 1) l'occurrence de l'entité participe au moins une fois à l'association.
- 0,N : (Plusieurs) l'occurrence de l'entité participe plusieurs fois à la relation.

Remarque

La cardinalité est située sur la patte de l'association reliée à l'entité pour laquelle elle est définie.

La cardinalité minimum ne doit jamais être supérieur à 1

g. Dimension d'une association

C'est le nombre d'entités qui participent à l'association.

Association réflexive : dimension = 1

Association N-aire : dimension N

h. Contrainte d'intégrité fonctionnelle (cif) entre entités

Une contrainte d'intégrité fonctionnelle est une association établie entre deux entités et qui exprime que l'une de ces entités est totalement identifiée par la connaissance de l'autre. On l'appelle aussi association père-fils ou maitre-esclave.
Une contrainte d'intégrité fonctionnelle est toujours établie entre deux entités au plus et ne peut jamais porter d'attributs. Elle est matérialisée par la présence de la cardinalité maximum 1 sur au moins une des pattes (0,1 ou 1,1).
On peut remplacer le nom de l'association par le symbole **CIF**.

II. Règles de construction d'un M.C.D.

a. Les règles de gestion

Elles expriment les contraintes de gestion, de manipulation ou encore d'utilisation des données. Elles constituent le règlement intérieur dans le processus d'élaboration du MCD.
Elles précisent les liens qui peuvent être établis ou non entre les occurrences des entités ou entre les occurrences des attributs.
Exemple : Un étudiant s'inscrit dans une filière
Cette règle de gestion exprime un lien de dépendance fonctionnelle entre l'attribut Matricule et l'attribut CodeFilière et un lien d'association entre l'entité Etudiant et l'entité Filière.
Dans le processus de construction du MCD, il convient de bien recenser la liste exhaustive de ces règles de gestion.

b. Le Dictionnaire des données

C'est un tableau permettant d'inventorier toutes les informations manipulées par le domaine d'étude, qui ont un sens et qui peuvent avoir au moins deux occurrences. Ce tableau se présente comme suit :

Propriétés	Désignation en clair	Type	Nature	Longueur	Observation
Numat	Numéro matricule de l'étudiant	N	E	6	
NomEtu	Nom de l'étudiant	C	E	15	
DatNai	Date de naissance de l'étudiant	D	E	10	jj/mm/aaaa
SexEtu	Sexe de l'étudiant	C	E	1	« M » ou « F »

La colonne Propriété permet de spécifier pour chaque information, son nom. Le nom doit être court, explicite et doit être un mot. Il ne doit pas comporter de caractère interdits tels que : (,) (;) (-) (:) etc. mais peut comporter des combinaisons de lettres, de chiffres et le trait de 8(_) ou underscore

Désignation en clair permet de donner une description explicite de la propriété

Nature peut prendre les valeurs suivantes :

- E : élémentaire qui signifie que la propriété est atomique c'est-à-dire ni calculée, ni concaténée, ni décomposable.
- Co : Concaténée qui signifie que la propriété est décomposable. Dans ce cas, la formule de

décomposition doit être spécifiée dans la colonne Observation et les informations variables entrant dans la décomposition doivent être ajoutés au dictionnaire des données.

- Ca : calculé qui signifie que la valeur de la propriété est issue d'un traitement ou de l'évaluation d'une formule avec des opérateurs numériques. Dans ce cas, la formule doit être spécifiée dans la colonne Observation et les informations variables qu'elle utilise doivent être ajoutées au dictionnaire des données.

Le type est le domaine de valeurs dans lequel une propriété peut puiser ses occurrences. On distingue les types suivants : Numérique N, caractère C, Logique L, Date D

La longueur est le nombre maximum de caractères qu'il faut pour représenter une occurrence de la propriété

La colonne Observation permet de faire tout commentaire qui pourrait rendre la propriété plus explicite.

c. La couverture minimale

C'est la liste de toutes les dépendances fonctionnelles élémentaires et directes c'est-à-dire des dépendances fonctionnelles débarrassées de toute augmentation et transitivité.
On distingue trois catégories de dépendance fonctionnelles :

- **Les dépendances fonctionnelles simples**

Ce sont les dépendances fonctionnelles élémentaires directes à sources non composées et dont le but ne contient pas d'identifiants
Exemple : Matricule → Nom_etu, sexe, Dat_nais

- **Les dépendances fonctionnelles à sources composées**

Ce sont les dépendances fonctionnelles élémentaires directes dont la source est composée d'au moins deux attributs en source et des attributs non identifiants en but.
Exemple : Code_matière, code_filière→Coefficient

- **Les dépendances fonctionnelles entre un identifiant en source et un identifiant en but**

Ce sont les dépendances fonctionnelles dans lesquelles la source est composée d'un identifiant et aussi le but, d'un identifiant.
Exemple : Un étudiant s'inscrit dans une et une seule filière : Matricule →Code_filière

d. La matrice des dépendances fonctionnelles

C'est un autre moyen de représenter la liste des dépendances fonctionnelles élémentaires directes. Mais, cette fois ci à partir d'un tableau à double entrée dans lequel on représente en en-tête de colonne, la liste de tous les identifiants (représentants les sources de dépendance fonctionnelles) et en en-tête de ligne, la liste de toutes les propriétés identifiants et non identifiants.
La matérialisation d'une DFED se fait en inscrivant le caractère étoile ou 1 à l'intersection de la source (entête de colonne) et du But (Entête de ligne)

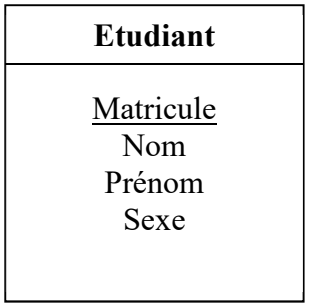
III. Construction du MCD à partir de la couverture minimale

La construction du MCD se fait à partir de la couverture minimale. Chacune des trois catégories de DFED

correspond à une représentation particulière au niveau du MCD.

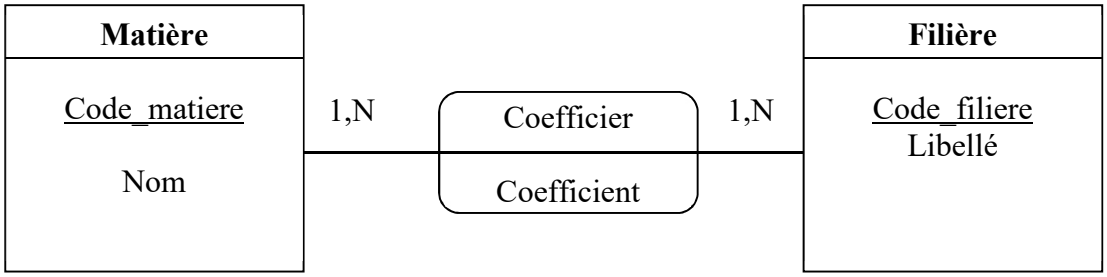
- La première catégorie de DFED (Les DFED simples) permet de représenter toutes les entités du MCD. En effet, toute dépendance fonctionnelle élémentaire directe entre une source non composée et des attributs non identifiants devient une entité dont l'identifiant est la source de la dépendance fonctionnelle élémentaire directe et dont les propriétés sont les attributs en but de la dépendance fonctionnelle.

Exemple : la dépendance fonctionnelle Matricule →Nom, prénom, sexe donnera l'entité Etudiant suivante :

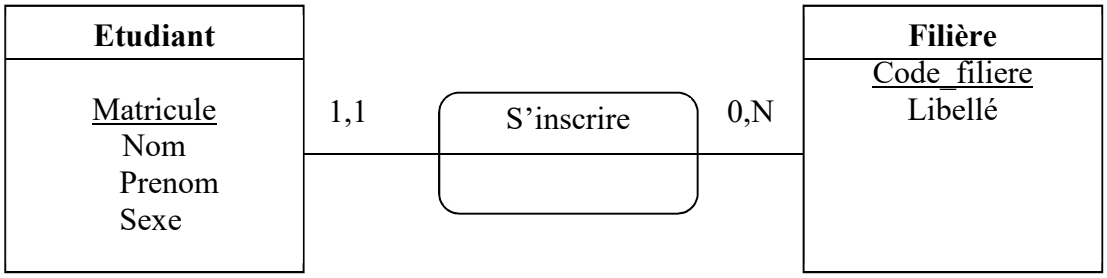


- La deuxième catégorie de DFED (Les DFED simples) permet de représenter toutes les associations plusieurs à plusieurs porteuses de propriétés du MCD. En effet, toute dépendance fonctionnelle élémentaire directe dont la source est composée devient une association entre les entités des identifiants qui composent la source. La cardinalité maximum sur toutes les pattes de l'association est N et les attributs en but de la dépendance fonctionnelle élémentaire directe deviennent les attributs de l'association.

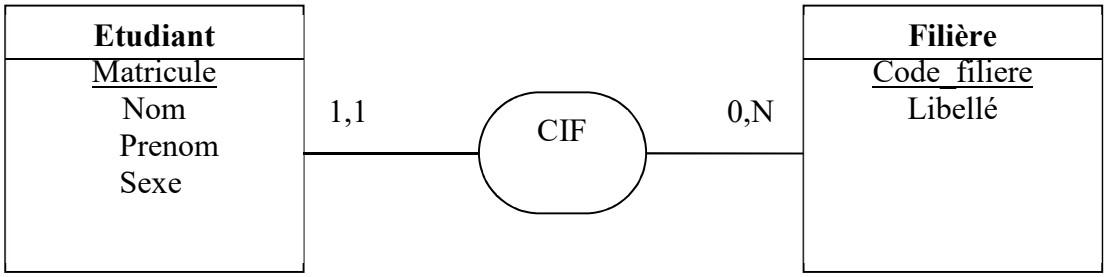
Exemple : la dépendance fonctionnelle Code_filiere, Code_matiere → coefficient donnera l'association suivante :



- La troisième catégorie de DFED (Les DFED entre identifiant et identifiant) permet de représenter toutes les associations père fils (CIF) du MCD. En effet, toute dépendance fonctionnelle élémentaire directe entre un identifiant et un identifiant devient une association CIF (Contrainte d'intégrité fonctionnelle) avec la cardinalité maximale 1 sur la patte de l'entité de l'identifiant en source de dépendance fonctionnelle.



Une telle association ne porte jamais de propriétés et elle peut être remplacée par le symbole CIF



Exercice d'application

Voir planche de Base de données. Exercice : gestion commerciale.

TAF

- Etablir le dictionnaire des données
- Présenter la couverture minimale
- Présenter la matrice des DFED
- Présenter le MCD

IV. Modélisation avancée

a. Les associations réflexives

Une association réflexive est une association qui fait intervenir au moins deux fois la même entité.

Elle peut être une association plusieurs à plusieurs. Dans ce cas, elle est représentée dans la couverture minimale par la DFED suivante : Id1,Id1→at1,…..,atN

L'identifiant de l'entité apparait deux fois en source de la DFED.

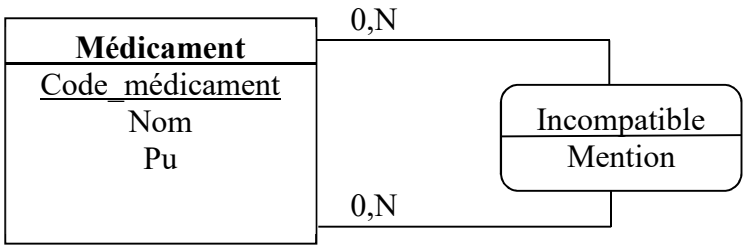
Exemple : Dans une entreprise pharmaceutique, on désire modéliser l'incompatibilité entre les médicaments. Un médicament peut être incompatible avec plusieurs autres médicaments. Pour chaque incompatibilité, une mention est définie : grave, moins grave, inoffensive etc. Un médicament a un code, un nom et un pu.

On aura la couverture minimale suivante :

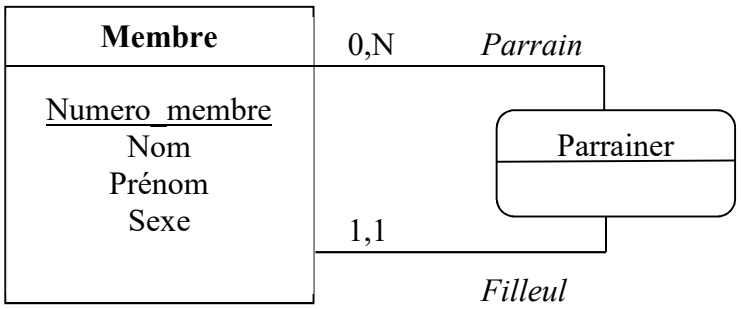
Code_médicament→Nom, Pu

Code_médicament, Code_médicament →Mention

Ce qui donnera le MCD suivant :

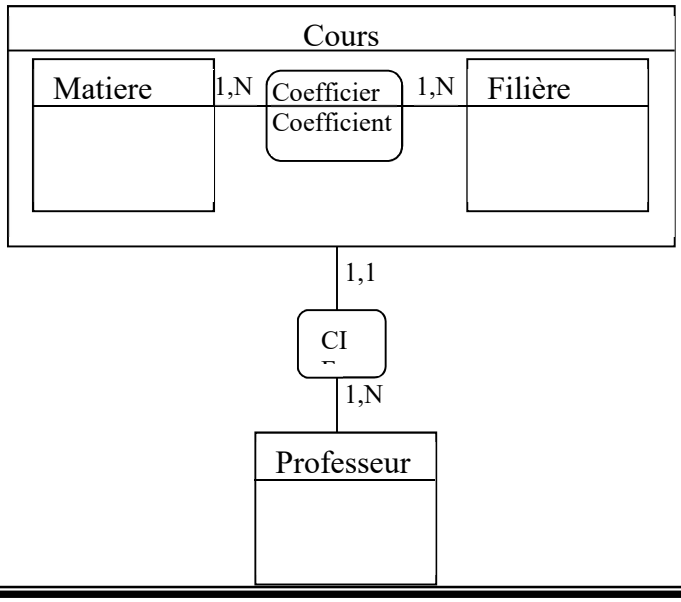


Cette association peut aussi être une association père fils. Dans ce cas, elle est représentée dans la couverture minimale par la DFED suivante :
Id1 → Id1
Exemple : On considère une association dans laquelle chaque membre est parrainé par un autre membre. Sachant que un membre est décrit par son numéro, son nom, son prénom et son sexe et ne peut être parrainé que par un seul membre. Tandis qu'un membre peut parrainer plusieurs filleuls.
Numero_membre→Nom, prenom, sexe
Numero_membre →Numero_membre



b. Les pseudo entités

Toute dépendance fonctionnelle élémentaire directe entre une source composée et un identifiant devient une contrainte d'intégrité fonctionnelle entre une pseudo entité matérialisé par une association entre les entités des identifiants en source de la DFED et l'entité de l'identifiant en but de la dépendance fonctionnelle.
Exemple soit les DFED
Code_filiere, Code_matiere →Num_prof
Code_filiere, Code_matiere →coefficient
qui signifient que une matière dans une filière n'a qu'un coefficient et est enseignée par un et un seul professeur



Chapitre III - Algèbre relationnelle

L'algèbre relationnelle est une collection d'opérations permettant d'opérer sur les concepts du modèle relationnel. Elle permet par exemple de sélectionner certains enregistrements d'une relation satisfaisant une condition ou encore de regrouper des enregistrements de relations différentes. Le résultat de toute opération de l'algèbre est une nouvelle relation. Cette propriété implique notamment qu'il n'y a pas de doublons dans le résultat et permet l'écriture d'expressions de calcul. Etant donnée, que le modèle relationnel est basé sur la théorie des ensembles, l'algèbre relationnelle utilise les opérateurs classiques de manipulation des ensembles (union, intersection, différence et produit cartésien) et introduit des opérateurs propres aux bases de données (sélection, projection, jointure, division).

a – <u>les opérations portant sur une seule relation</u>

- ***<u>la projection</u>***

Elle consiste à construire une nouvelle relation R' à partir d'une relation R en ne retenant que certains attributs de R. les tuples(enregistrements ou articles) en double sont éliminés.

Syntaxe

PROJECTION.<nom relation>(<attribut1>,…,<attribut n>)

Exemple soit la relation R (refprod, desipro, pu, qtes) présentée comme la table ci-dessous :

R			
refpro	desipro	qtes	pu
AC	Akassa congelé	10	250
PP	Poisson pourri	5	400
RB	Riz brûlé	5	275

R'=PROJECTION.R (refpro, desipro)

La relation R' ainsi créée ne contiendra que les champs refpro et desipro

R	
refpro	desipro
AC	Akassa congelé
PP	Poisson pourri
RB	Riz brûlé

- ***<u>La sélection</u>***

Elle consiste à créer une nouvelle relation R' à partir d'une relation R en ne conservant que les tuples répondant à une condition donnée. La relation R' ne sera donc constituée que de tuples ne répondant qu'aux critères de sélection.

Syntaxe

SELECTION.< nom relation>(<critère de sélection>)

Ex : soit la relation R de l'exemple précédent
R' est la relation contenant tous les produits dont le pu est inférieur à 400
R'=SELECTION.R (Pu<400)

R'			
refpro	**desipro**	**qtes**	**pu**
AC	Akassa congelé	10	250
RB	Riz brûlé	5	275

On peut limiter les champs à afficher. Pour cela, on fait une projection sur R'.
Ex : afficher la référence et la quantité de tous les produits dont la qté égale 5

- ***Le tri***

Cette opération consiste à créer une nouvelle relation R' à partir d'une relation R en classant les tuples par ordre alphabétique ou numérique croissant ou décroissant sur un ou plusieurs champs.

Syntaxe

TRI.< nom relation>(<Attribut 1> [CROISSANT|DECROISSANT], …, <Attribut N> [CROISSANT|DECROISSANT])

Ex : soit la relation R de l'exemple précédent
R' est la relation contenant tous les produits classés par ordre croissant des références
R'=TRI.R (Refpro **CROISSANT**)

R'			
refpro	desipro	qtes	pu
AC	Akassa congelé	10	250
PP	Poisson pourri	5	400
RB	Riz brûlé	5	275

b - <u>les opérations portant sur des relations de même schéma</u>

- ***L'union***

Cette opération s'opérant sur deux relations consiste à ajouter à la première relation les tuples de la 2ième relation qui ne sont identiques à aucun tuple de la première.

Syntaxe

R1 U R2
Ex : soient les relations R1 et R2 de même schéma tel que R1=R2(refpro, desipro, pu)

R1		
refpro	desipro	pu
AC	Akassa congelé	250
PP	Poisson pourri	400
RB	Riz brûlé	275

R2		
refpro	desipro	pu
AC	Akassa congelé	250
PP	Poisson pourri	400
PV	Piment vert	275
ALS	Aloko sec	375

Trouver R tel que R =R1 U R2

- ***L'intersection***

L'intersection de deux relations r1 et r2 de même schéma est une relation R3 contenant les tuples appartenant à la fois à R1 et à R2. il s'agira donc de sélectionner les tuples communs aux deux relations.

Syntaxe

INTERSECTION (R1, R2) ou R1 ∩ R2

Ex :

- ***La différence***

Cette opération permet de créer une relation R3 à partir de deux relations R1 et R2 de même schéma contenant des tuples appartenant à R1 et n'appartenant pas à R2.

Syntaxe

DIFFERENCE (R1, R2) ou R1-R2

l'ordre de la relation est très important. R1-R2 est différent de R2-R1

c – Les opérations portant sur des relations de schémas différents

- ***le produit ou produit cartésien***

Cette opération permet d'obtenir toutes les relations possibles de deux relations de schéma différent.

Syntaxe

PRODUIT (R1, R2) ou R1 x R2

Ex :

- ***La jointure***

La jointure de R1 et R2 selon une condition est l'ensemble des tuples du produit cartésien R1xR2 qui satisfont à la condition indiquée. Cette opération permet donc de créer un sous-ensemble du produit cartésien R1xR2 qui satisfait à une condition portant sur un attribut de chaque relation. Le cas le plus indiqué est la jointure naturelle appelée jointure Equi-jointure fondée sur l'égalité des valeurs d'un attribut commun aux deux relations. Un seul des deux attributs de même valeur est conservé dans la relation obtenue.

Syntaxe

JOINTURE (R1, R2, <condition>)

Ex

- ***La division***

C'est le quotient de la relation R de schéma R (a1, a2, … , an) par la sous relation S de schéma S (Ap+1, … , An) qui est une relation Q de schéma Q (a1, a2, … , a_q) formé de tous les tuples qui concaténés à chacun des tuples de S donne toujours un tuple de R.

Syntaxe

DIVISION (R, S) ou **R / S** ou **R : S**

Ex

Soit les relations Composer et Matière telles que :

Composer			
Matricule	Date compo	Cod_matiere	Note
001	05/02/95	Ang	10
002	03/04/99	Ang	5
003	03/04/99	FR	15
001	03/04/99	FR	2
002	05/02/95	BD	18
001	05/02/95	BD	13

Matière	
Code_matiere	Libellé
Ang	Anglais
FR	Français
BD	Base de données

On désire afficher les matricules des étudiants qui ont composé dans toutes les matières

D = Projection.Composer (Matricule, Cod_matiere)

d = Projection.Matière (Cod_matiere)

Q = Division (D,d)

d – Les expressions d'attribut et le regroupement (Agrégation)

- **Les expressions d'attribut**

  Une expression d'attribut est une combinaison d'attributs, d'opérateurs numériques et de constante pouvant être assimilée à une formule qui, évaluée donne un résultat ou une information non prévue dans la relation.

  Exemple soit la relation R (refprod, desipro, pu, qtes) présentée comme la table ci-dessous :

R			
refpro	desipro	qtes	pu
AC	Akassa congelé	10	250
PP	Poisson pourri	5	400
RB	Riz brûlé	5	275

On désire afficher pour chaque produit, sa référence et son montant

R'=PROJECTION.R (refpro, Pu*Qtes)

R	
refpro	Pu*Qtes
AC	2500
PP	2000
RB	1375

Remarque : On peut changer le nom de la colonne contenant l'expression d'attribut en utilisant la clause COMME.
R'=PROJECTION.R (refpro, Pu*Qtes COMME Montant)

R	
refpro	Montant
AC	2500
PP	2000
RB	1375

- Le regroupement
  Le regroupement est une opération qui permet à partir d'une relation R de créer une relation R' contenant la synthèse des enregistrements de R.
  Syntaxe
  R'=REGROUPEMENT (R ; <Liste des attributs de regroupement> ; <Liste des fonctions de regroupement>)
  Les attributs de regroupement sont les attributs qui serviront à regrouper les enregistrements
  Exemple :
  Soit la table Etudiant (Matricule, nom, Sexe, Codfil) illustrée par le tableau suivant :

Matricule	**Nom**	Sexe	Codfil
001	Magengo	M	IIR
002	Jolie	F	SB
003	Marteau	M	BF
004	Cailloux	F	IIR
005	Gravière	F	IIR
006	BLANC	M	CG
007	MAHIGÔH	F	IIR
008	Djangoni	F	CG

On désire afficher le nombre d'étudiants par filière : les étudiants seront donc regroupés par filière afin qu'on puisse les compter. Le champ de regroupement est donc Codfil
Si On désire afficher le nombre d'étudiants par filière et par sexe, les étudiants seront donc regroupés par filière et par sexe afin qu'on puisse les compter. Les champs de regroupement seront donc Codfil et Sexe.
Une fonction de regroupement est une fonction prédéfinie qu'on applique sur un groupe d'enregistrement pour en obtenir la synthèse c'est-à-dire pour obtenir un enregistrement. En somme, les attributs de regroupement permettent de créer des groupes d'enregistrement sur lesquels on applique une fonction de regroupement pour obtenir la synthèse de chaque groupe. On distingue différentes fonctions de regroupement :
Compte(*) ou compte (<attribut>) : cette fonction permet de compter le nombre d'enregistrements dans un groupe
Somme (<attribut>) : cette fonction permet de faire la somme des valeurs de l'attribut spécifié d'un groupe d'enregistrement.
Moyenne (<attribut>) : cette fonction permet de faire la moyenne arithmétique des valeurs de l'attribut spécifié d'un groupe d'enregistrement.
Min / Max (<attribut>) : cette fonction permet de rechercher la plus petite ou la plus grande valeur de l'attribut spécifié d'un groupe d'enregistrement
Premier / Dernier (<attribut>) : cette fonction permet de rechercher la première ou la dernière valeur de l'attribut spécifié d'un groupe d'enregistrement

Exemple :
Soit la table Etudiant (Matricule, nom, Sexe, Codfil) illustrée par le tableau suivant :

Matricule	Nom	Sexe	Codfil
001	Magengo	M	IIR
002	Jolie	F	SB
003	Marteau	M	BF
004	Cailloux	F	IIR
005	Gravière	F	IIR
006	BLANC	M	CG
007	MAHIGÔH	F	IIR
008	Djangoni	F	CG

Afficher le nombre d'étudiants par filière
R=REGROUPEMENT (Etudiant ; Codfil ; Compte(*) comme Nombre)

Codfil	Nombre
IIR	4
SB	1
BF	1
CG	2

Afficher le nombre d'étudiants par filière et par sexe
R=REGROUPEMENT (Etudiant ; Codfil, Sexe ; Compte(*) comme Nombre)

Codfil	Sexe	Nombre
IIR	M	1
IIR	F	3
SB	F	1
BF	M	1
CG	F	1
CG	M	1

Remarque
Le résultat d'un regroupement est une relation contenant seulement les attributs de regroupement et les fonctions de regroupement. L'opération qui suit un regroupement ne peut donc utiliser que les attributs de regroupement et les fonctions de regroupement.



Le langage SQL(Structured querry language) vient des travaux de recherche menés chez IBM. C'est un langage de requête et un langage de description des données qui est utilisé directement sur un terminal grâce à l'utilitaire interactif SQL ou dans un langage hote (Cobol, C, DbaseIV, Access, Mysql, SqlServer, Oracle, ect…)

DEFINITIONS

Requête

Question sur les données enregistrées dans vos tables ou demande d'exécution d'une action sur les données. Une requête permet de réunir les données de plusieurs tables en vue de leur utilisation comme source de données pour divers traitements
Vous pouvez utiliser des requêtes pour afficher, modifier et analyser des données de diverses façons.

Requêtes de définition des données

Requête propre à SQL qui contient des instructions DDL (Data Definition Language). Celles-ci vous permettent de créer ou de modifier la structure des tables de la base de données.

Requêtes de manipulation des données

Contrairement aux Requêtes de définition des données, les Requêtes de manipulation des données permettent d'agir sur les données des tables et non sur leurs structures.

II - LES INSTRUCTIONS DE REQUETES DE DEFINITION DES DONNEES

Utilisez ces instructions pour créer des requêtes Définition des données SQL qui créent, suppriment ou modifient des tables.

- **Instruction CREATE TABLE simplement**

Crée une nouvelle table.

Syntaxe

CREATE TABLE <*table*> (<*champ1*> < *type*> [(<*taille*>)] [<Contrainte de champ>] ,….., <*champN*> < *type*> [(<*taille*>)] [<Contrainte de champ>],[<Contrainte de table>])

L'instruction CREATE TABLE se compose des éléments suivants :

Élément	Description
table	Nom de la table à créer.

champ1, *champ2*	Nom du ou des champs à créer dans la nouvelle table. Vous devez créer au moins un champ.
type	Type de données du *champ* dans la nouvelle table.
taille	Taille du champ en caractères (pour les données de type Texte ou Binaire uniquement).

[<Contrainte de champ>] vous permet d'appliquer à un champ l'un des types de contraintes suivants :

- ***NOT NULL*** : Vous pouvez utiliser ces mots réservés pour spécifier qu'un champ ne peut contenir ne peut rester vide. Cela signifie qu'un enregistrement de la table ne peut être validé que si ce champ contient une valeur. Vous pouvez contraindre n'importe quel champ comme étant non vide.
CREATE TABLE Filière (Codfil INTEGER NOT NULL, Libellé CHAR (50) NOT NULL)

- ***UNIQUE*** : Vous pouvez utiliser ce mot réservé pour désigner un champ en tant que clé unique. Cela signifie que deux enregistrements de la table ne peuvent pas avoir la même valeur dans ce champ. Vous pouvez contraindre n'importe quel champ ou n'importe quelle liste de champs comme étant unique.
CREATE TABLE Filière (Codfil INTEGER ***UNIQUE***, Libellé CHAR (50) NOT NULL)

- ***PRIMARY KEY*** : Vous pouvez utiliser ces mots réservés pour désigner un champ ou ensemble de champs dans une table en tant que clé primaire. Toutes les valeurs de la clé primaire doivent être uniques, non **Null**, et il ne peut y avoir qu'une clé primaire par table. Ne définissez pas de contrainte PRIMARY KEY sur une table qui possède déjà une clé primaire. Une erreur se produira si vous le faites.
CREATE TABLE Filière (Codfil INT PRIMARY KEY, Libellé CHAR (50))

- ***REFERENCES table [déclencheur {RESTRICT | NO ACTION | SET NULL | SET DEFAULF | CASCADE}*** et Contrainte d'intégrité référentielle
Vous pouvez utiliser ce mot réservé pour désigner un champ en tant que clé étrangère. Si le ou les champs référencés sont la clé primaire de la table étrangère, vous n'avez pas besoin de spécifier les champs référencés. Par défaut, le moteur de base de données se comporte comme si la clé primaire de la table étrangère était les champs référencés.
CREATE TABLE Etudiant (Matricule INTEGER PRIMARY KEY, Nom CHAR (255), Codfil INTEGER REFERENCES Filière)

Les déclencheurs sont ON DELETE et ON UPDATE. Ils sont suivis des arguments entre accolades permettant de spécifier l'action à réaliser en cas d'effacement d'une ligne de la table faisant partie de la clé étrangère:

- CASCADE indique la suppression ou la modification en cascade des lignes de la table étrangère dont les clés étrangères correspondent aux clés primaires des lignes effacées
- RESTRICT ou NO ACTION est le comportement par défaut. Si l'on essaye de supprimer une valeur référencée par une clé étrangère, l'action est avortée et on obtient une erreur. NO ACTION a exactement le même effet.
- SET NULL place la valeur NULL dans la ligne de la table étrangère en cas d'effacement d'une valeur correspondant à la clé
- SET DEFAULT place la valeur par défaut dans la ligne de la table étrangère en cas d'effacement d'une valeur correspondant à la clé

- ***DEFAULT*** *<valeur>*

Permet de définir la valeur par défaut d'un attribut si aucune valeur ne lui est attribuée. Exemple, pour une table qui enregistre les clients, si la plupart des clients viennent de la ville de Cotonou, pour faciliter la saisie de l'utilisateur, la valeur par défaut peut être définie sur Cotonou. L'utilisateur n'aura donc plus besoin de saisir la ville pour les clients venant de Cotonou. La valeur apparaitra par défaut.

- ***CHECK*** *(<condition>)*

Les contraintes CHECK assurent l'intégrité de domaine en limitant les valeurs acceptées par une ou plusieurs colonnes. Vous pouvez créer une contrainte CHECK avec n'importe quelle expression logique (booléenne) qui retourne TRUE (vrai) ou FALSE (Faux) sur la base des opérateurs logiques.

Par exemple, il est possible de limiter la plage de valeurs d'une colonne Sexe en créant une contrainte CHECK qui autorise uniquement les données M et F. De cette façon, il est impossible d'entrer une valeur erronnée comme G pour dire Garçon. L'expression logique est la suivante : Sexe='M' Or Sexe='F'.

Vous pouvez appliquer plusieurs contraintes CHECK à une seule colonne (Contrainte d'attribut).

[<Contrainte de table>] vous permet de spécifier des contraintes liées à l'ensemble de la table comme :

- Une contrainte de clé primaire composée ou non de plusieurs attributs :

Syntaxe :
PRIMARY KEY (<champ1>,……..,<champN>)
Exemple :
CREATE TABLE Coefficier(Codmat int REFERENCES Matière, codfil int REFERENCES Filière, coefficient int, ***PRIMARY KEY*** (codmat,codfil))

- Ou une contrainte de domaine composée ou non de plusieurs attributs

Vous pouvez appliquer une seule contrainte CHECK à plusieurs colonnes en la créant au niveau des contraintes de table. Ainsi, vous pouvez utiliser une contrainte CHECK sur plusieurs colonnes pour confirmer que les lignes comportant la valeur **Marié** dans leur colonne **Situation matrimoniale** et la valeur **F** dans leur colonne **Sexe** possèdent forcément une valeur dans leur colonne Nom de jeune fille. Cela permet de vérifier plusieurs conditions au même emplacement.

CREATE TABLE Employé (Matricule int *PRIMARY KEY* , Nom char, Nom_jeune_fille char, sexe char, situation_matrimoniale char, CHECK (Sexe='F' and situation_matrimoniale='Marié' and Nom_jeune_fille <>Null)

- Ou une contrainte de clé étrangère composée ou non de plusieurs attributs

FOREIGN KEY (<attribut1>,…,<attributN>) REFERENCES <table> [(<attribut1>,…,<attributN>)]

CREATE TABLE Etudiant (Matricule INTEGER PRIMARY KEY, Nom CHAR (255), Codfil INTEGER, FOREIGN KEY (Codfil) REFERENCES Filière)

Remarque
La contrainte CHECK ne fonctionne pas en Access lors de la création d'une nouvelle table. Pour ajouter une contrainte de domaine en Access, vous créez d'abord la table et vous utilisez la commande suivante pour ajouter votre contrainte de domaine :
ALTER TABLE <Nom table> ADD CONSTRAINT <NomContrainte> CHECK (<Condition>);

Les Types de données

Il existe généralement trois types principaux en SQL quel que soit le SGBD

Type de données	Description
DATETIME	Une valeur date ou heure comprise entre l'année 100 et l'année 9999.
INTEGER	Un entier long compris entre – 2 147 483 648 et 2 147 483,647 (voir Remarques).
DECIMAL	Type de données numérique exact contenant des valeurs comprises entre 10^{28} - 1 et - 10^{28} - 1. Vous pouvez définir la précision (1 - 28) ainsi que l'échelle (0 - précision définie). La précision et l'échelle par défaut sont 18 et 0 respectivement.
TEXT	De zéro à 2.14 giga-octets au maximum.
CHAR	De zéro à 255 caractères.

En plus de ces types standards, la plupart des SGBD intègrent chacun des types supplémentaires :

A. Types de données MySQL (version 8.0)

Dans MySQL, il existe trois types de données principaux : chaîne, numérique et date et heure.

- **Types de données de chaîne**

Data type	Description
CHAR(size)	Une chaîne de caractères de LONGUEUR FIXE (peut contenir des lettres, des chiffres et des caractères spéciaux). Le paramètre de taille spécifie la longueur de la colonne en caractères - peut être de 0 à 255. Par défaut, c'est 1.
VARCHAR(size)	Une chaîne de caractères de longueur VARIABLE (peut contenir des lettres, des chiffres et des caractères spéciaux). Le paramètre de taille spécifie la longueur maximale de la chaîne en caractères - pouvant aller de 0 à 65535.
BINARY(size)	Égal à CHAR(), mais stocke les chaînes d'octets binaires. Le paramètre size spécifie la longueur de la colonne en octets. La valeur par défaut est 1
VARBINARY(size)	Égal à VARCHAR(), mais stocke les chaînes d'octets binaires. Le paramètre size spécifie la longueur maximale de la colonne en octets.
TINYBLOB	Pour les BLOB (Binary Large Objects). Longueur maximale : 255 octets
TINYTEXT	Contient une chaîne d'une longueur maximale de 255 caractères
TEXT(size)	Contient une chaîne d'une longueur maximale de 65 535 octets

BLOB(size)	Pour les BLOB (Binary Large Objects). Peut contenir jusqu'à 65 535 octets de données
MEDIUMTEXT	Contient une chaîne d'une longueur maximale de 16 777 215 caractères
MEDIUMBLOB	Pour les BLOB (Binary Large Objects). Peut contenir jusqu'à 16 777 215 octets de données
LONGTEXT	Contient une chaîne d'une longueur maximale de 4 294 967 295 caractères
LONGBLOB	Pour les BLOB (Binary Large Objects). Peut contenir jusqu'à 4 294 967 295 octets de données
ENUM(val1, val2, val3, ...)	Un objet chaîne qui ne peut avoir qu'une seule valeur, choisie parmi une liste de valeurs possibles. Vous pouvez répertorier jusqu'à 65 535 valeurs dans une liste ENUM. Si une valeur insérée ne figure pas dans la liste, une valeur vide sera insérée. Les valeurs sont triées dans l'ordre dans lequel vous les saisissez
SET(val1, val2, val3, ...)	Un objet chaîne pouvant avoir 0 ou plusieurs valeurs, choisies dans une liste de valeurs possibles. Vous pouvez lister jusqu'à 64 valeurs dans une liste SET

- **Types de données numériques**

Data type	**Description**
BIT(*size*)	Un type de valeur binaire. Le nombre de bits par valeur est spécifié en taille. Le paramètre size peut contenir une valeur comprise entre 1 et 64. La valeur par défaut pour la taille est 1.
TINYINT(*size*)	Un très petit entier. La plage signée va de -128 à 127. La plage non signée va de 0 à 255. Le paramètre size spécifie la largeur d'affichage maximale (qui est de 255).
BOOL	Zéro est considéré comme faux, les valeurs non nulles sont considérées comme vraies.
BOOLEAN	Égal à BOOL
SMALLINT(*size*)	Un petit entier. La plage signée va de -32 768 à 32 767. La plage non signée va de 0 à 65 535. Le paramètre size spécifie la largeur d'affichage maximale (qui est de 255).
MEDIUMINT(*size*)	Un entier moyen. La plage signée va de -8388608 à 8388607. La plage non signée va de 0 à 16777215. Le paramètre size spécifie la largeur d'affichage maximale (qui est de 255).

INT(*size*)	Un entier moyen. La plage signée va de -2147483648 à 2147483647. La plage non signée va de 0 à 4294967295. Le paramètre size spécifie la largeur d'affichage maximale (qui est de 255).
INTEGER(*size*)	Égal à INT(taille)
BIGINT(*size*)	Un grand entier. La plage signée va de -9223372036854775808 à 9223372036854775807. La plage non signée va de 0 à 18446744073709551615. Le paramètre size spécifie la largeur d'affichage maximale (qui est de 255).
FLOAT(*size*, *d*)	Un nombre à virgule flottante. Le nombre total de chiffres est spécifié en taille. Le nombre de chiffres après la virgule décimale est spécifié dans le paramètre d. Cette syntaxe est obsolète dans MySQL 8.0.17 et sera supprimée dans les futures versions de MySQL
FLOAT(*p*)	Un nombre à virgule flottante. MySQL utilise la valeur p pour déterminer s'il faut utiliser FLOAT ou DOUBLE pour le type de données résultant. Si p est compris entre 0 et 24, le type de données devient FLOAT(). Si p est compris entre 25 et 53, le type de données devient DOUBLE()
DOUBLE(*size*, *d*)	Un nombre à virgule flottante de taille normale. Le nombre total de chiffres est spécifié en taille. Le nombre de chiffres après la virgule décimale est spécifié dans le paramètre d
DOUBLE PRECISION(*size*, *d*)	
DECIMAL(*size*, *d*)	Un nombre exact à virgule fixe. Le nombre total de chiffres est spécifié en taille. Le nombre de chiffres après la virgule décimale est spécifié dans le paramètre d. Le nombre maximum pour la taille est 65. Le nombre maximum pour d est 30. La valeur par défaut pour la taille est 10. La valeur par défaut pour d est 0.
DEC(*size*, *d*)	Égal à DECIMAL(taille,d)

Remarque : Tous les types de données numériques peuvent avoir une option supplémentaire : UNSIGNED ou ZEROFILL. Si vous ajoutez l'option UNSIGNED, MySQL interdit les valeurs négatives pour la colonne. Si vous ajoutez l'option ZEROFILL, MySQL ajoute automatiquement également l'attribut UNSIGNED à la colonne.

- **Types de données de date et d'heure**

Data type	**Description**
DATE	Une date. Format : AAAA-MM-JJ. La plage prise en charge va de « 1000-01-01 » à « 9999-12-31 ».
DATETIME(*fsp*)	Une combinaison de date et d'heure. Format : AAAA-MM-JJ hh:mm:ss. La plage prise en charge va de « 1000-01-01 00:00:00 » à « 9999-12-31 23:59:59

	». Ajout de DEFAULT et ON UPDATE dans la définition de colonne pour obtenir une initialisation et une mise à jour automatiques à la date et à l'heure actuelles
TIMESTAMP(*fsp*)	Un horodatage. Les valeurs TIMESTAMP sont stockées sous forme de nombre de secondes depuis l'époque Unix ('1970-01-01 00:00:00' UTC). Format : AAAA-MM-JJ hh:mm:ss. La plage prise en charge va de « 1970-01-01 00:00:01 » UTC à « 2038-01-09 03:14:07 » UTC. L'initialisation et la mise à jour automatiques à la date et à l'heure actuelles peuvent être spécifiées à l'aide de DEFAULT CURRENT_TIMESTAMP et ON UPDATE CURRENT_TIMESTAMP dans la définition de la colonne.
TIME(*fsp*)	Une heure. Format : hh:mm:ss. La plage prise en charge va de « -838:59:59 » à « 838:59:59 ».
YEAR	Une année au format à quatre chiffres. Valeurs autorisées au format à quatre chiffres : 1901 à 2155 et 0000. MySQL 8.0 ne prend pas en charge l'année au format à deux chiffres.

B. Types de données SQL Server

- **Types de données de chaîne**

Data type	**Description**	**Max size**	**Storage**
char(n)	Chaîne de caractères à largeur fixe	8,000 characters	Largeur définie
varchar(n)	Chaîne de caractères à largeur variable	8,000 characters	2 bytes + number of chars
varchar(max)	Chaîne de caractères à largeur variable	1,073,741,824 characters	2 bytes + number of chars
text	Chaîne de caractères à largeur variable	2GB of text data	4 bytes + number of chars
nchar	Chaîne Unicode à largeur fixe	4,000 characters	Defined width x 2
nvarchar	Chaîne Unicode de largeur variable	4,000 characters	
nvarchar(max)	Chaîne Unicode de largeur variable	536,870,912 characters	
ntext	Chaîne Unicode de largeur variable	2GB of text data	
binary(n)	Chaîne binaire à largeur fixe	8,000 bytes	
varbinary	Chaîne binaire à largeur fixe	8,000 bytes	
varbinary(max)	Chaîne binaire à largeur fixe	2GB	
image	Chaîne binaire à largeur fixe	2GB	

- **Types de données numériques**

Data type	Description	Storage
bit	Entier pouvant être 0, 1 ou NULL	
tinyint	Permet les nombres entiers de 0 à 255	1 byte
smallint	Autorise les nombres entiers compris entre -32 768 et 32 767	2 bytes
int	Autorise les nombres entiers compris entre -2 147 483 648 et 2 147 483 647	4 bytes
bigint	Autorise les nombres entiers compris entre -9 223 372 036 854 775 808 et 9 223 372 036 854 775 807	8 bytes
decimal(p,s)	Numéros de précision et d'échelle fixes. Autorise les nombres de -10^38 +1 à 10^38 –1. Le paramètre p indique le nombre total maximum de chiffres pouvant être stockés (à gauche et à droite du point décimal). p doit être une valeur comprise entre 1 et 38. La valeur par défaut est 18. Le paramètre s indique le nombre maximum de chiffres stockés à droite du point décimal. s doit être une valeur comprise entre 0 et p. La valeur par défaut est 0	5-17 bytes
numeric(p,s)	Numéros de précision et d'échelle fixes. Autorise les nombres de -10^38 +1 à 10^38 –1. Le paramètre p indique le nombre total maximum de chiffres pouvant être stockés (à gauche et à droite du point décimal). p doit être une valeur comprise entre 1 et 38. La valeur par défaut est 18. Le paramètre s indique le nombre maximum de chiffres stockés à droite du point décimal. s doit être une valeur comprise entre 0 et p. La valeur par défaut est 0	5-17 bytes
smallmoney	Données monétaires de -214 748,3648 à 214 748,3647	4 bytes
money	Données monétaires de -922 337 203 685 477,5808 à 922 337 203 685 477,5807	8 bytes
float(n)	Données numériques de précision flottantes de -1,79E + 308 à 1,79E + 308. Le paramètre n indique si le champ doit contenir 4 ou 8 octets. float(24) contient un champ de 4 octets et float(53) contient un champ de 8 octets. La valeur par défaut de n est 53.	4 or 8 bytes
real	Données numériques de précision flottantes de -3,40E + 38 à 3,40E + 38	4 bytes

- **Types de données de date et d'heure**

Data type	Description	Storage
datetime	Du 1er janvier 1753 au 31 décembre 9999 avec une précision de 3,33 millisecondes	8 bytes
datetime2	Du 1er janvier 0001 au 31 décembre 9999 avec une précision de 100 nanosecondes	6-8 bytes
smalldatetime	Du 1er janvier 1900 au 6 juin 2079 avec une précision de 1 minute	4 bytes
date	Stockez une date uniquement. Du 1 janvier 0001 au 31 décembre 9999	3 bytes
time	Stockez une heure uniquement avec une précision de 100 nanosecondes	3-5 bytes
datetimeoffset	Identique à datetime2 avec l'ajout d'un décalage de fuseau horaire	8-10 bytes
timestamp	Stocke un numéro unique qui est mis à jour chaque fois qu'une ligne est créée ou modifiée. La valeur d'horodatage est basée sur une horloge interne et ne correspond pas au temps réel. Chaque table ne peut avoir qu'une seule variable d'horodatage	

- **Autres types de données**

Data type	Description
sql_variant	Stocke jusqu'à 8 000 octets de données de différents types de données, à l'exception du texte, du ntext et de l'horodatage
uniqueidentifier	Stocke un identifiant global unique (GUID)
xml	Stocke les données au format XML. Maximum 2 Go
cursor	Stocke une référence à un curseur utilisé pour les opérations de base de données
table	Stocke un ensemble de résultats pour un traitement ultérie

C. Types de données MS Access

Data type	Description	Storage
Text ou texte court	À utiliser pour le texte ou les combinaisons de texte et de chiffres. 255 caractères maximum	

Memo ou texte long	Le mémo est utilisé pour de plus grandes quantités de texte. Stocke jusqu'à 65 536 caractères. Remarque : Vous ne pouvez pas trier un champ mémo. Cependant, ils sont consultables	
Byte	Permet les nombres entiers de 0 à 255	1 byte
Integer	Autorise les nombres entiers compris entre -32 768 et 32 767	2 bytes
Long	Autorise les nombres entiers compris entre -2 147 483 648 et 2 147 483 647	4 bytes
Single	Virgule flottante simple précision. Gérera la plupart des décimales	4 bytes
Double	Virgule flottante double précision. Gérera la plupart des décimales	8 bytes
Currency	Utiliser pour la monnaie. Peut contenir jusqu'à 15 chiffres de dollars entiers, plus 4 décimales. Astuce : Vous pouvez choisir la devise du pays à utiliser	8 bytes
AutoNumber	Les champs AutoNumber attribuent automatiquement à chaque enregistrement son propre numéro, commençant généralement à 1	4 bytes
Date/Time	Utiliser pour les dates et les heures	8 bytes
Yes/No	Un champ logique peut être affiché comme Oui/Non, Vrai/Faux ou Activé/Désactivé. Dans le code, utilisez les constantes True et False (équivalentes à -1 et 0). Remarque : Les valeurs nulles ne sont pas autorisées dans les champs Oui/Non	1 bit
Ole Object	Peut stocker des images, de l'audio, de la vidéo ou d'autres BLOB (Binary Large Objects)	up to 1GB
Hyperlink	Contient des liens vers d'autres fichiers, y compris des pages Web	
Lookup Wizard	Vous permet de saisir une liste d'options, qui peuvent ensuite être choisies dans une liste déroulante	4 bytes

- **Créer une table à l'aide d'une autre table**

Une copie d'une table existante peut également être créée à l'aide de CREATE TABLE.
La nouvelle table obtient les mêmes définitions de colonnes. Toutes les colonnes ou des colonnes spécifiques peuvent être sélectionnées.
Si vous créez une nouvelle table en utilisant une table existante, la nouvelle table sera remplie avec les valeurs existantes de l'ancienne table.

Syntaxe

CREATE TABLE <*Nouvelle table*> AS
SELECT *column1, column2,...*

FROM <*Ancienne table*>
WHERE ....;

Le code SQL suivant crée une nouvelle table appelée « Etudiant » (qui est une copie de la table « Professeur »):

Exemple

CREATE TABLE Etudiant AS
SELECT Id, Nom
FROM Professeur;

- **Instruction ALTER TABLE**

Modifie la structure d'une table.

Syntaxe

- ***Ajouter un champ à une table***

MS Access
ALTER TABLE <*table*> ADD COLUMN <*champ*><*type*> [(<*taille*>)] [<Contrainte>]

Autres SGBD
ALTER TABLE <*table*> ADD <*champ*><*type*> [(<*taille*>)] [<Contrainte>]

- ***Modifier le type, la taille ou la contrainte d'un champ d'une table***

SQL Serveur/MS Access :
ALTER TABLE <*table*> ALTER COLUMN <*champ*><*type*> [(<*taille*>)] [<Contrainte>]

MySQL / Oracle (version antérieure 10G) :
ALTER TABLE <*table*> MODIFY COLUMN <*champ*><*type*> [(<*taille*>)] [<Contrainte>]

Oracle 10G et versions ultérieures :
ALTER TABLE <*table*> MODIFY <*champ*><*type*> [(<*taille*>)] [<Contrainte>]

- ***Supprimer le champ d'une table***
ALTER TABLE <*table*> DROP COLUMN <*champ*>

L'instruction ALTER TABLE se compose des éléments suivants :

Élément	**Description**
table	Nom de la table à modifier.
champ	Nom du champ à ajouter ou à supprimer dans la *table*. Ou nom du champ à modifier dans la *table*.
type	Type de donnée du *champ*.
taille	Taille du champ en caractères (pour les données de type Text ou Char uniquement).

Notes

L'instruction ALTER TABLE permet de modifier une table existante de plusieurs façons. Pour ce faire, vous pouvez :

- Utiliser ADD COLUMN pour ajouter un nouveau champ à la table. Spécifiez alors le nom du champ, son type de données et (dans le cas de champs Text et Char) sa taille (facultative). Par exemple, l'instruction suivante ajoute un champ de texte de 25 caractères appelé Notes (Remarques) à la table Employés (Employés) :

  ALTER TABLE Employés ADD COLUMN Notes TEXT(25)

  Si vous spécifiez l'indication NOT NULL pour un champ, les nouveaux enregistrements doivent contenir des données valides dans ce champ.

- Utilisez ALTER COLUMN pour changer le type de données d'un champ existant. Spécifiez le nom du champ, le nouveau type de données et (dans le cas de champs Text et Char) sa taille (facultative). Par exemple, l'instruction suivante modifie le type de données du champ matricule de la table Employés (Employés) (initialement défini comme Entier) en un champ de type Char de 10 caractères :

  ALTER TABLE Employés ALTER COLUMN matricule CHAR(10)

- Utiliser DROP COLUMN pour supprimer un champ. Spécifiez alors uniquement le nom du champ.

Remarque

Vous ne pouvez pas ajouter, supprimer ou modifier plusieurs champs à la fois.

- **Renommer une colonne**

Pour renommer une colonne dans un tableau, utilisez la syntaxe suivante :

ALTER TABLE <*table*> RENAME COLUMN <*Ancien nom*> to <Nouveau nom>;

- **Supprimer une table : Instruction DROP**

Supprime une table d'une base de données.

Syntaxe

DROP TABLE <*table*>

L'instruction DROP se compose des éléments suivants :

Élément	**Description**
table	Nom de la table à supprimer.

Notes
Vous devez fermer la table avant de la supprimer.

IV - INSTRUCTIONS DML

- **Instruction INSERT INTO**

Ajoute un ou plusieurs enregistrements à une table. C'est ce qu'on appelle une ***requête Ajout***.

Syntaxe

Requête Ajout avec plusieurs enregistrements :

Voir instruction SELECT

Requête Ajout avec un seul enregistrement :

INSERT INTO <nom de la table> [(<*champ1*>,…, <*champN*>)] VALUES (<*valeur1*>,…,< *valeurN*>)

L'instruction INSERT INTO comprend les éléments suivants :

Élément	Description
nom de la table	Nom de la table à laquelle on souhaite ajouter des enregistrements.
champ1, *champN*	Noms des champs auxquels on souhaite ajouter les données.
valeur1, *valeurN*	Valeurs à insérer dans des champs spécifiques du nouvel enregistrement. Chaque valeur vient s'insérer dans le champ correspondant à la position de la valeur dans la liste : *valeur1* s'insère dans le *champ1* du nouvel enregistrement, *valeur2* dans le *champ2* et ainsi de suite. Vous devez séparer les valeurs par une virgule, encadrer les chaines de caractère littérale par des guillemets simples (' ') ou des griffes (" ").

Notes
Si vous ne spécifiez pas tous les champs, la valeur par défaut ou ***Null*** s'insère à la place des colonnes manquantes. Les enregistrements sont ajoutés à la fin de la table.
.

- **Instruction UPDATE**

Crée une ***requête de mise à jour*** qui modifie les valeurs des champs d'une table spécifiée, selon des critères déterminés.

Syntaxe

UPDATE <*table*>
SET <champ1>=<*nouvellevaleur1*>[,……,<champN>=<*nouvellevaleurN*>]
[WHERE <*critère*>];

L'instruction UPDATE comprend les éléments suivants :

Élément	Description

Table	Nom de la table contenant les données que vous voulez modifier.
nouvellevaleur	Expression qui détermine la valeur à insérer dans un champ déterminé des enregistrements mis à jour.
critère	Expression qui détermine quels enregistrements seront mis à jour. Seuls les enregistrements correspondant à l'expression seront mis à jour.

Notes

Vous pouvez modifier simultanément plusieurs champs.

- **Instruction DELETE**

Crée une ***requête Suppression*** qui supprime des enregistrements dans une ou dans plusieurs des tables mentionnées dans la clause ***FROM*** qui correspond à la clause ***WHERE***.

Syntaxe

DELETE
FROM <*table*>
[WHERE <*critère*>]

L'instruction DELETE comprend les éléments suivants :

Élément	Description
table	Nom de la table dans laquelle des enregistrements sont supprimés.
critère	***Expression*** qui détermine quels sont les enregistrements à supprimer.

Notes

Vous pouvez utiliser DELETE pour supprimer des enregistrements impliqués dans une ***relation 1-n*** avec d'autres tables. Les opérations de ***suppression en cascade*** entraînent la suppression des enregistrements des tables placées du côté "plusieurs" de la relation lorsque l'enregistrement correspondant, placé du coté "un" de la relation, est lui-même supprimé de la requête. Par exemple, dans la relation établie entre les tables Clients et Commandes, la table Clients se situe du côté "un" tandis que la table Commandes est du côté "plusieurs". Supprimer un enregistrement dans Clients entraîne la suppression des enregistrements correspondants dans Commandes si l'option de suppression en cascade a été choisie.

Une requête de suppression efface des enregistrements dans leur intégralité et pas seulement les données de certains champs. Si vous désirez supprimer des valeurs contenues dans un champ déterminé, créez une ***requête de mise à jour*** pour remplacer les valeurs par ***Null***.

Important

- Une fois que vous avez supprimé des enregistrements à l'aide d'une requête de suppression ou de mise à jour, vous ne pouvez pas annuler l'opération. Si vous désirez savoir quels enregistrements seront supprimés ou modifiés, faites d'abord une ***requête Sélection*** avec les mêmes critères, examinez les résultats obtenus, puis exécutez la requête Suppression ou mise à jour.
- Conservez toujours des copies de sauvegarde de vos données. Si vous supprimez ou modifiez des enregistrements par inadvertance, vous pourrez les récupérer à partir de vos copies de sauvegarde.

- **Instruction SELECT**

Affiche des informations depuis la base de données sous la forme d'un jeu d'enregistrements.

Syntaxe

SELECT [<*étendue*>] <spécification des champs à afficher>
FROM <*liste des tables*>
[WHERE <Condition de jointure et condition de sélection avant regroupement>]
[GROUP BY <Liste des champs de regroupement>]
[HAVING <Condition de sélection après regroupement>]
[ORDER BY <champ1>[ASC ou DESC],…, <champN>[ASC ou DESC]
L'instruction SELECT se compose des éléments suivants :

Part	**Description**
étendue	L'un des prédicats suivants : ***ALL, DISTINCT, TOP***. Les prédicats permettent de limiter le nombre d'enregistrements renvoyés. Si aucun n'est précisé, ALL est choisi par défaut.
spécification des champs à afficher	Indique les champs qui seront sélectionnés dans le jeu de résultat.
liste des tables	Nom de la ou des tables contenant les données à utiliser dans la requête.

REMAROUE

On peut utiliser l'instruction SELECT pour ajouter un groupe d'enregistrements dans une table.

Syntaxe

Requête Ajout avec plusieurs enregistrements :

INSERT INTO <table *cible*> [(*champ1*[, *champ2*[, ...]])]
SELECT [*source.*]*champ1*[, *champ2*[, ...]
FROM *expressiontable*

L'instruction INSERT INTO comprend les éléments suivants :

Élément	**Description**
nom de la table	Nom de la table à laquelle on souhaite ajouter des enregistrements.
champ1, *champ2*	Noms des champs auxquels on souhaite ajouter les données, s'ils sont placés à la suite de l'argument *cible*, ou noms des champs à partir desquels on souhaite extraire les données, s'ils sont placés à la suite de l'argument *source*.
basededonnéesexterne	Chemin d'accès d'une ***base de données externe***. Pour obtenir une description du chemin, voir la clause ***IN***.
Source	Nom de la table ou de la requête à partir de laquelle on souhaite copier les enregistrements.
expressiontable	Nom de la ou des tables à partir desquelles les enregistrements sont insérés. Cet argument peut être constitué d'un seul nom de table, d'un nom de requête

	enregistrée ou d'un composé résultant d'une opération ***INNER JOIN, LEFT JOIN*** ou ***RIGHT JOIN***.
valeur1, *valeur2*	Valeurs à insérer dans des champs spécifiques du nouvel enregistrement. Chaque valeur vient s'insérer dans le champ correspondant à la position de la valeur dans la liste : *valeur1* s'insère dans le *champ1* du nouvel enregistrement, *valeur2* dans le *champ2* et ainsi de suite. Vous devez séparer les valeurs par une virgule et encadrer les zones de texte par des guillemets simples (' ').

- **Prédicats ALL, DISTINCT, TOP**

Spécifie les enregistrements sélectionnés par des requêtes ***SQL***.

Syntaxe

SELECT [ALL | DISTINCT | [TOP *n* [PERCENT]]]
FROM *table*

Une instruction ***SELECT*** contenant ces prédicats se compose des éléments suivants :

Élément	**Description**
ALL	Sélectionne tous les enregistrements qui remplissent les conditions de l' instruction SQL. Les deux exemples suivants sont équivalents et renvoient tous les enregistrements de la table Employés : SELECT ALL * FROM Employés;
DISTINCT	Omet tous les enregistrements pour lesquels les champs sélectionnés contiennent des données en double. Ainsi, pour être incluses dans les résultats de la requête, les valeurs de chaque champ répertorié dans l'instruction SELECT doivent être uniques. Par exemple, plusieurs employés répertoriés dans une table Employés (Employés) peuvent avoir le même nom. Si deux enregistrements contiennent Smith dans le champ Nom, l'instruction SQL suivante ne renvoie alors qu'un seul de ces enregistrements : SELECT DISTINCT Nom FROM Employés; Si vous omettez DISTINCT, cette requête renvoie les deux enregistrements Smith. Si la clause SELECT contient plusieurs champs, la combinaison des valeurs de tous ces champs doit être unique, pour un enregistrement donné, pour que ces valeurs apparaissent dans le résultat. Le résultat d'une requête qui utilise DISTINCT ne peut pas être mis à jour et ne reflète pas les modifications ultérieures effectuées par d'autres utilisateurs.
TOP *n* [PERCENT]	Renvoie un certain nombre d'enregistrements situés au début ou à la fin d'une plage d'enregistrement. Supposons que vous souhaitiez obtenir les noms des 25 premiers étudiants de la table étudiant :

	SELECT TOP 25 nom, prénom FROM étudiant; Si vous ne triez pas les enregistrements, la requête renverra une série de 25 enregistrements choisis arbitrairement parmi ceux de la table Etudiants qui remplissent les conditions de la clause WHERE si elle existe. Le prédicat TOP n'effectue pas de choix entre des valeurs égales. Dans l'exemple précédent, si, parmi les meilleurs résultats obtenus, le vingt-cinquième et le vingt-sixième ont obtenu la même moyenne, la requête renvoie 26 enregistrements. Vous pouvez également utiliser le mot réservé PERCENT pour renvoyer un certain pourcentage des premiers ou derniers enregistrements d'une plage d'enregistreent. Supposons qu'au lieu des 25 meilleurs étudiants, vous souhaitiez sélectionner 10 pour cent des étudiants: SELECT TOP 10 PERCENT nom, prénom FROM étudiant; La valeur qui suit TOP doit être un ***entier*** non signé. TOP n'affecte pas les possibilités de mise à jour de la requête.
table	Nom de la table contenant les enregistrements à extraire.

- **Spécification des champs à sélectionner**

Syntaxe

<spécification des champs à sélectionner>={ * | *table*.* | [*table*.]*champ1* [AS *alias1*] [, [*table*.]*champ2* [AS *alias2*] [, ...]]}

- utilisez l'astérisque pour sélectionner tous les champs de toutes les tables utilisées dans la requête.

- <table>.* permet de sélectionner tous les champs d'une table donnée.

- [<*table*>.]<*champ*> permet de lister les champs à sélectionner. Le nom de la table est facultatif. Il n'est nécessaire que lorsqu'un champ à sélectionner se retrouve dans deux tables utilisées dans la requête.

- [AS *alias1*] permet chaque fois que vous utilisez des ***fonctions d'agrégation*** ou des requêtes qui renvoient des noms d'attributs ou d'expressions d'attributs ambigus ou en double, de fournir un nom de remplacement assez explicite.

- **Clause FROM**

Spécifie les tables ou les requêtes qui contiennent les champs spécifiés dans l'instruction ***SELECT***.

Syntaxe

SELECT *listechamp*
FROM <*liste des tables*>

Une instruction SELECT contenant une clause FROM se compose des éléments suivants :

Élément	**Description**
Listechamp	Noms du ou des champs à extraire avec leurs ***alias*** de nom de champ, ***fonctions d'agrégation SQL***, prédicats de sélection (***ALL, DISTINCT, DISTINCTROW, ou TOP***) ou autres options d'instruction SELECT.
liste des tables	Expression qui identifie la ou les tables contenant les données à extraire. L'expression peut être constituée d'un seul nom de table, d'un nom de requête enregistrée ou d'un composé résultant d'une opération ***INNER JOIN***, ***LEFT JOIN*** ou ***RIGHT JOIN***.

Notes

Expressiontable peut être :

- Une table : dans ce cas, les données à utiliser dans cette requête proviennent toutes de cette table
- *Plusieurs tables : dans ce cas une jointure s'impose. Pour exprimer cette jointure, on peut :*

- *Lister les tables et spécifier les conditions de jointure dans la clause **where.** Exemple : afficher les nom et prénom des étudiants avec leur libellé de filière.*
- *Utiliser l'opération **INNER JOIN** qui Fusionne les enregistrements de deux tables lorsqu'un champ commun contient des valeurs identiques.*

Syntaxe

FROM *table1* INNER JOIN *table2* ON *table1.champ1 oprcomp table2.champ2*

L'opération INNER JOIN comprend les éléments suivants :

Élément	**Description**
table1, *table2*	Noms des tables depuis lesquelles les enregistrements sont fusionnés.
champ1, *champ2*	Noms des champs reliés entre eux. S'ils ne sont pas numériques, les champs doivent avoir le même ***type de données*** et contenir des données de nature identique, mais leurs noms peuvent être différents.
Oprcomp	Tout opérateur de comparaison relationnelle tel que "=","<", ">", "<=", ">=", ou "<>". "=," "<," ">," "<=," ">=," ou "<>." Mais égal est le plus utilisé.

- **Les autres types de jointure :**

a. Jointure externe gauche et droite : LEFT JOIN et RIGHT JOIN

Pour avoir dans le jeu de résultats, toutes les occurrences de l'une des tables et seulement celles de l'autre table pour lesquelles les champs joints sont égaux, on effectue une jointure externe gauche avec l'opérateur LEFT JOIN ou une jointure externe droite avec l'opérateur RIGHT JOIN.

Une jointure LEFT JOIN ou RIGHT JOIN peut être imbriquée dans une jointure INNER JOIN mais il est impossible d'imbriquer une jointure INNER JOIN dans une jointure LEFT JOIN ou RIGHT JOIN

b. Jointure complète

Vous pouvez aussi utiliser l'opérateur FULL OUTER JOIN pour renvoyer tous les enregistrements lorsqu'il existe une correspondance dans les enregistrements de la table de gauche (table1) ou de droite (table2) et même les enregistrements sans correspondance des deux tables.

En somme, il est une combinaison de LEFT JOIN et de RIGTH JOIN

Astuce : FULL OUTER JOIN et FULL JOIN sont les mêmes.

Syntaxe

SELECT <Liste des attributs> FROM table1 FULL OUTER JOIN table2
ON table1.column_name = table2.column_name
WHERE <condition> ;

Remarque : FULL OUTER JOIN renvoie tous les enregistrements correspondants des deux tables, que l'autre table corresponde ou non. Ainsi, s'il y a des lignes dans « Table1 » qui n'ont pas de correspondances dans « Table2 », ou s'il y a des lignes dans « Table2 » qui n'ont pas de correspondances dans « Table1 », ces lignes seront également sélectionnées.

La clause FROM est requise et doit figurer à la suite de toute instruction SELECT.

- **Clause WHERE**

Indique quels enregistrements des tables citées dans la clause ***FROM*** sont affectés par l'instruction ***SELECT*** **.**

Syntaxe

SELECT *listechamp*
FROM *expressiontable*
WHERE *critère*

Une instruction SELECT contenant une clause WHERE se compose des éléments suivants :

Élément	**Description**
Listechamp	Nom du ou des champs à extraire avec leurs ***alias*** de nom de champ, prédicats de sélection (***ALL, DISTINCT, DISTINCTROW, ou TOP***) ou autres options d'instruction SELECT.
expressiontable	Nom de la ou des tables contenant les données à extraire.
Critère	***Expression*** que les enregistrements doivent satisfaire pour être inclus dans le résultat de la requête.

Une clause WHERE peut contenir jusqu'à 40 expressions liées par des opérateurs logiques comme **And** et **Or**.

Lorsque vous entrez un nom de champ contenant un espace ou un signe de ponctuation, placez ce nom entre crochets ([ ]).
Si vous spécifiez l'argument *critère*, vous devez indiquer les ***littéraux date*** entre deux caractères #.
Lorsque vous utilisez des valeurs littérales de type caractère ou texte, encadrez les de griffes (")

LES OPERATEURS SQL

Quelques définitions

Expression

Toute combinaison d'opérateurs, de constantes, de valeurs littérales, de fonctions et de noms de champs (colonnes), de contrôles et de propriétés dont l'évaluation fournit une valeur unique. Vous pouvez utiliser les expressions en tant que paramètres pour un grand nombre de propriétés et d'arguments d'action ; pour définir des contrôles calculés dans des formulaires, des états et des pages d'accès aux données ; pour définir des critères (conditions de recherche) ou définir des champs calculés dans des requêtes ; et pour fixer des conditions dans des macros. Dans un projet Microsoft Access, les expressions vous permettent de définir des contraintes de validation.

Opérateur

Symbole ou mot, tel que > et Ou, qui indique l'opération à effectuer sur un ou plusieurs éléments. Microsoft Access possède plusieurs classes d'opérateurs, qui sont les opérateurs arithmétiques, logiques, de comparaison et de concaténation.

Critères

Ensemble de conditions limitatives, par exemple = « Danemark » (c'est-à-dire égal à Danemark) ou >30000, utilisées lors de la création d'une requête ou d'un filtre pour afficher un jeu spécifique d'enregistrements.

Les opérateurs

- **Opérateur Between...And**

Détermine si la valeur d'une expression est comprise dans un intervalle de valeurs donné. Vous pouvez utiliser cet opérateur à l'intérieur d'une instruction SQL

Syntaxe

<*Expression*> [**Not**] **Between** <*valeur1*> **And** <*valeur2*>

La syntaxe de l'opérateur **Between...And** comprend les éléments suivants :

Élément	Description
<*expression*>	Expression qui identifie le champ contenant les données que vous voulez évaluer.
valeur1, *valeur2*	Expressions auxquelles vous désirez confronter *expression*.

Remarques

Si la valeur de *expression* est comprise entre *valeur1*et *valeur2* (incluses), l'opérateur **Between...And** renvoie **True** ; sinon il renvoie **False**. Vous pouvez ajouter l'opérateur logique **Not** pour évaluer la condition contraire (autrement dit, savoir si *expression* se situe hors de l'intervalle défini par *valeur1*et *valeur2*).

- **Opérateur In**

Détermine si la valeur d'une expression est égale à l'une des valeurs comprises dans une liste donnée.

Syntaxe
<*expression*> [**Not**] **In(<***valeur1*>,< *valeur2*>, . . .,<*valeur n*>**)**

Remarques
La syntaxe de l'opérateur **In** comprend les éléments suivants :

Élément	**Description**
expression	Expression qui identifie le champ contenant les données que vous voulez évaluer.
valeur1, *valeur2*	Expression ou liste d'expressions que vous désirez confronter à <*expression*>.

Si on trouve <*expression*> dans la liste des valeurs, l'opérateur **In** renvoie la valeur **True**, dans le cas contraire, il renvoie **False**. Vous pouvez ajouter l'opérateur logique **Not** pour évaluer la condition contraire (autrement dit, savoir si <*expression*> ne figure pas dans la liste de valeurs communiquée.
Par exemple, vous pouvez utiliser l'opérateur **In** pour déterminer quelles commandes sont expédiées vers un ensemble de régions données.

```
SELECT *
FROM
WHERE [Région livraison] In ('zou','Collines','atlantique')
```

- **Opérateur Like**

Compare une expression chaîne avec un modèle dans une expression SQL.

Syntaxe
<*expression*> **Like** <*modèle*>
La syntaxe de l'opérateur **Like** comprend les éléments suivants :

Élément	**Description**
expression	Expression SQL utilisée dans une clause WHERE.
modèle	Chaîne ou chaîne de caractères constante à laquelle *expression* est comparée.

Remarques
Vous pouvez utiliser l'opérateur **Like** pour rechercher dans un champ des valeurs correspondant au modèle que vous spécifiez. Pour *modèle*, vous pouvez spécifier la valeur complète, par exemple, Like "Martin"), ou vous pouvez utiliser des caractères génériques pour rechercher un intervalle de valeurs (par exemple Like "Ma*").
Dans une expression, vous pouvez utiliser l'opérateur **Like** pour comparer une valeur de champ avec une expression de chaîne. Par exemple, si vous spécifiez Like "C*" dans une requête SQL, la requête renvoie toutes les valeurs de champ commençant par la lettre C. Dans une requête Paramètre, vous pouvez demander à l'utilisateur de préciser un modèle de chaîne à rechercher.
Dans l'exemple suivant, une requête renvoie les données commençant par la lettre P, suivie d'une lettre comprise entre A et F puis de trois chiffres :
Like "P[A-F]###"
Le tableau suivant vous montre comment utiliser **Like** pour évaluer des expressions avec différents modèles.

Type d'occurrence	**Modèle**	**Occurrence (renvoie True)**	**Aucune occurrence (renvoie False)**
Plusieurs caractères	a*a	aa, aBa, aBBBa	aBC
	* ab*	abc, AABB, Xab	aZb, bac
Caractère spécial	a[*]a	a*a	aaa
Plusieurs caractères	ab*	abcdefg, abc	cab, aab
Caractère isolé	a?a	aaa, a3a, aBa	aBBBa

Chiffre isolé	a#a	a0a, a1a, a2a	aaa, a10a
Intervalle de caractères	[a-z]	f, p, j	2, &
Hors d'un intervalle	[!a-z]	9, &, %	b, a
Autre qu'un chiffre	[!0-9]	A, a, &, ~	0, 1, 9
Combinaison	a[!b-m]#	An9, az0, a99	abc, aj0

Les caractères génériques
L'astérisque (*), le point d'interrogation (?), le signe dièse (#), le point d'exclamation (!), le trait d'union (-) et les crochets ([ ]) sont des caractères génériques. Vous pouvez utiliser ces caractères dans des requêtes et des expressions pour inclure tous les enregistrements, noms de fichier ou autres éléments dont l'intitulé commence par certains caractères ou correspond à une séquence particulière. Vous pouvez également utiliser les caractères génériques et les caractères correspondants pour affiner une recherche dans le cadre d'une instruction SQL.

Microsoft Access

Symbole	**Exemple**	**Utilisation**
*	*wh** trouve what, white et why **at* trouve cat, bat et what	Représente un nombre quelconque de caractères et peut être placé en tout point de la chaîne de caractères.
?	*b?ll* trouve ball, bell et bill	Représente un seul caractère.
#	*1#3* trouve 103, 113, 123	Représente un seul chiffre.
[ ]	*b[ae]ll* trouve ball et bell mais pas bill	Représente un seul caractère parmi ceux indiqués entre crochets.
!	*b[!ae]ll* trouve bill et bull mais pas bell ni ball	Représente tout caractère ne figurant pas entre crochets.
-	*b[a-c]d* trouve bad, bbd et bcd	Représente l'un des caractères de la plage indiquée.

Autres SGBD

Symbole	**Exemple**	**Utilisation**
%	*Wh%* trouve what, white et why *%at* trouve cat, bat et what	Représente un nombre quelconque de caractères et peut être placé en tout point de la chaîne de caractères.
_	*B_ll* trouve ball, bell et bill	Représente un seul caractère. (tiret de 8 ou underscore)
[ ]	*b[ae]ll* trouve ball et bell mais pas bill	Représente un seul caractère parmi ceux indiqués entre crochets. *
^	*b[^ae]ll* trouve bill et bull mais pas bell ni ball	Représente tout caractère ne figurant pas entre crochets. *
-	*b[a-c]d* trouve bad, bbd et bcd	Représente l'un des caractères de la plage indiquée. *
{}		Représente tout caractère échappé**

Remarque

* Non pris en charge dans les bases de données PostgreSQL et MySQL.

** Pris en charge uniquement dans les bases de données Oracle.

Les caractères * (astérisque), ? (point d'interrogation), # (dièse), [(crochet ouvrant) peuvent être considérés comme des caractères ordinaires dans une expression uniquement s'ils sont entourés de crochets.

- **Clause GROUP BY**

La clause GROUP BY combine en un seul enregistrement les enregistrements comportant des valeurs identiques dans la liste des champs spécifiés. Une valeur de synthèse est créée pour chaque enregistrement si vous incluez une ***fonction d'agrégation SQL***, par exemple **Sum** ou **Count**, dans l'instruction ***SELECT***.

Syntaxe

SELECT *listechamp*
FROM *table*
WHERE *critère*
[GROUP BY *grouplistechamp*]

Une instruction SELECT contenant une clause GROUP BY se compose des éléments suivants :

Élément	**Description**
listechamp	Noms du ou des champs à extraire avec leurs ***alias*** de nom de champ, ***fonctions d'agrégation SQL***, prédicats de sélection ***(ALL, DISTINCT ou TOP***) ou autres options d'instruction SELECT.
Table	Nom de la table contenant les enregistrements à extraire. Pour plus d'informations, reportez-vous à la rubrique relative à la clause ***FROM***.
Critère	Critères de sélection. Si l'instruction contient une clause ***WHERE***, SQL regroupe les valeurs après avoir appliqué les conditions WHERE aux enregistrements.
listechampgroupe	Noms des champs (10 au maximum) utilisés pour regrouper les enregistrements. L'ordre des noms de champs dans *listechampgroupe* détermine les niveaux de regroupement, du plus élevé au plus bas.

Notes

La clause GROUP BY est facultative.
Les valeurs de synthèse sont omises si l'instruction SELECT ne contient aucune fonction d'agrégation SQL.
Les valeurs ***Null*** des champs GROUP BY sont regroupées et ne sont pas omises. Toutefois, les valeurs **Null** ne sont évaluées dans aucune fonction d'agrégation SQL.

Utilisez la clause WHERE pour exclure les lignes que vous ne souhaitez pas regrouper

Tous les champs de la liste de champs SELECT doivent être inclus dans la clause GROUP BY ou, sous forme d'arguments, dans une fonction d'agrégation SQL.

Les fonctions d'agrégation

➢ Fonctions First et Last

Renvoient la valeur d'un champ du premier ou du dernier enregistrement contenu dans le jeu de résultats renvoyé par une requête.

Syntaxe
First(<*expression*>**)**
Last(<*expression*>**)**
L'argument <*expression*> représente une expression de chaîne qui identifie le champ contenant les données que vous souhaitez utiliser ou une expression qui exécute un calcul sur les données contenues dans ce champ. Les opérandes associés à <*expression*> peuvent comprendre le nom d'un champ de table, une constante ou une fonction (qui peut être soit intrinsèque, soit définie par l'utilisateur, à l'exclusion de toute autre fonction d'agrégation SQL).

➢ Fonction Avg

Calcule la moyenne arithmétique d'un ensemble de valeurs contenues dans un champ spécifié dans une requête.

Syntaxe
Avg(<*expression*>**)**
L'argument <*expression*> représente une expression de chaîne qui identifie le champ contenant les données numériques dont vous souhaitez calculer la moyenne ou une expression qui exécute un calcul sur la base des données contenues dans ce champ. Les opérandes associés à <*expression*> peuvent comprendre le nom d'un champ de table, une constante ou une fonction (qui peut être soit intrinsèque, soit définie par l'utilisateur, à l'exclusion de toute autre fonction d'agrégation SQL).

Remarques
La moyenne calculée par **Avg** est la moyenne arithmétique (la somme des valeurs divisée par le nombre de valeurs). Vous pouvez utiliser **Avg**, par exemple, pour calculer le coût moyen du transport des marchandises. La fonction **Avg** n'intègre aucun champ **Null** à ses calculs.

➢ Fonction Count

Calcule le nombre d'enregistrements renvoyés par une requête.

Syntaxe
Count(<*expression*>**)**
L'argument <*expression*> représente une expression de chaîne qui identifie le champ contenant les données dont vous souhaitez obtenir le nombre ou une expression qui exécute un calcul sur les données contenues dans ce champ. Les opérandes associés à <*expression*> peuvent comprendre le nom d'un champ de table ou une fonction (qui peut être soit intrinsèque, soit définie par l'utilisateur, à l'exclusion de toute autre fonction d'agrégation SQL). Vous pouvez compter toutes sortes de données, y compris les données de type text.

Remarques
Vous pouvez utiliser la fonction **Count** pour compter le nombre d'enregistrements présents dans une requête sous-jacente. Par exemple, vous pouvez utiliser la fonction **Count** pour compter le nombre de commandes expédiées vers un pays donné.
Bien que l'argument <*expression*> puisse exécuter un calcul sur un champ, la fonction **Count** ne fait que dénombrer les enregistrements. Le type des valeurs contenues dans les enregistrements est sans importance.

La fonction **Count** ne compte pas les enregistrements possédant des champs **Null** sauf si l'argument <*expression*> est remplacé par le caractère générique astérisque (*). Si vous utilisez un astérisque, la fonction **Count** calcule le nombre total des enregistrements en incluant ceux qui contiennent des champs **Null**. **Count(*)** s'exécute beaucoup plus rapidement que **Count(**[*nom_de_colonne*]**)**. L'astérisque doit être entré sans apostrophes (' ').
Si <*expression*> identifie plusieurs champs, la fonction **Count** ne compte un enregistrement que si au moins un de ses champs n'est pas **Null**. Si tous les champs spécifiés sont **Null**, l'enregistrement n'est pas comptabilisé. Les noms des champs doivent être séparés par une perluète (&).

- **Fonction Sum**

Renvoie la somme d'une série de valeurs contenues dans un champ spécifié dans une requête.

Syntaxe
Sum(<*expression*>**)**
L'argument <*expression*> représente une expression de chaîne qui identifie le champ contenant les données numériques dont vous souhaitez calculer la somme ou une expression qui exécute un calcul sur la base des données contenues dans ce champ. Les opérandes associés à <*expression*> peuvent comprendre le nom d'un champ de table, une constante ou une fonction (qui peut être soit intrinsèque, soit définie par l'utilisateur, à l'exclusion de toute autre fonction d'agrégation SQL).

Remarques
La fonction **Sum** totalise les valeurs contenues dans un champ. Par exemple, vous pouvez utiliser la fonction **Sum** pour déterminer le coût total des frais de transport.
La fonction **Sum** ignore les enregistrements contenant des champs Null.

- **Fonctions Min et Max**

Renvoient la valeur minimale ou maximale d'une série de valeurs contenues dans un champ spécifié dans une requête.

Syntaxe
Min(<*expression*>**)**
Max(<*expression*>**)**
L'argument <*expression*> représente une expression de chaîne qui identifie le champ contenant les données que vous souhaitez évaluer ou une expression qui exécute un calcul sur les données contenues dans ce champ. Les opérandes associés à <*expression*> peuvent comprendre le nom d'un champ de table, une constante ou une fonction (qui peut être soit intrinsèque, soit définie par l'utilisateur, à l'exclusion de toute autre fonction d'agrégation SQL).

Remarques

Vous pouvez utiliser les fonctions **Min** et **Max** pour déterminer les valeurs les plus basses et les plus élevées contenues dans un champ en fonction du mode d'agrégation spécifié. Par exemple, vous pouvez utiliser ces fonctions pour renvoyer les coûts de transport des marchandises les plus bas et les plus élevés. Si aucune agrégation n'est spécifiée, toute la table est utilisée.

- **Clause HAVING**

Spécifie les enregistrements regroupés à afficher dans une instruction ***SELECT*** dotée d'une clause GROUP BY. Une fois que GROUP BY a regroupé les enregistrements, HAVING affiche tous les

enregistrements qui ont été regroupés par la clause ***GROUP BY*** et qui répondent aux critères de la clause HAVING.

Syntaxe

SELECT *listechamp*
FROM *table*
WHERE *critèresélection*
GROUP BY *listechampgroupe*
[HAVING *critèregroupe]*

Une instruction SELECT contenant une clause HAVING se compose des éléments suivants :

Élément	**Description**
listechamp	Noms du ou des champs à extraire avec leurs ***alias*** de nom de champ, ***fonctions d'agrégation SQL***, prédicats de sélection (***ALL, DISTINCT ou TOP***) ou autres options d'instruction SELECT.
Table	Nom de la table contenant les enregistrements à extraire. Pour plus d'informations, reportez-vous à la rubrique relative à la clause ***FROM.***
critèresélection	Critères de sélection. Si l'instruction contient une clause ***WHERE***, le ***moteur de base de données Microsoft Jet*** regroupe les valeurs après avoir appliqué les conditions WHERE aux enregistrements.
listechampgroupe	Noms des champs (10 au maximum) utilisés pour regrouper les enregistrements. L'ordre des noms de champs dans *listechampgroupe* détermine les niveaux de regroupement, du plus élevé au plus bas.
critèregroupe	Expression qui détermine quels sont les enregistrements regroupés à afficher.

Notes

La clause HAVING est facultative.

HAVING est similaire à WHERE, qui détermine quels sont les enregistrements à sélectionner. Une fois que GROUP BY a regroupé les enregistrements, HAVING détermine quels sont les enregistrements qui seront affichés.
Une clause HAVING peut contenir jusqu'à 40 expressions liées par des opérateurs logiques comme **And** et **Or**.

- **Clause ORDER BY**

Trie, par ordre croissant ou décroissant, les enregistrements résultants d'une requête en fonction du ou des champs spécifiés.

Syntaxe

SELECT *listechamp*
FROM *table*
WHERE *critèressélection*
[ORDER BY *champ1* [ASC | DESC][, *champ2* [ASC | DESC]][, ...]]

Une instruction SELECT contenant une clause ORDER BY se compose des éléments suivants :

Élément	**Description**
listechamp	Noms du ou des champs à extraire avec leurs ***alias*** de nom de champ, ***fonctions d'agrégation SQL***, prédicats de sélection (***ALL, DISTINCT, ou TOP***) ou autres options d'instruction ***SELECT***.
Table	Nom de la table contenant les enregistrements à extraire. Pour plus d'informations, reportez-vous à la rubrique relative à la clause ***FROM***.
critèresélection	Critères de sélection. Si l'instruction contient une clause ***WHERE***, le ***moteur de base de données Microsoft Jet*** trie les valeurs après avoir appliqué les conditions WHERE aux enregistrements.
champ1, *champ2*	Noms des champs sur lesquels doit porter le tri.

Notes

La clause ORDER BY est facultative. Mais si vous n'incluez pas ORDER BY, les données sont affichées sans être triées.
L' ordre de tri par défaut est l'ordre croissant (de A à Z et de 0 à 9). Dans les deux exemples suivants, les employés sont triés sur leur nom :

```
SELECT Name, FirstName
FROM Employés
ORDER BY FirstName;
```

OU

```
SELECT Name, FirstName
FROM Employés
ORDER BY FirstName ASC;
```

Pour effectuer un tri par ordre décroissant (de Z à A et de 9 à 0), ajoutez le mot réservé DESC à la fin de chaque champ que vous souhaitez trier par ordre décroissant. Dans les deux exemples suivants, les employés sont sélectionnés, puis triés par ordre décroissant :

```
SELECT Name, Salariés
FROM Employés
ORDER BY Salariés DESC, Name;
```

Si vous spécifiez un champ contenant des données de type ***Memo*** ou ***OLE Object*** dans la clause ORDER BY, une erreur se produit. Le moteur de base de données Microsoft Jet ne peut pas trier des champs contenant ces types de données.

ORDER BY est généralement le dernier élément d'une ***instruction SQL***.

Vous pouvez inclure des champs supplémentaires dans une clause ORDER BY. Les enregistrements sont triés tout d'abord en fonction du premier champ spécifié après ORDER BY. Ensuite, les enregistrements pour lesquels ce champ contient des valeurs identiques sont triés en fonction de la valeur du second champ spécifié, et ainsi de suite.

V. SOUS-REQUETES SQL

Une sous-requête se compose d'une instruction ***SELECT*** imbriquée dans une autre instruction SELECT***, INSERT...INTO, DELETE*** ou ***UPDATE, etc.***, ou encore dans une autre sous-requête.

1. Les sous requêtes simples

Ce sont les sous requêtes utilisées dans la clause WHERE ou HAVING d'une requête pour fournir à celle-ci des informations de comparaison dont la requête principale a besoin pour s'exécuter

Syntaxe

Vous pouvez utiliser trois variantes de syntaxe pour créer une sous-requête simple :

WHERE ou HAVING <attribut> <opérateur de comparaison> [ANY | ALL | SOME] (*instructionsql*)
WHERE ou HAVING <attribut> [NOT] IN (*instructionsql*)
WHERE ou HAVING [NOT] EXISTS (*instructionsql*)

Une sous-requête comprend les éléments suivants :

Élément	**Description**
comparaison	***Expression*** et opérateur de comparaison qui compare l'expression avec les résultats de la sous-requête.
expression	Expression que l'on recherche dans le jeu de résultats de la sous-requête.
instructionsql	Instruction SELECT respectant le format et les règles conventionnelles des autres instructions SELECT. Elle doit être encadrée par des parenthèses.

Notes

Vous pouvez utiliser une sous-requête au lieu d'une expression, dans la liste de champs d'une instruction SELECT ou dans une clause ***WHERE*** ou ***HAVING***. Dans une sous-requête, vous utilisez une instruction SELECT pour fournir un jeu d'une ou plusieurs valeurs spécifiques à évaluer dans l'expression de la clause WHERE ou HAVING.

Utilisez les prédicats ANY ou SOME (l'effet est identique) pour rechercher par comparaison les enregistrements de la requête principale en correspondance avec n'importe quel enregistrement de la sous-requête. Dans l'exemple suivant, la requête renvoie tous les produits (products) dont le prix unitaire (UnitPrice) est supérieur au prix de n'importe quel produit vendu avec une remise (discount) de 25 pour cent ou davantage.

```
SELECT * FROM Products
WHERE UnitPrice > ANY
(SELECT UnitPrice FROM OrderDetails
WHERE Discount >= .25);
```

Utilisez le prédicat ***ALL*** pour rechercher par comparaison les enregistrements de la requête principale en correspondance avec tous les enregistrements de la sous-requête. Si dans l'exemple précédent, vous avez remplacé ANY par ALL, la requête ne renvoie que les produits dont le prix unitaire est supérieur aux prix de tous les produits vendus avec une remise de 25 pour cent ou davantage. La recherche est plus restrictive.

Utilisez le prédicat IN pour rechercher les enregistrements de la requête principale pour lesquels on trouve des enregistrements avec une valeur identique dans la sous-requête. Dans l'exemple suivant, la requête renvoie tous les produits vendus avec une remise de 25 pour cent ou davantage.

```
SELECT * FROM Products
WHERE ProductID IN
(SELECT ProductID FROM OrderDetails
WHERE Discount >= .25);
```

Inversement, vous pouvez utiliser NOT IN pour rechercher les enregistrements de la requête principale pour lesquels on ne trouve aucun enregistrement avec une valeur identique dans la sous-requête.

Utilisez le prédicat EXISTS (avec le mot réservé facultatif NOT) dans des comparaisons vrai/faux pour déterminer si la sous-requête renvoie des enregistrements.

Vous pouvez également utiliser des ***alias*** de nom de table dans une sous-requête, pour faire référence à des tables répertoriées dans une clause ***FROM*** située hors de la sous-requête. Dans l'exemple suivant, la requête renvoie les noms des employés dont les salaires sont supérieurs ou égaux au salaire moyen de l'ensemble des employés ayant la même qualité. L'alias affecté à la table Employés est "T1" :

```
SELECT Name
First name, Title, Salariés
FROM Employés AS T1
WHERE Salariés >=
(SELECT Avg(Salariés)
FROM Employés
WHERE T1.Title = Employés.Title) Order by Title;
```

Dans l'exemple précédent, le ***mot réservé*** AS est facultatif.

Certaines sous-requêtes sont autorisées dans les ***requêtes croisées*** mais uniquement comme prédicats (ceux de la clause WHERE). Les sous-requêtes ne sont pas autorisées dans les requêtes croisées pour ce qui concerne les sorties (dans la liste de SELECT).

2. Les Sous requêtes de la clause FROM

Elles sont utilisées pour fournir à une requête des enregistrements ne provenant pas d'une table.

Syntaxe

FROM <table1>, <Sous-requête> <alias>, …..
Soit les tables Etudiant et filière
Etudiant

Matricule	Nom	Sexe	Codfil
1	Magengo	M	IRT
2	Djangoni	F	MMI
3	Mandelai	F	MMI
4	Séchard	M	IRT
5	Mahigôh	M	MMI
6	Bitôh	F	IRT

Filière

CodFil	Libfil	Matricule(Responsable)
MMI	Management international	3
IRT	Ingénierie informatique et Réseau	6

Le matricule dans la table filière représente le matricule de l'étudiant responsable de la filière

On désire afficher pour chaque étudiant, son nom, le code de sa filière et le nom du responsable.

Le seul attribut Nom de la table Etudiant représente aussi bien les nom des étudiants que ceux des responsables. On est alors obligé de dissocier les responsables des autres étudiants à l'aide d'une requête avant de pouvoir les afficher distinctement.

Select E.Nom AS [Nom étudiant], R.Nom as [Nom respo], E.Codfil
From Etudiant E,(Select Nom, Codfil from Etudiant,filière Where Etudiant.Matricule=Filière.Matricule) R
Where E.Codfil=R.Codfil

Elle peut aussi être utilisée pour effectuer un comptage sans répétition (dans le cas d'une division par exemple)

3. L'Union, l'Intersection et la Différence

- **UNION**

L'objectif de la commande **UNION** de SQL est de combiner les résultats de deux requêtes. Une restriction de **UNION** est que toutes les colonnes correspondantes doivent inclure le même type de données. Aussi, lors de l'utilisation de **UNION**, seules les valeurs distinctes sont sélectionnées (similaire à **SELECT DISTINCT**).

La syntaxe est comme suit :

[instructions SQL 1]
UNION
[instructions SQL 2]

Supposons que nous avons les deux tables suivantes :

Table ***Etudiant***

Matricule	Nom	Prénom
1	Magengo	Guttembert
2	Djangoni	Mandelai
3	Plakaly	Monyo

Table ***Prof***

NumProf	Nom	Tel
1	Plakaly	65666768
2	Djangoni	95959897
3	Gansê	90959192

4	Madjrè	89988860

et pour retrouver les numéro et nom de tous les profs et les étudiants il faudra utiliser l'instruction SQL suivante :

SELECT Numprof, nom FROM Etudiant
UNION
SELECT Matricule, nom FROM Prof

Résultat :

Matricule	Nom
1	Magengo
2	Djangoni
3	Plakaly
1	Plakaly
3	Gansê
4	Madjrè

Notez qu'en saisissant « **SELECT DISTINCT**» pour les deux instructions SQL, nous obtiendrons le même jeu de résultats.

L'objectif de la commande **UNION ALL** de SQL est aussi de combiner ensemble les résultats de deux requêtes. La différence entre **UNION ALL** et **UNION** est la suivante : **UNION** sélectionne seulement des valeurs distinctes et **UNION ALL** sélectionne toutes les valeurs.

Remarque
Il est possible après une union de créer un attribut et de lui affecter des valeurs pour différencier les occurrences du résultat de l'union

Syntaxe

SELECT <attribut1>,…,<attributN>, <valeur> AS <Nouvel attribut> FROM <Table1>
UNION
SELECT <attribut1>,…,<attributN>, <valeur> FROM <Table2>

Exemple

SELECT Numprof, nom, 'Etudiant' AS Type FROM Etudiant
UNION
SELECT Matricule, nom, 'Professeur' FROM Prof

- **INTERSECT (Intersection en algèbre relationel)**

À l'instar de la commande **UNION**, **INTERSECT** opère aussi sur deux instructions SQL. La différence entre les deux commandes est la suivante : **UNION** agit essentiellement comme un opérateur **OR (OU)** operator (la valeur est sélectionnée si elle apparaît dans la première ou la deuxième instruction), la commande **INTERSECT** agit comme un opérateur **AND (ET)** operator (la valeur est sélectionnée seulement si elle apparaît dans les deux instructions).

La syntaxe est comme suit :

[instructions SQL 1]
INTERSECT
[instructions SQL 2]

En utilisant les deux tables précédentes et pour retrouver tous les étudiants qui sont encore des profs, il faudra utiliser l'instruction SQL suivante :

SELECT Numprof, nom FROM Etudiant
INTERSECT
SELECT Matricule, nom FROM Prof

Résultat :

NumProf	Nom
2	Djangoni

Donner le résultat de la requête :

SELECT nom FROM Etudiant
INTERSECT
SELECT nom FROM Prof

Notez que la commande **INTERSECT** ne retournera que des valeurs distinctes.

- **MINUS (différence en algèbre relationnelle)**

MINUS opère sur deux instructions SQL. Elle prend tous les résultats de la première instruction SQL, puis soustrait ceux de la deuxième instruction SQL pour obtenir la réponse finale. Si la deuxième instruction SQL comprend des résultats qui ne sont pas inclus dans la première instruction SQL, ils seront ignorés.

Syntaxe
[instructions SQL 1]
MINUS
[instructions SQL 2]

Poursuivons avec le même exemple pour trouver tous les étudiants qui ne sont pas des profs, il faudra utiliser l'instruction SQL suivante :

SELECT Numprof, nom FROM Etudiant
MINUS
SELECT Matricule, nom FROM Prof

Notez que la commande **MINUS** ne retournera que des valeurs distinctes.

VI. AUTRES INSTRUCTIONS SQL

1. L'expression SQL CASE

L'expression CASE s'inscrit dans le groupe des expressions d'attribut qui permettent de générer un nouvel attribut. Elle passe par des conditions et renvoie une valeur lorsque la première condition est remplie (comme une instruction if-then-else). Ainsi, une fois qu'une condition est vraie, elle arrêtera la lecture et renverra le résultat. Si aucune condition n'est vraie, elle renvoie la valeur dans la clause ELSE.
S'il n'y a aucune partie ELSE et qu'aucune condition n'est vraie, il renvoie NULL.

Syntaxe

```
CASE
   WHEN condition1 THEN result1
   WHEN condition2 THEN result2
   WHEN conditionN THEN resultN
   ELSE result
END [AS <alias>]
```

Exemple

```
Select Matricule, Nom,
case
        When sexe='F' then 'Féminin'
        When sexe='M' then 'Masculin'
        Else 'Pas de sexe'
END AS Sexe

from Etudiant;
```

2. Commentaires SQL

Les commentaires sont utilisés pour expliquer des sections d'instructions SQL ou pour empêcher l'exécution d'instructions SQL.

Remarque : Les commentaires ne sont pas pris en charge dans les bases de données Microsoft Access !

c. Commentaires sur une seule ligne

Les commentaires sur une seule ligne commencent par --.
Tout texte entre -- et la fin de la ligne sera ignoré (ne sera pas exécuté).
L'exemple suivant utilise un commentaire sur une seule ligne comme explication :

Exemple

```
--Afficher tous les clients
SELECT * FROM Client;
```

L'exemple suivant utilise un commentaire sur une seule ligne pour ignorer la fin d'une ligne :

```
SELECT * FROM Client -- WHERE Ville='Bénin';
```

L'exemple suivant utilise un commentaire sur une seule ligne pour ignorer une instruction :

```
--SELECT * FROM Client;
SELECT * FROM Produit;
```

d. Commentaires sur plusieurs lignes

Les commentaires sur plusieurs lignes commencent /*et se terminent par */.
Tout texte entre /* et */ sera ignoré.
L'exemple suivant utilise un commentaire sur plusieurs lignes comme explication :

Exemple

```
/*Afficher tous les champs
de tous les clients
de la table Client:*/

SELECT * FROM Client;
```

L'exemple suivant utilise un commentaire sur plusieurs lignes pour ignorer de nombreuses instructions :

```
/*SELECT * FROM Client;
SELECT * FROM Produits;
SELECT * FROM Commandes;
SELECT * FROM Categories;*/
SELECT * FROM Fournisseur;
```

Pour ignorer juste une partie d'une instruction, utilisez également le commentaire /* */.
L'exemple suivant utilise un commentaire pour ignorer une partie d'une ligne :

```
SELECT NomClient, /*City,*/ Pays FROM Client;
```

L'exemple suivant utilise un commentaire pour ignorer une partie d'une instruction :

```
SELECT * FROM Client WHERE (NomClient LIKE 'L%'
OR NomClient LIKE 'R%' /*OR NomClient  LIKE 'S%'
OR NomClient  LIKE 'T%'*/ OR NomClient  LIKE 'W%')
AND Pays='USA'
ORDER BY NomClient ;
```

3. Fonctions SQL NULL (IFNULL(), ISNULL(), COALESCE() et NVL())

Lorsque dans une expression d'attribut (Formule pour générer une valeur dans une requête SQL), vous utilisez un attribut qui peut contenir une valeur NULL, vous obtiendrez des résultats inattendus ou des erreurs.

Les SGBD vous offrent des fonctions qui permettrons à SQL de savoir comment interpréter la valeur NULL.

e. MySQL

La fonction MySQL IFNULL() vous permet de renvoyer une valeur alternative si une expression est NULL :

```
SELECT NomProduit, Pu * IFNULL(QtéStock, 0) FROM Produit;
```

ou nous pouvons utiliser la fonction COALESCE()

SELECT NomProduit, Pu * COALESCE(QtéStock, 0) FROM Produit;

f. Serveur SQL

La fonction SQL Server ISNULL()vous permet de renvoyer une valeur alternative lorsqu'une expression est NULL :

SELECT NomProduit, Pu * ISNULL(QtéStock, 0) FROM Produit;

ou nous pouvons utiliser la fonction COALESCE()

SELECT NomProduit, Pu * COALESCE(QtéStock, 0) FROM Produit;

g. MS Access

La fonction MS Access IsNull()renvoie VRAI (-1) si l'expression est une valeur nulle, sinon FAUX (0) :

SELECT NomProduit, Pu * IIF(IsNull(QtéStock, 0, QtéStock) FROM Produit;

h. Oracle

La fonction Oracle NVL()obtient le même résultat :

SELECT NomProduit, Pu * NVL (QtéStock, 0) FROM Produit;

ou nous pouvons utiliser la fonction COALESCE()

SELECT NomProduit, Pu * COALESCE(QtéStock, 0) FROM Produit;

4. Clause SQL TOP, LIMIT, FETCH FIRST ou ROWNUM

La clause SELECT TOP est utilisée pour spécifier le nombre d'enregistrements à renvoyer.
Elle est utile sur les grandes tables contenant des milliers d'enregistrements. Le renvoi d'un grand nombre d'enregistrements peut avoir un impact sur les performances.
Tous les systèmes de bases de données ne prennent pas en charge cette clause SELECT TOP.

MySQL prend en charge la clause LIMIT permettant de sélectionner un nombre limité d'enregistrements, tandis qu'Oracle utilise FETCH FIRST n ROWS ONLY et ROWNUM

Syntaxe SQL Server/MS Access :

SELECT TOP <*nombre*> | *percent* <*liste des attributs*>
FROM <*source de données*>
WHERE <*condition*>;

Syntaxe MySQL :

SELECT <*liste des attributs*>
FROM <*source de données*>
WHERE <*condition*>
LIMIT <*nombre*>;

Syntaxe Oracle 12 :

SELECT <*liste des attributs*>
FROM <*source de données*>
ORDER BY <*liste des attributs de tri*>
FETCH FIRST <*nombre*> ROWS ONLY;

Ancienne syntaxe Oracle :

SELECT <*liste des attributs*>
FROM <*source de données*>
WHERE ROWNUM <= <*nombre*>;

Ancienne syntaxe Oracle (avec ORDER BY) :

SELECT *
FROM (SELECT <*liste des attributs*> FROM <*source de données*> ORDER BY <*liste des attributs de tri*>)
WHERE ROWNUM <= <*nombre*>;

Exemple

i. **LIMIT**

L'instruction SQL suivante montre l'exemple équivalent pour MySQL et Sélectionne les 3 premiers enregistrements de la table Clients :

*SELECT * FROM Clients*
LIMIT 3;

j. **RÉCUPÉRER EN PREMIER**

L'instruction SQL suivante montre l'exemple équivalent pour Oracle et Sélectionne les 3 premiers enregistrements de la table Clients :

*SELECT * FROM Clients*
FETCH FIRST 3 ROWS ONLY;

k. **Exemple de POURCENTAGE**

L'instruction SQL suivante sélectionne les premiers 50 % des enregistrements de la table "Clients" (pour SQL Server/MS Access) :

*SELECT TOP 50 PERCENT * FROM Clients;*

L'instruction SQL suivante montre l'exemple équivalent pour Oracle :

*SELECT * FROM Customers FETCH FIRST 50 PERCENT ROWS ONLY;*

l. AJOUTER une CLAUSE WHERE

L'instruction SQL suivante sélectionne les trois premiers enregistrements de la table « Clients », où le pays est « Allemagne » (pour SQL Server/MS Access) :

*SELECT TOP 3 * FROM Clients WHERE Pays='Allemagne';*

L'instruction SQL suivante montre l'exemple équivalent pour MySQL :

*SELECT * FROM Clients WHERE Pays='Allemagne' LIMIT 3;*

L'instruction SQL suivante montre l'exemple équivalent pour Oracle :

*SELECT * FROM Customers WHERE Pays='Allemagne' FETCH FIRST 3 ROWS ONLY;*

m. AJOUTER le mot-clé ORDER BY

Ajoutez le mot-clé ORDER BY lorsque vous souhaitez trier le résultat et renvoyez les 3 premiers enregistrements du résultat trié.

Pour SQL Server et MS Access :

*SELECT TOP 3 * FROM Clients ORDER BY NomClient DESC;*

L'instruction SQL suivante montre l'exemple équivalent pour MySQL :

*SELECT * FROM Clients ORDER BY NomClient DESC LIMIT 3;*

L'instruction SQL suivante montre l'exemple équivalent pour Oracle :

*SELECT * FROM Clients ORDER BY CustomerName DESC FETCH FIRST 3 ROWS ONLY;*

5. Contrainte SQL PAR DÉFAUT

La contrainte DEFAULT est utilisée pour définir une valeur par défaut pour une colonne.

La valeur par défaut sera ajoutée à tous les nouveaux enregistrements, si aucune autre valeur n'est spécifiée.

sur CREATE TABLE

Le code SQL suivant définit une valeur par DEFAULT pour la colonne « Ville » lors de la création de la table « Personnes » :

Mon SQL / SQL Server / Oracle / MS Access :

```
CREATE TABLE Client (
    ID int NOT NULL,
    Nom char(255) NOT NULL,
    Prenom char(255),
    Age int,
    Ville char(255) DEFAULT 'Cotonou'
);
```

La contrainte DEFAULT peut également être utilisée pour insérer des valeurs système, en utilisant des fonctions comme : GETDATE()

```
CREATE TABLE Commandes (
    ID int NOT NULL,
    N°Bon int NOT NULL,
    DateCommande date DEFAULT GETDATE()
);
```

sur ALTER TABLE

Pour créer une contrainte DEFAULT sur la colonne "Ville" alors que la table est déjà créée, utilisez la requête SQL suivante :

MySQL :

```
ALTER TABLE Client ALTER Ville SET DEFAULT 'Cotonou';
```

SQL Server:

```
ALTER TABLE Client ADD CONSTRAINT df_City DEFAULT 'Cotonou' FOR Ville;
```

MS Access:

```
ALTER TABLE Client ALTER COLUMN Ville SET DEFAULT 'Cotonou';
```

Oracle:

```
ALTER TABLE Client MODIFY Ville DEFAULT 'Cotonou';
```

SUPPRIMER une contrainte DEFAULT

Pour supprimer une contrainte DEFAULT, utilisez le code SQL suivant :

MySQL :

ALTER TABLE Client ALTER Ville DROP DEFAULT;

SQL Serveur/Oracle/MS Access :

ALTER TABLE Client ALTER COLUMN Ville DROP DEFAULT;

6. Gestion des INDEX

a. Créer un index

L'instruction CREATE INDEX est utilisée pour créer des index dans des tables.

Les index sont utilisés pour récupérer les données de la base de données plus rapidement qu'autrement. Les utilisateurs ne peuvent pas voir les indexs, ils sont simplement utilisés pour accélérer les recherches/requêtes.

Remarque : La mise à jour d'une table avec des index prend plus de temps que la mise à jour d'une table sans (car les index nécessitent également une mise à jour). Ainsi, créez uniquement des index sur les colonnes qui seront fréquemment recherchées.

Syntaxe

Crée un index sur une table. Les valeurs en double sont autorisées :

CREATE INDEX <*NomIndex*> ON <*table*> (<*attribut1*>, <*attribut1*>,...);

Syntaxe CRÉER UN INDEX UNIQUE

Crée un index unique sur une table. Les valeurs en double ne sont pas autorisées :

CREATE UNIQUE INDEX <*NomIndex*> ON <*table*> (<*attribut1*>, <*attribut1*>,...);

Remarque : La syntaxe de création d'index varie selon les différentes bases de données. Par conséquent : Vérifiez la syntaxe de création d'index dans votre base de données.

Exemple de création d'un index

L'instruction SQL ci-dessous crée un index nommé « idx_Nom » sur la colonne « Nom » de la table « Client » :

CREATE INDEX idx_Nom ON Client (Nom);

Si vous souhaitez créer un index sur une combinaison de colonnes, vous pouvez lister les noms de colonnes entre parenthèses, séparés par des virgules :

CREATE INDEX idx_Nom ON Client (Nom, Prenom);

b. Supprimer un INDEX

L'instruction DROP INDEX est utilisée pour supprimer un index dans une table.

MS Access:

DROP INDEX <*NomIndex*> ON <*table*>;

SQL Server:

DROP INDEX <*table*>.<*NomIndex*>;

DB2/Oracle :

DROP INDEX <*NomIndex*> ;

MySQL :

ALTER TABLE <*table*> DROP INDEX <*NomIndex*> ;

7. Champ SQL INCREMENT AUTOMATIQUE

L'incrémentation automatique permet de générer automatiquement un numéro unique lorsqu'un nouvel enregistrement est inséré dans une table.
Il s'agit souvent du champ de clé primaire que nous aimerions voir créé automatiquement à chaque fois qu'un nouvel enregistrement est inséré.

a. Syntaxe pour MySQL

L'instruction SQL suivante définit la colonne « Personid » comme étant un champ de clé primaire à incrémentation automatique dans la table « Persons » :

```
CREATE TABLE Persons (
   Personid int NOT NULL AUTO_INCREMENT,
   Nom varchar(255) NOT NULL,
   Prenom varchar(255),
   Age int,
   PRIMARY KEY (Personid)
);
```

MySQL utilise le mot-clé AUTO_INCREMENT pour effectuer une fonctionnalité d'incrémentation automatique. Par défaut, la valeur de départ de AUTO_INCREMENT est 1 et elle sera incrémentée de 1 pour chaque nouvel enregistrement.

Pour que la séquence AUTO_INCREMENT commence par une autre valeur, utilisez l'instruction SQL suivante :

```
ALTER TABLE Persons AUTO_INCREMENT=100;
```

b. Syntaxe pour SQL Server

L'instruction SQL suivante définit la colonne « Personid » comme étant un champ de clé primaire à incrémentation automatique dans la table « Persons » :

```
CREATE TABLE Persons (
    Personid int IDENTITY(1,1) PRIMARY KEY,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int
);
```

MS SQL server utilise le mot-clé IDENTITY pour exécuter une fonctionnalité d'incrémentation automatique.

Dans l'exemple ci-dessus, la valeur de départ de IDENTITY est 1 et elle sera incrémentée de 1 pour chaque nouvel enregistrement.

Astuce : Pour spécifier que la colonne "Personid" doit commencer à la valeur 10 et s'incrémenter de 5, remplacez-la par IDENTITY(10,5).

c. Syntaxe pour MS Access

L'instruction SQL suivante définit la colonne « Personid » comme étant un champ de clé primaire à incrémentation automatique dans la table « Persons » :

```
CREATE TABLE Persons (
    Personid AUTOINCREMENT PRIMARY KEY,
    LastName char(255) NOT NULL,
    FirstName char(255),
    Age int
);
```

MS Access utilise le mot-clé AUTOINCREMENT pour effectuer une fonctionnalité d'incrémentation automatique. Par défaut, la valeur de départ de AUTOINCREMENTest 1 et elle sera incrémentée de 1 pour chaque nouvel enregistrement.

Astuce : Pour spécifier que la colonne "Personid" doit commencer à la valeur 10 et s'incrémenter de 5, modifiez l'auto-incrémentation en AUTOINCREMENT(10,5).

d. Syntaxe pour Oracle

Dans Oracle, le code est un peu plus délicat.

Vous devrez créer un champ auto-incrémenté avec l'objet séquence (cet objet génère une séquence de numéros).

```
CREATE SEQUENCE seq_person
MINVALUE 1
START WITH 1
```

```
INCREMENT BY 1
CACHE 10;
```

Le code ci-dessus crée un objet séquence appelé seq_person, qui commence par 1 et incrémentera de 1. Il mettra également en cache jusqu'à 10 valeurs pour les performances. L'option cache spécifie combien de valeurs de séquence seront stockées en mémoire pour un accès plus rapide.

Pour insérer un nouvel enregistrement dans la table "Persons", nous devrons utiliser la fonction nextval (cette fonction récupère la valeur suivante de la séquence seq_person) :

```
INSERT INTO Persons (Personid,FirstName,LastName)
VALUES (seq_person.nextval,'Magengo','Guttembert');
```

L'instruction SQL ci-dessus insérerait un nouvel enregistrement dans la table « Personnes ». La colonne "Personid" se verrait attribuer le numéro suivant de la séquence seq_person. La colonne « FirstName » serait définie sur « Magengo » et la colonne « LastName » serait définie sur « Guttembert ».

VII. LES FONCTIONS SQL

1. Fonction de chaines de caractère

a. CONCAT()

Dans le langage SQL la fonction CONCAT() permet de concaténer les valeur de plusieurs colonnes pour ne former qu'une seule chaîne de caractère. Cette fonction SQL peut se révéler pratique pour mettre bout-à-bout les valeurs de plusieurs colonnes pour n'en afficher qu'une. C'est donc pratique pour afficher un résultat facile à lire tout en maintenant les données dans plusieurs colonnes pour une bonne maintenance des données.

Syntaxe

Pour concaténer plusieurs chaînes de caractère, il convient d'utiliser une requête SQL avec la syntaxe suivante:

CONCAT(colonne1, colonne2)
Il est intéressant de noter que la fonction peut prendre plus de 2 paramètres (sauf pour Oracle).

Dans un SELECT

SELECT <Liste des attributs>, CONCAT(<Attribut1>,…,<Attribut2>) FROM <Source de données>

b. LENGTH() / LEN()

Dans le langage SQL la fonction LENGTH() permet de retourner la longueur d'une chaîne de caractères. **A noter :** la fonction se base sur le nombre d'octets. Un caractère multi-octet compte comme un seul caractère. Autrement dit, un caractère tel que "œ" compte comme un seul caractère.

Nom de la fonction selon le Système de Gestion de Base de Données (SGBD) :

- **LENGTH()** MySQL, PostgreSQL et Oracle
- **LEN()** SQL Server

c. REPLACE()

La fonction REPLACE dans le langage SQL permet de remplacer des caractères alphanumériques dans une chaîne de caractère. Cela sert particulièrement à mettre à jour des données dans une base de données ou à afficher des résultats personnalisés.

Syntaxe

REPLACE(<Chaîne d'entrée>, < Texte à remplacer >, <Texte de remplacement>)

La fonction comporte 3 paramètres :

- ***Chaîne d'entrée*** : peut être un attribut, une expression d'attribut ou une valeur littérale. C'est la chaine de caractère dans laquelle une sous chaine sera remplacée ;
- ***Texte à remplacer*** : peut être un attribut, une expression d'attribut ou une valeur littérale. C'est la sous chaine contenue dans la chaine d'entrée et qui sera remplacée ;
- ***Texte de remplacement*** : peut être un attribut, une expression d'attribut ou une valeur littérale. C'est la chaine de caractère qui viendra remplacer le texte à remplacer.

Exemple

SELECT REPLACE('Hello tout le monde', 'Hello', 'Bonjour');

Résultat

Bonjour tout le monde

Le mot «Hello» a été remplacé dans la chaîne initiale par le mot « Bonjour ».

Autre Exemple

Soit la **Table site** suivante :

id	date_ajout	site	url
1	2013-02-05	Wikipedia	http://fr.wikipedia.org/
2	2013-02-11	Google	http://www.google.fr/
3	2013-03-16	Facebook	http://www.facebook.com/
4	2013-03-16	Facebook	http://www.facebook.com/home.php
5	2013-03-16	Facebook	http://www.facebook.com/feed/friends

On voudrait remplacer le nom de domaine pour toutes les URLs de Facebook. Pour cela, il est possible d'utiliser la requête SQL suivante:

```
UPDATE site
SET url = REPLACE(url, 'www.facebook.com', 'fr-fr.facebook.com')
WHERE url LIKE '%www.facebook.com%'
```

Résultat

id	date_ajout	site	url
1	2013-02-05	Wikipedia	http://fr.wikipedia.org/
2	2013-02-11	Google	http://www.google.fr/
3	2013-03-16	Facebook	http://fr-fr.facebook.com/
4	2013-03-16	Facebook	http://fr-fr.facebook.com/home.php
5	2013-03-16	Facebook	http://fr-fr.facebook.com/feed/friends

Compatibilité
Cette fonction est très commune et fonctionne à la fois avec MySQL, PostgreSQL, Oracle et SQL Server.

Attention à l'ordre des paramètres
Les développeurs web qui ont l'habitude du langage PHP doivent se méfier de l'ordre des paramètres étant donné que ce n'est pas le même que la fonction **str_replace()**. L'ordre des paramètres de la fonction PHP est comme présenté ci-dessous:
str_replace ($texte_recherche , $texte_remplacement , $contenu_initial

d. SOUNDEX()

La fonction SOUNDEX() dans le langage SQL permet de retourner 4 caractères en fonction de la sonorité d'une chaîne de caractère. Cette chaîne de retour est régulièrement utilisée pour évaluer la similarité sonore entre 2 chaînes. Une utilisation courante consiste à utiliser cette fonction SQL pour améliorer la pertinence d'un algorithme dans le cas où un utilisateur se trompe dans l'orthographe d'un mot.

Attention **:** par défaut le système se base sur les sonorités anglaises. Dès lors, rien ne garantit un bon fonctionnement pour les autres langues.

Syntaxe
L'utilisation de cette fonction se fait via la syntaxe suivante :

```
SELECT SOUNDEX('hello');
```

Cette requête SQL retourne les résultats suivants :
H400

Exemple
Sonorité de "john" et "jone"

```
SELECT SOUNDEX('john'), SOUNDEX('jone');
```

Résultat :

SOUNDEX('john')	SOUNDEX('jone')
J500	J500

Rechercher un utilisateur à partir de son prénom mal orthographié

Table utilisateur :

id	prenom	nom
1	John	Smith
2	Joël	Lemaitre
3	Valéry	Buisson

La fonction SOUNDEX() va permettre de récupérer la liste des utilisateurs en fonction d'un nom, du moment que le nom se prononce de la même façon.

```
SELECT id, prenom, nom
FROM utilisateur
WHERE SOUNDEX(prenom) = SOUNDEX('jone')
```

Résultat :

id	prenom	nom
1	John	Smith

La fonction peut donc aider un formulaire de recherche à trouver des résultats potentiellement pertinents.

e. SUBSTRING()

La fonction SUBSTRING() dans le langage SQL (ou SUBSTR()) est utilisée pour extraire une partie d'une chaîne, par exemple pour tronquer un texte.

Compatibilité **:**

- **MySQL / PostgreSQL / Oracle:** SUBSTR() ou SUBSTRING()
- **SQL Server / Firebird :** SUBSTRING()
- **SQLite :** SUBSTR()

Syntaxe

La fonction SUBSTRING() peut s'utiliser de 4 façons différentes, que voici :

- **SUBSTRING(chaine, debut) :** retourne la chaîne de caractère de "chaine" à partir de la position définie par "debut" (position en nombre de caractères)
- **SUBSTRING(chaine FROM debut) :** idem que précédent
- **SUBSTRING(chaine, debut, longueur) :** retourne la chaîne de caractère "chaine" en partant de la position définie par "debut" et sur la longueur définie par "longueur"
- **SUBSTRING(chaine FROM debut FOR longueur) :** idem que précédent

A savoir : il ne faut pas oublier que la fonction est aussi parfois appelée SUBSTR().

Compatibilité

Il est important de savoir que ces syntaxes ne sont pas toutes comprises de certains Système de Gestion de Base de Données (SGBD). Pour savoir si un SGBD comprend une de ces syntaxes, il convient de consulter les 2 tableaux récapitulatifs ci-dessous :

Fonction SUBSTR() :

SGBD	SUBSTR(string, start)	SUBSTR(string, start, length)	SUBSTR(string FROM start)	SUBSTR(string FROM start FOR length)
MySQL	✔	✔	✔	✔
PostgreSQL	✔	✔	✗	✗
Oracle	✔	✔	✔	✔
SQL Server	✗	✗	✗	✗
SQLite	✔	✔	✗	✗
Firebird	✗	✗	✗	✗

Fonction SUBSTRING() :

SGBD	SUBSTRING(string, start)	SUBSTRING(string, start, length)	SUBSTRING(string FROM start)	SUBSTRING(string FROM start FOR length)
MySQL	✔	✔	✔	✔
PostgreSQL	✗	✗	✔	✔
Oracle	✔	✔	✔	✔
SQL Server	✗	✔	✗	✗
SQLite	✗	✗	✗	✗
Firebird	✗	✗	✔	✔

Exemple
Imaginons une base de données qui contient une liste de pays en français et en anglais.

Table pays :

id	nom_fr_fr	nom_en_gb
1	FRANCE	FRANCE
2	ESPAGNE	SPAIN

id	nom_fr_fr	nom_en_gb
3	ALLEMAGNE	GERMANY
4	CHINE	CHINA

Une requête SQL va être construite pour extraire les informations suivantes :

- L'identifiant du pays
- Le nom du pays en français
- Les 2 premiers caractères du nom en français (soit à partir du 1er caractère et sur une longueur de 2 caractères)
- Tout le nom en français, sans les 2 premiers caractères (soit tout le nom à partir du 3ème caractère)

Pour récupérer toutes ces données, il convient d'utiliser la requête SQL suivante :

```
SELECT id, nom_fr_fr, SUBSTR(nom_fr_fr, 1, 2), SUBSTR(nom_fr_fr, 3)
FROM pays
```

Cette requête va retourner les résultats suivants :

id	nom_fr_fr	SUBSTR(nom_fr_fr, 1, 2)	SUBSTR(nom_fr_fr, 3)
1	FRANCE	FR	ANCE
2	ESPAGNE	ES	PAGNE
3	ALLEMAGNE	AL	LEMAGNE
4	CHINE	CH	INE

f. LEFT()

Dans le langage SQL, la fonction LEFT() permet de retourner le nombre souhaité de caractères parmi les premiers caractères d'une chaîne. Cette fonction va tronquer volontairement le texte d'entré à la longueur désiré.
Elle est similaire à la fonction SUBSTR() qui permet d'extraire un bout de chaîne, à la différence qu'elle permet uniquement d'extraire les caractères situé au début.

Compatibilité **:** la fonction est compatible notamment avec MySQL, PostgreSQL, SQL Server et certaines version de Oracle. Si un système ne possède pas cette fonction, il est possible de la recréer grâce à la fonction SUBSTR() ou SUBSTRING().

Syntaxe

```
LEFT( chaine, longueur );
```

Le premier paramètre correspond au texte à tronquer. Le deuxième paramètre représente le nombre de caractères souhaité pour la valeur de sortie.

g. RIGHT()

Dans le langage SQL la fonction RIGHT() permet d'extraire la fin d'une chaîne de caractères en définissant la longueur souhaité. La fonction permet de tronquer une chaîne en partant de la fin, sur le nombre de caractères souhaités.

Compatibilité **:** la fonction est compatible notamment avec MySQL, PostgreSQL, SQL Server et certaines version de Oracle. Si un système ne possède pas cette fonction, il est possible de la recréer grâce à la fonction SUBSTR() ou SUBSTRING().

Syntaxe

La fonction s'utilise dans une requête SQL en respectant la syntaxe ci-dessous :
RIGHT(chaine, longueur);

chaine correspond à la chaîne de caractère d'entré, tandis que le paramètre "longueur" est un nombre qui correspond à la longueur maximum de la chaîne de retour.

h. REVERSE()

Dans le langage SQL, la fonction REVERSE() permet de retourner une chaîne de caractère en inversant l'ordre des caractères. Le premier caractère devient le dernier, le deuxième devient l'avant dernier, et ainsi de suite.

La fonction gère les caractères multi-octet tel que "œ".

Compatibilité **:** la fonction SQL REVERSE() est notamment utilisée sous MySQL, PostgreSQL et SQL Server.

Syntaxe

REVERSE(chaine);

Exemple

La fonction SQL REVERSE() s'utilise pour inverser l'ordre des caractères. Ces quelques exemples illustres l'utilisation de la fonction :

```
SELECT REVERSE('abc'); -- retourne : cba
SELECT REVERSE('123'); -- retourne : 321
```

i. TRIM(), LTRIM(), RTRIM()

Dans le langage SQL la fonction TRIM() permet de supprimer des caractères au début et en fin d'une chaîne de caractère. Le plus souvent la fonction TRIM() permet de supprimer les caractères invisibles, c'est-à-dire les caractères tels que l'espace, la tabulation, le retour à la ligne ou bien même le retour chariot. Une telle fonction peut se révéler utile pour économiser de l'espace dans une base de données ou pour afficher proprement des données.

Syntaxe

SELECT TRIM(<Chaine de caractère>);

Le retour de cette fonction permet d'obtenir uniquement la chaine de caractère sans les espaces de début et de fin.

Il est intéressant de savoir que dans certains Système de Gestion de Base de Données (SGBD) tel que MySQL ou PostgreSQL cette fonction peut supprimer certains caractères spécifiques.

- ***MySQL, PostgreSQL, Firebird et Oracle***

Supprimer un caractère spécifique en début et en fin de chaîne :
TRIM(BOTH 'x' FROM 'xxxExemplexxx');
Résultat : "Exemple"

Supprimer un caractère spécifique uniquement en début de chaîne :
TRIM(LEADING 'x' FROM 'xxxExemplexxx');
Résultat : "Exemplexxx"

Supprimer un caractère spécifique uniquement en fin de chaîne :
TRIM(TRAILING 'x' FROM 'xxxExemplexxx');
Résultat : "xxxExemple"

- ***SQLite***

Supprimer un caractère spécifique en début et en fin de chaîne :
TRIM('x', 'xxxExemplexxx');
Résultat : Exemple

Remarque
Ltrim et Rtrim font la même chose que TRIM seulement à droite ou à gauche

j. SQL LPAD()

Dans le langage SQL, la fonction LPAD() permet de compléter une chaîne de caractère jusqu'à ce qu'elle atteigne la taille souhaitée, en ajoutant des caractères au début de cette chaîne.
La fonction LPAD() s'utilise avec 2 ou 3 paramètres. Le premier paramètre correspond à la chaîne de caractères d'entrée, le second paramètre définie la longueur souhaitée et le 3ème paramètre correspond à la chaîne de complétion. Ce dernier paramètre est facultatif sur certains Systèmes de Gestion de Base de Données (SGBD) tels que Oracle ou PostgreSQL (mais pas pour MySQL).
Important : si la longueur souhaitée est plus petite que la taille actuelle de la chaîne de caractère, alors la chaîne retournée sera tronquée pour faire la taille exacte qui est souhaitée.

Syntaxe
LPAD(<Chaine à compléter>,<Longueur souhaitée>,<chaine de complétion>)

Cette fonction SQL peut s'utiliser simplement dans des requêtes SQL, comme le montre les requêtes suivantes :

```
SELECT LPAD('azerty', 4, 'x');  -- retourne 'azer'
SELECT LPAD('azerty', 6, 'x');  -- retourne 'azerty'
SELECT LPAD('azerty', 10, 'x'); -- retourne 'xxxxazerty'
SELECT LPAD('azerty', 10);      -- retourne '    azerty' (fonctionne avec Oracle ou PostgreSQL, mais pas avec MySQL)
SELECT LPAD('azerty', 9, 'xy'); -- retourne 'xyxazerty'
SELECT LPAD('123', 6, '0');     -- retourne '000123'
```

Remarque **:** par défaut si le troisième paramètre n'est pas complété la chaîne de complétion sera un espace vide.

k. UPPER(), LOWER(), UCASE(), LCASE()

Ces fonctions dans le langage SQL permettent de transformer tous les caractères d'une chaîne de caractère en majuscule (UPPER(), UCASE()) ou en minuscule (LOWER(), LCASE()).

Attention **:** il faut surement faire attention à l'encodage utilisé. Avec MySQL, la fonction utilise par défaut le jeu **ISO 8859-1 Latin1**.

Remarque **:** la fonction gère les caractères multi-octets (exemple : œ).
Compatibilité **:** les fonctions UPPER() et LOWER() fonctionnent notamment sous MySQL, PostgreSQL, SQLite, SQL server et Oracle. UCASE() et LCASE() fonctionnent sous MySQL et Oracle

Syntaxe

```
SELECT UPPER(<Chaine d'entrée>);
```

l. LOCATE(), CHARINDEX()

En SQL, la fonction LOCATE() permet de chercher la position d'une sous-chaîne de caractère dans une chaine de caractère. La fonction retourne zéro si la chaîne de caractère n'a pas été trouvée.
Cette fonction peut se révéler très utile lorsqu'elle est couplée à la fonction SUBSTR() (alias SUBSTRING()) pour pouvoir remplacer des caractères dans une chaîne.
Pour les utilisateurs de SQL Server la fonction s'appelle CHARINDEX() et fonctionne de la même façon.

Remarque **:** la fonction LOCATE() s'assimile à la fonction PHP strpos().

Syntaxe

```
LOCATE( <chaine_recherchée>, <chaine_source>, [<Position de départ>] );
```

Dans cette requête, la fonction va chercher "chaine_recherchée" dans "chaine_source". Si la chaîne existe et est présente, alors la fonction retournera la position du premier caractère.

La recherche commence toujours à partir du premier caractère de "chaine_source" et si la sous chaîne n'est pas trouvée, la fonction retournera **0**.
Il est possible de chercher une sous chaîne en commençant à chercher à partir d'une position. La requête SQL suivante permet de chercher la chaîne "test" dans le champ "nom_colonne" en commençant à chercher à partir de la position 4.

```
LOCATE( 'test', nom_colonne, 4 );
```

Ce 3ème paramètre est optionnel et peut être utilisé s'il faut ignorer une première partie.

m. INSTR()

Dans le langage SQL, la fonction INSTR() est utilisée par MySQL et Oracle pour obtenir la position d'une occurrence dans une chaîne de caractères. Cette fonction permet donc de savoir si une chaîne est présente dans une autre chaîne et d'obtenir par la même occasion sa position.
La fonction est identique à LOCATE() à la seule différence que les 2 paramètres qui la composent sont inversés.

Syntaxe

```
INSTR(<chaine_source>, <chaine_recherchée>);
```

La fonction INSTR() va chercher et retourner la position de <chaine_recherchée> dans <chaine_source>. Si la chaîne <chaine_recherchée> n'est pas présente dans <chaine_source>, alors la fonction retournera la position 0.
Il faut également savoir que si la chaîne <chaine_recherchée> est présente plusieurs fois dans <chaine_source>, alors ce sera seulement la position de la première occurrence qui sera retournée.

Exemple simple

INSTR('hello world!' , 'world');

Résultat : 7

Imaginons maintenant une table qui contient des verbes de la langue française.
Table "verbe" :

id	verbe
1	être
2	avoir
3	aller
4	manger

Si on souhaite réduire cette liste pour obtenir uniquement les verbes qui se termine par "er", il est possible dans un premier temps de vérifier la présence et la position des lettres "er" dans le mot du verbe. Il est donc possible d'utiliser une requête SQL comme celle ci-dessous:

```
SELECT id, verbe, INSTR( verbe, 'er' )
FROM verbe
```

Résultat :

id	verbe	INSTR(verbe, 'er')
1	être	0
2	avoir	0
3	aller	4
4	manger	5

Ce résultat démontre bien qu'il est possible de filtrer les mots qui ne possèdent pas de la succession de lettres "er". Toutefois, il peut y avoir des verbes qui utilises les lettres "er" en plein milieu du mot, alors que dans notre cas, on souhaite vérifier si le verbe se termine par "er".
Il serait possible d'utiliser en plus la fonction LENGTH() pour connaître la longueur du verbe et ainsi déterminer si la position de "er" correspond bien au 2 dernières lettres du verbe.

2. Fonctions mathématiques / numérique

a. Exemple de fonctions mathématiques utiles

- ABS() retourner la valeur absolue d'un nombre [MySQL, PostgreSQL, SQL Server]
- ACOS() retourne l'arc cosinus [MySQL, SQL Server]
- ASIN() retourne l'arc sinus [MySQL, SQL Server]

- ATAN2() ou ATAN() retourne la tangente de 2 arguments [MySQL]
- ATAN() retourne l'arc tangente [MySQL, SQL Server]
- ATN2() retourne l'angle en radian entre un axe et un rayon [SQL Server]
- CBRT() retourne la racine carrée de l'argument [PostgreSQL]
- CEIL() obtenir la valeur entière supérieure d'un nombre [MySQL, PostgreSQL]
- CEILING() obtenir la valeur entière supérieure d'un nombre [MySQL, SQL Server]
- CONV() convertir des nombres entre différentes bases de nombre [MySQL]
- COS() obtenir le cosinus d'une valeur numérique [MySQL, SQL Server]
- COT() retourne la cotangante [MySQL, SQL Server]
- CRC32() calcule une valeur de contrôle de redondance cyclique [MySQL]
- DEGREES() convertir un angle en radians en degrés [MySQL, PostgreSQL, SQL Server]
- EXP() calculer l'exponentiel d'un nombre [MySQL, PostgreSQL, SQL Server]
- FLOOR() obtenir la valeur entière inférieure d'un nombre [MySQL, PostgreSQL, SQL Server]
- LN() retourne le logarithme naturel (cf. logarithme népérien) de l'argument [MySQL, PostgreSQL]
- LOG10() retourne le logarithme base 10 de l'argument [MySQL, SQL Server]
- LOG2() retourne le logarithme base 2 de l'argument [MySQL]
- LOG() retourne le logarithme naturel du premier argument [MySQL, PostgreSQL, SQL Server]
- MOD() retourner le modulo d'un nombre (le reste d'une division) [MySQL, PostgreSQL]
- PI() obtenir la valeur de PI (cf. 3,14159) [MySQL, PostgreSQL, SQL Server]
- POW() retourne l'argument à la puissance spécifiée [MySQL, PostgreSQL]
- POWER() retourne l'argument à la puissance spécifiée [MySQL, SQL Server]
- RADIANS() convertir un angle en dégrés en radians [MySQL, PostgreSQL, SQL Server]
- RAND() obtenir un nombre aléatoire. Pratique également pour classer des résultats de manière aléatoire [MySQL, SQL Server]
- RANDOM() valeur aléatoire entre 0.0 et 1.0 [PostgreSQL]
- ROUND() arrondir un nombre a virgule [MySQL, PostgreSQL, SQL Server]
- SETSEED() définir l'échantillon pour d'importants appels à la fonction aléatoire [PostgreSQL]
- SIGN() indique si l'argument est supérieur, inférieur ou égal à zéro [MySQL, PostgreSQL, SQL Server]
- SIN() retourner le sinus d'un nombre [MySQL, SQL Server]
- SQRT() calculer la racine carrée d'un nombre [MySQL, PostgreSQL, SQL Server]
- SQUARE() retourne la racine carrée de la valeur de type float spécifiée [SQL Server]
- TAN() retourner la tangente d'un nombre [MySQL, SQL Server]
- TRUNC() tronquer à un nombre à virgule en un nombre entier ou en un nombre avec le nombre de décimal souhaité [PostgreSQL]
- TRUNCATE() tronquer un nombre à virgule [MySQL]

b. RAND()

En SQL la fonction RAND() permet de sélectionner un nombre aléatoire à virgule, compris entre 0 et 1. Le résultat de cette fonction sera différent à chaque fois que la fonction est exécutée dans une requête SQL.

Cette fonction se révèle pratique avec un ORDER BY pour classer des résultats aléatoirement. Toutefois, même si c'est possible il faut essayer d'éviter cette méthode qui n'est pas particulièrement efficace en terme de performances.

La fonction peut aussi être utilisée dans une clause WHERE. Cela peut être pratique pour sélectionner un résultat aléatoirement.

Syntaxe

```
RAND();
```

Exemple
Imaginons une application qui utilise une base de données des principales villes de France. Pour le bien de ce cours, nous allons utiliser une table qui contient 10 villes françaises.
Table ville :

id	ville	departement
1	Paris	Paris
2	Marseille	Bouches-du-Rhône
3	Lyon	Rhône
4	Toulouse	Haute-Garonne
5	Nice	Alpes-Maritimes
6	Nantes	Loire-Atlantique
7	Strasbourg	Bas-Rhin
8	Montpellier	Hérault
9	Bordeaux	Gironde
10	Lille	Nord

Il est possible de trier les résultats aléatoirement en utilisant la syntaxe suivante :

```
SELECT *
FROM ville
ORDER BY RAND()
```

A chaque fois que la requête sera exécutée, celle-ci retournera un résultat aléatoire. L'un des résultats possibles sera donc le suivant:

id	ville	departement
2	Marseille	Bouches-du-Rhône
10	Lille	Nord
6	Nantes	Loire-Atlantique
7	Strasbourg	Bas-Rhin
9	Bordeaux	Gironde
1	Paris	Paris
5	Nice	Alpes-Maritimes
3	Lyon	Rhône
4	Toulouse	Haute-Garonne
8	Montpellier	Hérault

Sélectionner un résultat aléatoirement

En couplant cette fonction SQL avec la fonction ROUND() qui permet d'arrondir un nombre a virgule à un entier, il est possible de retourner un nombre entier plutôt qu'un nombre a virgule flottante. Ce résultat entier peut être utilisé pour sélectionner une ligne parmi un set d'enregistrement. Voici une requête SQL utilisant cette méthode:

```
SELECT *
FROM ville
WHERE id = ROUND( RAND() * 9 ) + 1
```

Voici une explication concernant cette requête :

- RAND() * 9 permet de générer un nombre a virgule compris entre 0 et 9
- ROUND() permet d'arrondir le nombre généré pour obtenir exclusivement des nombres entiers
- + 1 permet d'ajouter 1 au résultat précédent. Ainsi, le nombre sera compris entre 1 et 10

Cette requête va donc se baser sur un nombre entier entre 1 et 10 pour sélectionner une ville aléatoirement dans la table "ville". L'un des résultats possibles est donc le suivant:

id	ville	departement
7	Strasbourg	Bas-Rhin

Attention, il faut faire attention avec cette méthode qu'il n'y a pas de trou entre les identifiants sous peine de ne retourner aucun résultat.

c. ROUND()

Dans le langage SQL la fonction ROUND() permet d'arrondir un résultat numérique. Cette fonction permet soit d'arrondir sans utiliser de décimal pour retourner un nombre entier (c'est-à-dire : aucun chiffre après la virgule), ou de choisir le nombre de chiffre après la virgule.

Syntaxe

```
ROUND(<Valeur>[,<Nombre de chiffre après la virgule>])
```

Exemple

Imaginons une table qui contient des noms d'utilisateurs et le résultat d'un algorithme quelconque. Le résultat de l'algorithme pourrait être un indice utilisé par un moteur de recherche pour trouver l'utilisateur le plus pertinent pour une recherche. Ce résultat est un chiffre à virgule.

Table "resultat" :

id	utilisateur	algorithme
1	Thomas	78.4216857
2	Marie	69.1294526
3	Sophie	63.5236241
4	John	24.6578954

```
SELECT id, utilisateur, algorithme, ROUND(algorithme)
FROM resultat
```

Résultat **:**

id	utilisateur	algorithme	ROUND(algorithme)
1	Thomas	78.4216857	78

id	utilisateur	algorithme	ROUND(algorithme)
2	Marie	69.1294526	69
3	Sophie	63.5236241	64
4	John	24.6578954	25

Ce résultat démontre que la fonction a supprimée les chiffres après la virgule. Il est intéressant de noter que le résultat est arrondis à l'entier le plus proche (arrondit au chiffre entier supérieur si les décimales sont supérieur à 0.5).

Arrondir à 2 chiffres après la virgules

Il est possible d'arrondir aux 2 chiffres après la virgule en utilisant la requête SQL suivante:

```
SELECT id, utilisateur, algorithme, ROUND(algorithme, 2)
FROM `resultat`
```

Résultat **:**

id	utilisateur	algorithme	ROUND(algorithme, 2)
1	Thomas	78.4216857	78.42
2	Marie	69.1294526	69.13
3	Sophie	63.5236241	63.52
4	John	24.6578954	24.66

Arrondir le résultat à la dizaine

Il est possible d'utiliser une valeur négative dans le second argument pour obtenir une valeur arrondit à la dizaine, centaine ou plus selon la valeur indiquée. Cela peut être effectué en utilisant la requête suivante:

```
SELECT id, utilisateur, algorithme, ROUND(algorithme, -1)
FROM resultat
```

Résultat **:**

id	utilisateur	algorithme	ROUND(algorithme, -1)
1	Thomas	78.4216857	80
2	Marie	69.1294526	70
3	Sophie	63.5236241	60
4	John	24.6578954	20

3. Fonctions de dates et d'heures

Fonctions SQL utiles pour les dates et les heures

- AGE() soustraire 2 dates [PostgreSQL]

- ADDDATE() ajouter une période sous forme d'heures à une date [MySQL]
- ADDTIME() ajouter une période sous forme d'une date à une autre date [MySQL]
- CONVERT_TZ () convertir d'une "timezone" à une autre [MySQL]
- CURDATE() récupérer la date courante [MySQL]
- CURRENT_DATE() synonyme de CURDATE() [MySQL, PostgreSQL]
- CURRENT_TIME() synonyme de CURTIME() [MySQL, PostgreSQL]
- CURRENT_TIMESTAMP() synonyme de NOW() [MySQL, PostgreSQL, SQL Server]
- CURTIME() Return the current time [MySQL]
- DATE() extraire une date à partir d'une chaîne contenant une valeur au format DATE ou DATETIME [MySQL]
- DATE_ADD() ajouter une valeur au format TIME à une date [MySQL]
- DATE_FORMAT() formater la date pour l'afficher selon le format choisi [MySQL]
- DATE_PART() extraire un élément d'un DATETIME (cf. heure, minute, jour, mois …) [PostgreSQL]
- DATE_SUB() soustraire une valeur au format TIME à une date [MySQL]
- DATE_TRUNC() tronquer un DATETIME avec la précision souhaitée (cf. mois, jour, heure, minute …) [PostgreSQL]
- DATEADD() ajoute un élément (minute, heure, jour, mois, année …) à une date spécifiée [SQL Server]
- DATEDIFF() déterminer le nombre de jours entre 2 dates [MySQL, SQL Server]
- DATENAME() retourner une partie d'une date (cf. minute, heure, jour, semaine, mois …) [SQL Server]
- DATEPART() retourne un entier qui représente la partie d'une date (cf. minute, heure, jour, mois, année …) [SQL Server]
- DAY() synonyme de DAYOFMONTH() [MySQL, SQL Server]
- DAYNAME() retourne le nom du jour de la semaine [MySQL]
- DAYOFMONTH() retourner le jour dans le mois (de 1 à 31) [MySQL]
- DAYOFWEEK() retourner le jour dans la semaine (1=dimanche, 2=lundi, 3=mardi …) [MySQL]
- DAYOFYEAR() retourner le jour dans l'année (de 1 à 366) [MySQL]
- EXTRACT() extraire une partie d'une date [MySQL, PostgreSQL]
- FROM_DAYS() convertir un nombre de jour en une date [MySQL]
- FROM_UNIXTIME() convertir un timestamp UNIX en un date au format DATETIME [MySQL]
- GET_FORMAT() retourne le format d'une date dans une chaîne de caractère [MySQL]
- GETDATE() obtenir la date courante du système, sans le décalage horaire [SQL Server]
- GETUTCDATE() obtenir la date courante UTC [SQL Server]
- HOUR() extraire le nombre d'heure pour une heure au format HH:MM:SS [MySQL]
- ISDATE() retourne 1 si la valeur en paramètre est dans l'un des formats suivants : TIME, DATE ou DATETIME [SQL Server]
- ISFINITE() tester pour savoir si une date ou une période de temps est finie [PostgreSQL]
- JUSTIFY_HOURS() ajuster un intervalle de 24 heures en tant que "1 jour" ou un intervalle de 30 jours en tant que "1 mois" [PostgreSQL]
- LAST_DAY() retourner le dernier jour du mois d'une date [MySQL]
- LOCALTIME() synonyme de NOW() [MySQL, PostgreSQL]
- LOCALTIMESTAMP() synonyme de NOW() [MySQL, PostgreSQL]
- MAKEDATE() retourne une date à partir d'une année et du numéro du jour dans cette année [MySQL]
- MAKETIME() créer une heure au format TIME à partir d'une heure, des minutes et du nombre de secondes [MySQL]
- MICROSECOND() retourne le nombre de microsondes à partir d'une heure ou d'un DATETIME [MySQL]

- MINUTE() extraire le nombre de minutes d'une heure au format HH:MM:SS [MySQL]
- MONTH() extraire le numéro du mois à partir d'une date [MySQL, SQL Server]
- MONTHNAME() retourne le nom du mois [MySQL]
- NOW() obtenir la date courante [MySQL, PostgreSQL]
- PERIOD_ADD() ajoute un nombre définie de mois à une période [MySQL]
- PERIOD_DIFF() retourne le nombre de mois entre 2 périodes définies [MySQL]
- QUARTER() Return the quarter from a date argument [MySQL]
- SEC_TO_TIME() convertir un nombre de secondes en une heure et minutes au format HH:MM:SS [MySQL]
- SECOND() extraire le nombre de secondes d'une heure au format HH:MM:SS [MySQL]
- STR_TO_DATE() convertir une chaîne de caractère en date [MySQL]
- SUBDATE() synonyme de DATE_SUB() lorsque la fonction est invoquée avec 3 arguments [MySQL]
- SUBTIME() soustraire une heure [MySQL]
- SWITCHOFFSET() retourne une valeur DATETIMEOFFSET en changeant le fuseau horaire [SQL Server]
- SYSDATE() retourne la date et l'heure courante [MySQL]
- SYSDATETIME() retourne le DATETIME (date et heure) de l'ordinateur sur lequel est installé la base de données [SQL Server]
- SYSDATETIMEOFFSET() retourne le DATETIMEOFFSET du système avec le décalage du fuseau horaire inclus [SQL Server]
- SYSUTCDATETIME() retourne le DATETIME en heure UTC de l'ordinateur sur lequel la base est installées [SQL Server]
- TIME_FORMAT() formater une date dans un autre format [MySQL]
- TIME_TO_SEC() convertir une heure de format HH:MM:SS en nombre de secondes [MySQL]
- TIME() extraire l'heure/minutes/secondes au format HH:MM:SS à partir d'une date [MySQL]
- TIMEDIFF() retourne la durée entre 2 heures [MySQL]
- TIMESTAMP() permet de convertir une DATE au format DATETIME [MySQL]
- TIMESTAMPADD() ajoute un intervalle à une expression au format DATETIME [MySQL]
- TIMESTAMPDIFF() soustraire un intervalle à partir d'une expression au format DATETIME [MySQL]
- TIMEOFDAY() retourne la date et heure courante [PostgreSQL]
- TO_DAYS() retourne le nombre de jour à partir d'une date [MySQL]
- TO_SECONDS() Return the date or datetime argument converted to seconds since Year 0 [MySQL]
- TODATETIMEOFFSET() retourne un DATETIMEOFFSET à partir d'un DATETIME [SQL Server]
- UNIX_TIMESTAMP() retourner le timestamp UNIX (nombre de secondes depuis le 1er janvier 1970) [MySQL]
- UTC_DATE() retourne la date GMT courante [MySQL]
- UTC_TIME() retourne l'heure GMT courante [MySQL]
- UTC_TIMESTAMP() retourne la date et heure GMT courante [MySQL]
- WEEK() déterminer le numéro de la semaine dans une année, à partir d'une date [MySQL]
- WEEKDAY() déterminer le jour de la semaine à partir d'une date (0=lundi, 1:mardi, 2=mercredi …) [MySQL]
- WEEKOFYEAR() déterminer le numéro de la semaine dans une année, à partir d'une date [MySQL]
- YEAR() extraire l'année d'une date [MySQL, SQL Server]
- YEARWEEK() retourne l'année et la semaine à partir d'une date [MySQL]

4. Fonctions de conversion

a. SQL CAST()

La fonction CAST() dans le langage SQL est une fonction de transtypage qui permet de convertir une données d'un type en un autre. Il est par exemple possible de transformer une date au format DATETIME en DATE, ou l'inverse.

Syntaxe

CAST(*expression* AS *type*);

Dans cette syntaxe, "expression" correspond à la valeur a transformer, tandis que "type" contient le type de données que l'ont souhaite obtenir. Ce type de données peut être l'une des valeurs suivantes (dépend du système de gestion de base de données) :

- BINARY
- CHAR
- DATE
- DATETIME
- TIME

b. CONVERT() (MySQL et SQL Server)

La fonction SQL CONVERT(), dans les systèmes MySQL et SQL Server, permet de convertir une donnée d'un type en un autre, de façon semblable à CAST(). La fonction permet par exemple de convertir une données de type FLOAT en INTEGER ou une DATE en DATETIME.

Avec les Système de Gestion de Base de Donnée (SGBD) PostgreSQL et Oracle, la fonction CONVERT() permet de convertir une chaîne de caractère d'un encodage en un autre, tel qu'une conversion de UTF-8 à Latin-1 (ISO-8859-1).

Syntaxe

- **MySQL**

CONVERT(expression, type);

Le paramètre "expression" correspond à la donnée qui doit subir le transtypage. Le type de données peut être BINARY, CHAR, DATE, DATETIME, INTEGER ou TIME.

- **SQL Server**

CONVERT (type, expression);

Le paramètre "expression" correspond également à la donnée qui doit changer de type. Le type quand à lui peut contenir plus de possibilité, allant du type BINARY à TEXT, en passant par CHAR, VARCHAR, DATETIME, TIMESTAMP et bien d'autres.

- **PostgreSQL et Oracle**

La fonction SQL CONVERT() utilisée sous PostgreSQL ou Oracle permet de changer l'encodage d'une chaîne de caractère. La fonction permet par exemple de convertir un texte en UTF-8 en un texte Latin-1, c'est à dire en ISO-8859-1.

Syntaxe

SELECT CONVERT(texte, encodage_source, encodage_fin);

Le premier paramètre correspond à la chaîne de caractère à convertir, avec un type tel qu'un VARCHAR. Le deuxième paramètre correspond à l'encodage de cette chaîne de caractère. L'encodage doit correspondre pour que la fonction réalise le changement d'encodage convenablement. Le dernier paramètre correspond à l'encodage souhaité.

Remarque

Les mots-clés à utiliser pour définir la source d'encodage sont défini dans les documentations officielles de PostgreSQL et de Oracle. Ces 2 système utilisent des mots-clés différents pour nommer les encodages.

Exemple

La requête ci-dessous est un exemple fictif pour convertir une chaîne de caractère sous PostgreSQL de l'encodate UTF-8 à un encodage Latin-1.

```
SELECT CONVERT('texte en utf8', 'UTF8', 'LATIN1');
-- résultat : 'texte en utf-8'
```

c. GROUP_CONCAT()

Dans le langage SQL, la fonction GROUP_CONCAT() permet de regrouper les valeurs non nulles d'un groupe en une chaîne de caractère. Cela est utile pour regrouper des résultats en une seule ligne au lieu d'avoir autant de ligne qu'il y a de résultat dans ce groupe.

L'intérêt le plus flagrant de cette fonction consiste à concaténer les valeurs d'une colonne correspondant à une jointure avec une autre table. De cette manière, il n'est pas forcément nécessaire d'effectuer un traitement supplémentaire des résultats pour exploiter les valeurs.

La fonction GROUP_CONCAT() est actuellement disponible que pour MySQL. Les autres Systèmes de Gestion de Base de Données (SGBD) tel que PostgreSQL, SQLite ou SQL Server n'intègrent pas cette fonctionnalité.

Syntaxe

```
SELECT `id`, GROUP_CONCAT( `nom_colonne` )
FROM `table`
GROUP BY `id`
```

Dans cette requête SQL, les valeurs de la colonne "nom_colonne" seront concaténées et séparée par défaut par une virgule.

Il est possible de choisir le caractère qui sert de séparateur en respectant la syntaxe suivante :

```
SELECT `id`, GROUP_CONCAT( `nom_colonne` SEPARATOR ' ' )
FROM `table`
GROUP BY `id`
```

Dans cette seconde requête SQL, les valeurs seront séparées par un espace. Il est intéressant de savoir que le séparateur peut être vide pour ne pas mettre de séparateur dans la concaténation.

Exemple

Imaginons un blog qui possède des articles et auxquels il est possible d'associer des mots-clés. Les articles sont enregistrés dans la table "article" ont un titre, une date de création, une date de publication et une catégorie. Les mots-clés sont ajoutés dans une table "mot_cle" et sont associés à chaque article grâce à une clé étrangère (cf. il s'agit d'une "foreign key").

Table "article" :

id	titre	date_creation	date_publication	categorie
1	3 outils informatiques à connaître	2012-07-11	2012-07-18	information

id	titre	date_creation	date_publication	categorie
2	Améliorer les performances d'une application web	2013-04-03	2013-04-06	tutorial
3	Protection contre les failles de sécurités	2013-08-22	2013-08-29	tutorial
4	Astuces de codage pour les gros projets	2013-11-24	2013-12-01	astuces
5	Encrypter les données d'une application	2013-12-02	2013-12-04	astuces

Table “mot_cle” :

id	article_id	mot	date_creation
1	1	MySQL	2012-07-18
2	1	PostgreSQL	2012-07-18
3	1	Excel	2012-07-18
4	3	Failles XSS	2013-08-28
5	3	Attaque par force brute	2013-08-29
6	4	Pear	2013-12-01
7	4	Astuce	2013-12-01

Lister les catégories et concaténer les titres

```
SELECT `categorie`, GROUP_CONCAT(`titre`) AS concat_titre
FROM `article`
GROUP BY `categorie`
```

Résultats :

categorie	concat_titre
information	3 outils informatiques à connaître
tutorial	Améliorer les performances d'une application web,Protection contre les failles de sécurités
astuces	Astuces de codage pour les gros projets, Encrypter les données d'une application

Lister les articles et concaténer les mots-clés

Les articles peuvent être affichés tout en associant la liste des mots-clés de chacun de ces articles. Pour cela, il est possible d’exécuter la requête suivante :

```
SELECT `titre`, `date_publication`, GROUP_CONCAT(`mot` SEPARATOR " ") AS concat_mot
FROM `article`
LEFT JOIN `mot_cle` ON `article_id` = `article`.`id`
```

```
GROUP BY `article`.`id`
```

Résultats **:**

titre	date_publication	GROUP_CONCAT(mot)
3 outils informatiques à connaître	2012-07-18	MySQL PostgreSQL Excel
Améliorer les performances d'une application web	2013-04-06	NULL
Protection contre les failles de sécurités	2013-08-29	Failles XSS Attaque par force brute
Astuces de codage pour les gros projets	2013-12-01	Astuce Pear
Encrypter les données d'une application	2013-12-04	NULL

Le résultat de cette requête permet d'obtenir en une seule requête les articles et les mots-clés qui sont stockés dans une autre table. Grâce à la modification du séparateur, les mots-clés sont séparés par un espace.

Erreurs courantes

- **Attention au séparateur**

Il convient de bien définir le séparateur et de vérifier que celui-ci ne va pas interférer avec le contenu des colonnes. Par exemple, si le séparateur est une virgule et doit séparer des mots-clés, il convient de s'assurer qu'aucun mot-clé ne contient de virgule. Le cas échéant, il ne sera pas possible de savoir si la virgule est le séparateur ou s'il est inclus dans un mot.

- **Attention à la limite de longueur**

Dans l'hypothèse ou la requête contient de nombreux résultats, il convient de s'assurer que la limite de longueur n'est pas dépassée. Il est possible de redéfinir cette longueur via le paramètre "group_concat_max_len". Voici une requête pour modifier cette longueur :

```
SET SESSION group_concat_max_len = 1000000;
```

- **Attention au multiple GROUP_CONCAT() dans une requête SQL**

Il convient de faire attention à des éventuels doublons dans les résultats lorsque plusieurs champs GROUP_CONCAT() existent au sein d'une même requête SQL.

Chapitre 7 : Sécurité des bases de données

I. Injection SQL

L'injection SQL est une technique d'injection de code ou morceau de requête sql qui peut détruire votre base de données.
L'injection SQL est l'une des techniques de piratage Web les plus courantes.
L'injection SQL consiste à placer du code malveillant dans des instructions SQL, via une entrée de page Web.

1. SQL dans les pages Web

L'injection SQL se produit généralement lorsque vous demandez une entrée à un utilisateur, comme son nom d'utilisateur ou ID d'utilisateur, et au lieu d'un nom ou identifiant, l'utilisateur vous donne une instruction SQL que vous exécuterez **sans le savoir** sur votre base de données.

L'exemple suivant crée une instruction SELECT en ajoutant une variable (txtUserId) à une chaîne de caractère. La variable est récupérée à partir de l'entrée utilisateur (getRequestString) :

```
txtUserId = getRequestString("UserId");
txtSQL = "SELECT * FROM Users WHERE UserId = " + txtUserId;
```

Le reste de ce chapitre décrit les dangers potentiels liés à l'utilisation des entrées utilisateur dans les instructions SQL.

d. L'injection SQL basée sur 1 = 1 est toujours vraie

Dans l'exemple précédent, le but initial du code était de créer une instruction SQL pour sélectionner un utilisateur, avec un identifiant utilisateur donné.

Si rien n'empêche un utilisateur de saisir une "mauvaise" entrée, l'utilisateur peut saisir une entrée "intelligente" comme celle-ci :

ID de l'utilisateur: 105 OR 1=1

Ensuite, l'instruction SQL ressemblera à ceci :

```
SELECT * FROM Users WHERE UserId = 105 OR 1=1;
```

Le SQL ci-dessus est valide et renverra TOUTES les lignes de la table "Utilisateurs", puisque **OR 1=1** est toujours VRAI.

L'exemple ci-dessus vous semble-t-il dangereux ? Que se passe-t-il si le tableau « Utilisateurs » contient des noms et des mots de passe ?

L'instruction SQL ci-dessus est à peu près la même que celle-ci :

```
SELECT UserId, Name, Password FROM Users WHERE UserId = 105 or 1=1;
```

Un pirate informatique peut accéder à tous les noms d'utilisateur et mots de passe d'une base de données, en insérant simplement 105 OR 1=1 dans le champ de saisie.

e. L'injection SQL basée sur ""="" est toujours vraie

En SQL, l'expression ""="" est une condition qui évalue toujours à vrai. Cela signifie que si cette condition est utilisée dans une clause WHERE d'une requête SQL, elle permettra à toutes les lignes de la table de satisfaire la condition, car une chaîne vide (") est toujours égale à une autre chaîne vide (").

Par exemple, dans une requête SQL :

SELECT * FROM <table> WHERE <attribut> = <valeur> OR " = "

Cette condition OR "=" sera toujours vraie, ce qui signifie que toutes les lignes de la table seront retournées, indépendamment de la valeur de la colonne <attribut>.

Il est important de noter que l'utilisation de cette condition est rarement utile dans des requêtes SQL réelles, car elle ne filtre pas les données de manière significative. En fait, elle est potentiellement utilisée pour introduire des failles de sécurité si elle est utilisée de manière inappropriée, par exemple dans des requêtes dynamiquement générées où elle pourrait être exploitée pour des attaques par injection SQL.

Voici un exemple de connexion d'un utilisateur sur un site Web :

Nom d'utilisateur:
John Doe

Mot de passe:
myPass

uName = getRequestString("username");
uPass = getRequestString("userpassword");

sql = 'SELECT * FROM Users WHERE Name ='" + uName + "' AND Pass ='" + uPass + "'"

Résultat

SELECT * FROM Users WHERE Name ="John Doe" AND Pass ="myPass"

Un pirate informatique peut accéder aux noms d'utilisateur et aux mots de passe d'une base de données en insérant simplement " OR ""=" dans la zone de texte du nom d'utilisateur ou du mot de passe :

Nom d'utilisateur:

Mot de passe:

Le code sur le serveur créera une instruction SQL valide comme celle-ci :

Résultat

```
SELECT * FROM Users WHERE Name ="" or ""="" AND Pass ="" or ""=""
```

Le SQL ci-dessus est valide et renverra toutes les lignes de la table "Utilisateurs", puisque **OR ""=""** est toujours VRAI.

f. Injection SQL basée sur des instructions SQL par lots

La plupart des bases de données prennent en charge les instructions SQL par lots.

Un lot d'instructions SQL est un groupe de deux ou plusieurs instructions SQL, séparées par des points-virgules.

L'instruction SQL ci-dessous renverra toutes les lignes de la table "Utilisateurs", puis supprimera la table "Fournisseurs".

Exemple

```
SELECT * FROM Users; DROP TABLE Fournisseurs ;
```

Regardez l'exemple suivant :

Exemple

```
txtUserId = getRequestString("UserId");
txtSQL = "SELECT * FROM Users WHERE UserId = " + txtUserId;
```

Et l'entrée suivante :

ID de l'utilisateur: 105; DROP TABL

L'instruction SQL valide ressemblerait à ceci :

Résultat

```
SELECT * FROM Users WHERE UserId = 105; DROP TABLE Fournisseur;
```

2. Comment protéger ses pages web de l'injection SQL ?

g. Utiliser les paramètres SQL

Pour protéger un site Web contre l'injection SQL, vous pouvez utiliser des paramètres SQL.

Les paramètres SQL sont des valeurs ajoutées à une requête SQL au moment de l'exécution, de manière contrôlée.

Exemple ASP.NET

```
txtUserId = getRequestString("UserId");
txtSQL = "SELECT * FROM Users WHERE UserId = @0";
db.Execute(txtSQL,txtUserId);
```

Notez que les paramètres sont représentés dans l'instruction SQL par un marqueur @.

Le moteur SQL vérifie chaque paramètre pour s'assurer qu'il est correct pour sa colonne et qu'il est traité littéralement, et non comme faisant partie du SQL à exécuter.

Un autre exemple

```
txtNam = getRequestString("NomClient");
txtAdd = getRequestString("Adresse");
txtCit = getRequestString("Ville");
txtSQL = "INSERT INTO Client (NomClient,Adresse,Ville) Values(@0,@1,@2)";
db.Execute(txtSQL,txtNam,txtAdd,txtCit);
```

Les exemples suivants montrent comment créer des requêtes paramétrées dans certains langages Web courants.

DÉCLARATION SELECT DANS ASP.NET :

```
txtUserId = getRequestString("UserId");
sql = "SELECT * FROM Customers WHERE CustomerId = @0";
command = new SqlCommand(sql);
command.Parameters.AddWithValue("@0",txtUserId);
command.ExecuteReader();
```

INSÉRER DANS LA DÉCLARATION DANS ASP.NET :

```
txtNam = getRequestString("CustomerName");
txtAdd = getRequestString("Address");
txtCit = getRequestString("City");
txtSQL = "INSERT INTO Customers (CustomerName,Address,City) Values(@0,@1,@2)";
command = new SqlCommand(txtSQL);
command.Parameters.AddWithValue("@0",txtNam);
command.Parameters.AddWithValue("@1",txtAdd);
command.Parameters.AddWithValue("@2",txtCit);
command.ExecuteNonQuery();
```

INSÉRER DANS LA DÉCLARATION EN PHP :

```
$stmt = $dbh->prepare("INSERT INTO Customers (CustomerName,Address,City)
VALUES (:nam, :add, :cit)");
$stmt->bindParam(':nam', $txtNam);
$stmt->bindParam(':add', $txtAdd);
$stmt->bindParam(':cit', $txtCit);
$stmt->execute();
```

II. Gestion des utilisateurs

En dehors des commandes DDL (Data Definition Language) et DML (Data Manipulation Language) qui ont fait l'objet des chapitres précédents, il existe également les commandes DCL (Data Control Language) dans SQL.
Les commandes DCL sont utilisées pour gérer les autorisations et les privilèges dans une base de données. Les principales commandes DCL sont GRANT (accorder des autorisations), REVOKE (révoquer des autorisations) et DENY (refuser des autorisations).

1. Création d'un utilisateur

h. SQL server

Dans SQL Server, un "login" est utilisé pour se connecter au serveur SQL Server lui-même, tandis qu'un "utilisateur" est utilisé pour se connecter à une base de données spécifique sur ce serveur. Un "login" est créé au niveau du serveur et peut être utilisé pour se connecter à n'importe quelle base de données sur ce serveur, tandis qu'un "utilisateur" est créé au niveau de la base de données et est spécifique à cette base de données. Un "login" doit être associé à un "utilisateur" dans une base de données spécifique pour que l'utilisateur puisse accéder à cette base de données.

CREATE LOGIN <Login> WITH PASSWORD = '<Mot de passe>' ;
USE [nom_de_la_base_de_donnees];
CREATE USER <Utilisateur1> FOR LOGIN <Login>;
Remarque :
Le Login et le nom d'utilisateur peuvent être les mêmes
Le même login peut être utilisé pour créer plusieurs utilisateurs différents. Cela peut être utile dans des scénarios où vous avez plusieurs utilisateurs qui doivent avoir des accès différents à la même base de données. Chaque utilisateur peut avoir ses propres autorisations et rôles au sein de la base de données, même s'ils partagent le même login pour se connecter au serveur SQL Server.
Seul le mot de passe est encadré de quotte ou apostrophe.

Pour supprimer un Login :
DROP LOGIN <Nom du login>

S'il y avait des mappages entre cette connexion et un ou plusieurs utilisateurs de base de données (un par base de données), ces utilisateurs sont toujours là. Vous devez les rechercher et les supprimer.

i. Avec MySQL

Syntaxe

CREATE USER 'username'[@'Serveur'] IDENTIFIED BY 'password'

CREATE USER ‘username’[@’%’] IDENTIFIED BY ‘password’

L'option @'Serveur' spécifie que cet utilisateur peut se connecter uniquement depuis le serveur spécifié. Vous pouvez également utiliser '%' pour permettre à l'utilisateur de se connecter depuis n'importe quel hôte. C’est la valeur par défaut
Après avoir créé l'utilisateur, vous devrez accorder les autorisations appropriées à cet utilisateur en utilisant la commande GRANT pour lui permettre d'accéder à certaines bases de données ou tables.
Le nom d’utilisateur et le mot de passe sont encadrés de quotte ou apostrophe.

j. Avec Oracle

Syntaxe

CREATE USER nom_utilisateur IDENTIFIED BY mot_de_passe
DEFAULT TABLESPACE users TEMPORARY TABLESPACE temp QUOTA UNLIMITED ON users;

Le nom d’utilisateur et le mot de passe ne sont pas encadrés de quotte ou apostrophe.

Les options DEFAULT TABLESPACE, TEMPORARY TABLESPACE et QUOTA ne sont pas obligatoires lors de la création d'un utilisateur. Si vous ne les spécifiez pas, Oracle utilisera les paramètres par défaut définis au niveau du système pour ces valeurs.

Cependant, il est généralement recommandé de spécifier au moins le tablespace par défaut pour l'utilisateur afin de s'assurer que les objets créés par cet utilisateur seront stockés dans un emplacement approprié. De même, spécifier un tablespace temporaire par défaut peut être utile pour garantir que les opérations temporaires de l'utilisateur sont correctement gérées.

Quant au quota, si vous ne le spécifiez pas, l'utilisateur aura un quota par défaut défini au niveau du système. Spécifier un quota peut être utile si vous souhaitez limiter la quantité d'espace qu'un utilisateur peut utiliser dans un tablespace spécifique.

2. Modifier le mot de passe d’un utilisateur

k. Avec SQL server

ALTER LOGIN <nom_utilisateur> WITH PASSWORD = '<nouveau mot de passe>'

l. Avec Mysql

SET PASSWORD FOR <'utilisateur'@'Serveur'> = PASSWORD('<nouveau_mot_de_passe>');

m. Avec oracle

ALTER USER <nom_utilisateur> IDENTIFIED BY <nouveau_mot_de_passe>;

3. Supprimer un utilisateur

DROP USER <nom_utilisateur1> [, <nom_utilisateur2>, …]

4. Gestion des rôles ou groupes

Un rôle peut être vu comme un ensemble d'utilisateur ou un acteur en UML. Il est souvent intéressant de grouper les utilisateurs pour faciliter la gestion des droits : de cette façon, les droits peuvent être donnés ou supprimés pour tout un groupe

- ***Pour créer un rôle***

CREATE ROLE <nom_du_rôle>;

- ***Supprimer un rôle***

DROP ROLE <nom_du_rôle>;

- ***Affecter un utilisateur à un rôle***

ALTER ROLE <Nom du rôle> **ADD MEMBER** <Nom utilisateur>;

5. Gestion des privilèges

a. Accorder des privilèges

On distingue deux types de privilège : Les privilèges Système et les privilèges Objet
Les privilèges système sont des permissions liées aux actions. Ils permettent l'exécution d'actions sur des types d'objets. La gestion de ce type de privilège est globale. Chacun de ces privilèges est donné sur tout objet d'un certain type. Exemple CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW, CREATE PROCEDURE, etc.
Un privilège objet va porter sur un objet individuel nommé. Par exemple, Je donne le droit de faire un select dans la table Employé. Par opposition, un privilège système est un privilège qui ne porte pas sur un objet individuel nommé. Autrement dit, tout droit qui n'est pas de type objet, est de type système. Les privilèges, qu'ils soient d'ailleurs de type système ou objet, vont être attribués par la commande GRANT, et vont être révoqués par la commande REVOKE. Les privilèges prennent effet immédiatement dès leur attribution…

Un privilège système est le droit d'exécuter un ordre SQL
Chaque ordre SQL a au moins un privilège système associé qui porte le même nom que l'ordre SQL Par exemple l'ordre CREATE TABLE possède un privilège système associé CREATE TABLE qui donne le droit de créer une table dans son propre schéma

- **Syntaxe pour accorder un privilège système**

GRANT <Liste des privilèges systèmes> To <Utilisateur> [WITH ADMIN OPTION] pour Oracle

GRANT <Liste des privilèges systèmes> To <Utilisateur> [WITH GRANT OPTION]

<Liste des privilèges systèmes> peut être :

- La liste des privilèges systèmes : <Privilège système 1>, …, <Privilège système N>
- Tous les privilèges systèmes : ALL PRIVILEGES

<Utilisateur> peut être :

- La liste des utilisateurs auxquels les privilèges seront donnés : <Utilisateur1>,…,<UtilisateurN>
- Tous les utilisateurs : PUBLIC

WITH ADMIN OPTION : donne à l'utilisateur le droit d'assigner, de retirer, de modifier ou de supprimer les privilèges reçus

- **Syntaxe pour accorder un privilège objet**

GRANT <Liste des privilèges objet> [(<liste des attributs>)]|ALL ON <objet> TO <Utilisateur> [WITH GRANT OPTION];

<Liste des privilèges objet> peut être :

- La liste des privilèges objet : <Privilège objet 1>, …, <Privilège objet N>
- Tous les privilèges objet : ALL

<liste des attributs> : Attributs sur lesquels s'applique le privilege au cas où l'objet est une table et que le privilege ne s'applique pas à l'ensemble des attributs mais seulement à quelques attributs de la table

<Objet> objet sur lequel s'applique le privilège. Une table, une vue, etc.

<Utilisateur> peut être :

- La liste des utilisateurs ou des rôles auxquels les privilèges seront donnés : <Utilisateur1>,…,<UtilisateurN>
- Tous les utilisateurs : PUBLIC

WITH GRANT OPTION permet à l'utilisateur de donner les privilèges objets à d'autres utilisateurs et rôles

Remarque

Dans SQL server, tous les objets que vous créez doivent être associés à un schéma. Si aucun schéma n'est spécifié lors de la création d'un objet, cet objet est automatiquement associé au schéma par défaut nommé dbo qui signifie "database owner" (propriétaire de la base de données)

Si l'utilisateur créé n'a pas de contrôle total sur ce schéma, vous ne pourrez y effectuer aucune opération de création, modification ou suppression d'objet. Si par défaut, l'utilisateur créé n'a pas le contrôle sur le schéma dbo, vous pouvez lui accorder ce contrôle grâce à la commande suivante :

GRANT CONTROL ON SCHEMA::dbo **TO** <nom_utilisateur>;

Cette commande accordera à l'utilisateur spécifié le contrôle total sur le schéma "dbo", lui permettant de créer, modifier et supprimer des objets dans ce schéma.

b. Retirer des privilèges (révoquer ou refuser) : Instructions REVOKE et DENY

Les commandes REVOKE et DENY sont toutes deux utilisées pour retirer les autorisations et les privilèges sur les objets de la base de données, mais elles ont des effets légèrement différents :

REVOKE : utilisée pour révoquer des autorisations précédemment accordées à un utilisateur ou à un rôle. Lorsque vous révoquez une autorisation à l'aide de la commande REVOKE, vous annulez explicitement une autorisation qui avait été accordée précédemment. Par exemple, si vous aviez accordé à un utilisateur le droit de SELECT sur une table, vous pouvez révoquer ce droit à l'aide de la commande REVOKE.

DENY : La commande DENY est utilisée pour refuser explicitement une autorisation à un utilisateur ou à un rôle. Lorsque vous utilisez la commande DENY, vous empêchez un utilisateur ou un rôle d'exécuter une action spécifique sur un objet de la base de données. Par exemple, si vous refusez le droit de SELECT sur une table à un utilisateur, il ne pourra pas sélectionner des données à partir de cette table, même s'il avait précédemment reçu ce droit par le biais d'une autre autorisation.

En résumé, la principale différence entre REVOKE et DENY est que REVOKE annule une autorisation qui avait été accordée précédemment, tandis que DENY empêche explicitement un utilisateur ou un rôle d'exécuter une action sur un objet de la base de données.
La commande DENY empêche de manière permanente un utilisateur ou un rôle d'exécuter une action spécifique sur un objet de la base de données. Lorsque vous utilisez la commande DENY pour refuser une autorisation, cette interdiction reste en place tant qu'elle n'est pas explicitement révoquée à l'aide de la commande REVOKE.
En d'autres termes, une fois qu'une autorisation est refusée à l'aide de la commande DENY, elle reste en vigueur jusqu'à ce qu'elle soit explicitement levée en utilisant la commande REVOKE. Même si l'utilisateur ou le rôle dispose d'autres autorisations qui pourraient potentiellement lui permettre d'exécuter l'action refusée, la commande DENY prévaut et empêche l'exécution de cette action spécifique sur l'objet de la base de données concernée.

- **Syntaxe pour retirer un privilège système**

REVOKE <Liste des privilèges systèmes> TO | FROM <Utilisateur> [CASCADE CONSTRAINTS];

DENY <Liste des privilèges systèmes> TO <Utilisateur> [CASCADE CONSTRAINTS];

<Liste des privilèges systèmes> peut être :

- La liste des privilèges systèmes : <Privilège système 1>, …, <Privilège système N>
- Tous les privilèges systèmes : ALL PRIVILEGES

<Utilisateur> peut être :

- La liste des utilisateurs auxquels les privilèges seront donnés : <Utilisateur1>,…,<UtilisateurN>
- Tous les utilisateurs : PUBLIC

- **Syntaxe pour retirer un privilège objet**

REVOKE <Liste des privilèges objet> ON <objet> TO | FROM <Utilisateur> [CASCADE CONSTRAINTS]

DENY <Liste des privilèges objet> ON <objet> TO <Utilisateur> [CASCADE CONSTRAINTS]

<Liste des privilèges objet> peut être :

- La liste des privilèges objet : <Privilège objet 1>, …, <Privilège objet N>
- Tous les privilèges objet : ALL

<Objet> objet sur lequel le privilège doit être retiré. Une table, une vue, etc.

<Utilisateur> peut être :

- La liste des utilisateurs ou des rôles auxquels les privilèges seront retirés : <Utilisateur1>,…,<UtilisateurN>
- Tous les utilisateurs : PUBLIC

Remarque

Dans SQL Server, voici une liste des principaux privilèges d'objet que vous pouvez accorder à un utilisateur ou à un rôle :

- SELECT : Permet à l'utilisateur de lire les données d'une table ou d'une vue.
- INSERT : Autorise l'utilisateur à ajouter de nouvelles lignes de données dans une table.
- UPDATE : Permet à l'utilisateur de modifier les données existantes dans une table.
- DELETE : Autorise l'utilisateur à supprimer des lignes de données d'une table.
- REFERENCES : Permet à l'utilisateur de créer une contrainte de clé étrangère qui référence une autre table.
- EXECUTE : Autorise l'utilisateur à exécuter une procédure stockée ou une fonction.
- CONTROL : Accorde le contrôle total sur un objet, y compris le droit de modifier ou de supprimer l'objet.

Ces privilèges peuvent être accordés individuellement sur des objets spécifiques tels que des tables, des vues, des procédures stockées, des fonctions, des schémas, etc. en utilisant la commande GRANT dans SQL Server.

III. Sauvegarde d'une base de données

L'instruction BACKUP DATABASE est utilisée dans SQL Server pour créer une sauvegarde complète d'une base de données SQL existante.

Avec MySql, utiliser l'instruction MYSQLDUMP
Avec Oracle, utiliser l'instruction EXPDP

Syntaxe Sql Server

BACKUP DATABASE <Nom de la base de données> **TO DISK** = <Chemin> [**WITH DIFFERENTIAL**];

La clause **WITH DIFFERENTIAL** permet de sauvegarder uniquement les parties de la base de données qui ont été modifiées depuis la dernière sauvegarde complète de la base de données.

Syntaxe MySql

MYSQLDUMP -u <nom_utilisateur> -p <nom_base_de_données> > <nom_fichier_sauvegarde.sql>

- L'option -u permet de spécifier le nom d'utilisateur avec lequel vous souhaitez vous connecter à la base de données.
- L'option -p indique à la commande mysqldump de demander le mot de passe de l'utilisateur MySQL après son exécution.

Syntaxe Oracle

EXPDP ‹nom_user>/<mot_de_passe>@//<adresse_ip_ou_nom_hôte>:<port>/<nom_base_de_données>
DIRECTORY=<chemin_répertoire_sauvegarde> **DUMPFILE**=<nom_fichier_sauvegarde>.dmp

Exemple

L'instruction SQL suivante crée une sauvegarde complète de la base de données existante « testDB » sur le disque D :

BACKUP DATABASE testDB TO DISK = 'D:\backups\testDB.bak';

Astuce : Sauvegardez toujours la base de données sur un lecteur différent de la base de données réelle. Ensuite, si vous rencontrez une panne de disque, vous ne perdrez pas votre fichier de sauvegarde ainsi que la base de données.

Exemple de sauvegarde différentielle

BACKUP DATABASE testDB
TO DISK = 'D:\backups\testDB.bak'
WITH DIFFERENTIAL;

Une sauvegarde différentielle réduit le temps de sauvegarde (puisque seules les modifications sont sauvegardées).

IV. Conclusion sur la sécurité

La sécurité des bases de données est un aspect essentiel de la gestion des données, car les bases de données contiennent souvent des informations sensibles et critiques pour une organisation. Voici quelques bonnes pratiques pour assurer la sécurité des bases de données :

1. Authentification et autorisation : Utiliser des mécanismes d'authentification robustes tels que les mots de passe forts, l'authentification à deux facteurs, etc. De plus, utiliser des rôles et des privilèges appropriés pour limiter l'accès aux données aux utilisateurs autorisés.

2. Chiffrement : Chiffrer les données sensibles stockées dans la base de données, ainsi que les communications entre les applications et la base de données.

3. Mise à jour et patching : Maintenir le système de gestion de base de données (SGBD) à jour en appliquant les correctifs de sécurité et les mises à jour recommandées par le fournisseur.

4. Surveillance et journalisation : Mettre en place des mécanismes de surveillance pour détecter les activités suspectes et les violations de sécurité. Journaliser les événements de sécurité pour permettre une analyse ultérieure en cas d'incident.

5. Contrôle d'accès : Limiter l'accès physique et logique aux serveurs de base de données, aux fichiers de données et aux autres ressources associées.

6. Sauvegarde et reprise après sinistre : Mettre en place des stratégies de sauvegarde régulières et des plans de reprise après sinistre pour assurer la disponibilité et l'intégrité des données en cas d'incident.

7. Test de vulnérabilité : Réaliser régulièrement des tests de vulnérabilité et des audits de sécurité pour identifier et corriger les failles de sécurité potentielles.

En suivant ces bonnes pratiques, les organisations peuvent renforcer la sécurité de leurs bases de données et réduire les risques liés à la perte, à la divulgation ou à la compromission des données sensibles.